Contents

Aprendizaje Automático con Humano en el Bucle: Incomprendido **1**
Estructura del libro . . . 1

Declaración de Originalidad **3**

Resumen **5**

Agradecimientos **7**

¿Qué es el Aprendizaje Automático con Humano en el Bucle? **9**
La Paradoja de la Automatización . . . 9
Definición del Aprendizaje Automático con Humano en el Bucle . . . 9
Una Breve Historia . . . 10
¿Por qué HITL? El caso del juicio humano . . . 11
El Bucle de Retroalimentación . . . 12
Lo que HITL No Es . . . 12
La Economía de la Retroalimentación Humana . . . 13
Descripción General del Manual . . . 13

Una Taxonomía de la Interacción Humano–Máquina **15**
Niveles de Automatización . . . 15
Participación Activa Frente a Pasiva . . . 16
Anotador Único Frente a Múltiples Anotadores . . . 17
Un Marco Unificado . . . 17

Anotación y Etiquetado de Datos **19**
Tipos de Anotación . . . 19
Directrices de Anotación . . . 20
Medición de la Calidad de Anotación: Concordancia entre Anotadores . . . 20
Manejo de Desacuerdos . . . 22
Ruido en las Etiquetas y Sus Efectos . . . 23
Coste y Rendimiento de la Anotación . . . 24

Aprendizaje Activo **25**
La Configuración del Aprendizaje Activo . . . 25
Fundamentos Teóricos . . . 25
Estrategias de Consulta . . . 26
Estimación de Incertidumbre para Modelos Profundos . . . 27
Un Bucle Completo de Aprendizaje Activo . . . 29
El Problema del Arranque en Frío . . . 31
Aprendizaje Activo por Lotes . . . 31

Criterios de Parada . . . 31
Cuándo Funciona el Aprendizaje Activo (y Cuándo No) . . . 32

Aprendizaje Automático Interactivo **33**
Principios del Aprendizaje Automático Interactivo . . . 33
Comparación con el Aprendizaje Activo . . . 33
Interacción de Iniciativa Mixta . . . 34
Factores Humanos en el IML . . . 34
Los Tipos de Retroalimentación en el IML en la Práctica . . . 35
Caso de Estudio: Google Teachable Machine . . . 35
Implementación de un Bucle IML Simple . . . 36
La Prueba de la Abuela . . . 37
IML y Modelos de Fundación . . . 38

Aprendizaje por Refuerzo a partir de Retroalimentación Humana **39**
El Problema de la Alineación . . . 39
La Tubería de RLHF . . . 39
Una Demostración Simplificada de RLHF . . . 41
Desafíos en RLHF . . . 42
IA Constitucional (RLAIF) . . . 43

Aprendizaje a partir de Demostraciones **45**
Clonación de Comportamiento . . . 45
Aprendizaje por Refuerzo Inverso . . . 46
GAIL: Aprendizaje por Imitación Adversarial Generativo . . . 47
Clonación de Comportamiento en PLN: Un Ejemplo Práctico . . . 47
Comparación de los Métodos de Aprendizaje por Imitación . . . 48
Aplicaciones . . . 49

Aprendizaje a partir de Comparaciones y Rankings **51**
Por Qué las Comparaciones Son Mejores que las Valoraciones . . . 51
El Modelo de Bradley-Terry . . . 52
El Modelo de Thurstone . . . 53
Agregación de Rankings . . . 54
Bandidos Duelos . . . 54
Aprendizaje de Preferencias para Modelos de Lenguaje . . . 54
Recopilación de Datos de Preferencia de Alta Calidad . . . 55

HITL en Procesamiento de Lenguaje Natural **57**
El Problema Constitutivo en la Anotación de PLN . . . 57
Anotación de Clasificación de Texto . . . 58
Reconocimiento de Entidades Nombradas . . . 58
Extracción de Relaciones y Anotación Semántica . . . 59
Posedición de Traducción Automática (MTPE) . . . 59
IA Conversacional y Anotación de Diálogo . . . 60
Etiquetado Programático y Supervisión Débil . . . 60
RLHF para Modelos de Lenguaje: La Realidad de la Anotación . . . 61
El Bucle de Retroalimentación Anotación–Modelo . . . 62
Evaluación de Modelos de PLN Generativos . . . 62

HITL en Visión por Computadora **65**
Cómo las Decisiones de Anotación Se Convierten en Sesgos del Modelo . . . 65
Anotación de Clasificación de Imágenes . . . 66

Detección de Objetos: Anotación con Cajas Delimitadoras . . . 66
Segmentación Semántica y de Instancias . . . 68
Aprendizaje Activo para Visión por Computadora . . . 68
Aprendizaje Semisupervisado con Guía Humana . . . 70
Anotación de Vídeo . . . 70

HITL en Salud y Ciencia **73**
Análisis de Imágenes Médicas . . . 73
Anotación Clínica de PLN . . . 74
Consideraciones Regulatorias . . . 75
Anotación de Datos Científicos . . . 76
Gestión de Anotadores Especialistas . . . 77
Una Tubería de Aprendizaje Activo HITL para Imágenes Médicas . . . 78

Plataformas y Herramientas de Anotación **81**
El Panorama de las Plataformas de Anotación . . . 81
Comparación de Características de Plataformas de Anotación . . . 82
Anotación como Código . . . 83

Crowdsourcing y Control de Calidad **85**
Plataformas de Crowdsourcing . . . 85
Diseño de Tareas para el Crowdsourcing . . . 85
Modelos Estadísticos para la Agregación de Etiquetas . . . 87
Redundancia y Estrategia de Agregación . . . 89

Evaluación y Métricas **91**
Métricas Centradas en el Modelo . . . 91
Métricas de Eficiencia de la Anotación . . . 92
Evaluación Humana . . . 93
Medición de la Alineación con la Intención Humana . . . 95
Pruebas A/B en Sistemas Desplegados . . . 95

Equidad, Sesgo y Ética **97**
Fuentes de Sesgo en los Sistemas HITL . . . 97
Equidad en los Sistemas HITL . . . 98
Bienestar de los Anotadores . . . 99
Privacidad en la Anotación . . . 99
Anotación Adversarial e Intoxicación de Datos . . . 100
Ética Institucional . . . 100

Direcciones Futuras **103**
Modelos de Fundación y el Papel Cambiante de la Anotación . . . 103
LLMs como Anotadores . . . 104
Supervisión Débil a Escala . . . 104
Aprendizaje Continuo con Supervisión Humana . . . 105
HITL Multimodal . . . 105
Supervisión Escalable . . . 105
La División del Trabajo en Evolución . . . 106
Problemas de Investigación Abiertos . . . 106

Bibliografía **109**

Caso de Estudio: Limen — Un Humano en el Bucle de Todo **111**

El Problema con el Diseño Convencional de Sistemas Operativos 111
Arquitectura en el Umbral 112
El Diseño de Interacción 113
Limen como Sistema HITL 114
El Punto Más Profundo 115

Sobre el Abandono **117**

Aprendizaje Automático con Humano en el Bucle: Incomprendido

Lazaros Toumanidis

Este manual examina una de las áreas más críticas —e incomprendidas— de la inteligencia artificial moderna: la integración del ser humano en el bucle de aprendizaje. Desde la anotación de datos y el aprendizaje activo hasta el RLHF y el aprendizaje a partir de demostraciones, el juicio humano sigue siendo invaluable.

El libro está dirigido a investigadores, ingenieros y estudiantes que deseen comprender cuándo, por qué y cómo los seres humanos deben participar en el ciclo de aprendizaje.

:::{admonition} Edición en español :class: note Esta es la traducción al español. La edición original en inglés constituye la referencia principal. :::

Estructura del libro

Parte I — Fundamentos — establece el marco conceptual: qué significa el HITL ML y cómo se clasifican los niveles de automatización.

Parte II — Técnicas Fundamentales — aborda los métodos fundamentales: anotación de datos, aprendizaje activo e interactivo.

Parte III — Aprendizaje a partir de Retroalimentación Humana — cubre el RLHF, el aprendizaje a partir de demostraciones y de comparaciones.

Parte IV — Aplicaciones — examina ejemplos prácticos en PLN, visión por computadora y sistemas médicos.

Parte V — Sistemas y Práctica — aborda las herramientas, plataformas y métricas que todo sistema HITL requiere en la práctica.

Parte VI — Ética y Horizontes — trata las cuestiones de equidad, sesgo y el futuro de la relación humano-algoritmo.

Declaración de Originalidad

Presentado en cumplimiento parcial no satisfecho de los requisitos para el grado de Doctor en Filosofía.

Por la presente declaro que el trabajo presentado en este manual es mío, salvo donde no lo es.

Las ideas de estas páginas fueron elaboradas con la asistencia de: investigadores que publicaron artículos y en ocasiones los pusieron detrás de muros de pago; anotadores cuyo trabajo hizo posible los modelos pero cuyos nombres no aparecen en ninguna sección de agradecimientos; contribuidores de código abierto que escribieron las bibliotecas utilizadas en cada ejemplo de código; y el texto acumulado de la civilización humana, ingerido y comprimido por grandes modelos de lenguaje que no tienen opinión alguna sobre nada de esto.

Los errores son míos. Los conocimientos fueron colaborativos.

Confirmo que este trabajo no ha sido presentado para ningún otro grado o calificación profesional, premio académico, beca, expediente de titularidad o publicación en LinkedIn, salvo que se indique lo contrario.

Confirmo asimismo que no se ha incluido en este trabajo ninguna sección de comparación con el Estado del Arte (SOTA). El lector no encontrará una tabla que clasifique 47 métodos en 12 benchmarks, en la que el método propuesto alcanza mejoras estadísticamente significativas en 9 de ellos ($\dagger p < 0.05$). No es porque dicha tabla no pudiera construirse, sino porque su construcción estaría al margen del propósito.

El propósito es este: el humano está en el bucle. Siempre lo ha estado. Este libro intenta comprender qué significa eso.

Firmado: Un Humano en el Bucle

Fecha: Cuando usted esté leyendo esto

Institución: Ninguna. Ese es el punto.

Sobre el subtítulo :class: note

Incomprendido hace referencia al campo, al papel del humano en él, y al lector, todo a la vez. Si cree que se refiere únicamente a usted, puede tener razón. Si cree que se refiere únicamente a las máquinas, está confirmando el punto del subtítulo.

Resumen

Aprendizaje Automático con Humano en el Bucle: Incomprendido

La narrativa dominante del aprendizaje automático sitúa al ser humano como un participante temporal: presente durante la recopilación de datos y el entrenamiento del modelo, y progresivamente eliminado a medida que la automatización mejora. Este manual sostiene que dicha narrativa es errónea, que siempre lo ha sido, y que comprender *por qué* lo es constituye una de las cosas más útiles en la práctica que puede hacer un profesional del aprendizaje automático.

El Aprendizaje Automático con Humano en el Bucle (HITL ML) es la disciplina que hace que la participación humana en los sistemas de aprendizaje sea deliberada, estructurada y continua. En lugar de tratar la retroalimentación humana como un coste a minimizar, HITL la trata como una señal que hay que comprender: de dónde proviene, qué codifica, cómo recopilarla de manera eficiente y cómo incorporarla fielmente en modelos que se comporten de la manera en que sus diseñadores y usuarios realmente pretenden.

Este manual ofrece un tratamiento exhaustivo del campo a lo largo de seis partes. La Parte I establece los fundamentos y una taxonomía de la interacción humano–máquina. La Parte II cubre las técnicas clásicas: anotación y etiquetado de datos, aprendizaje activo y aprendizaje automático interactivo. La Parte III aborda los paradigmas modernos de alineación: el aprendizaje por refuerzo a partir de retroalimentación humana (RLHF), el aprendizaje a partir de demostraciones y el aprendizaje de preferencias. La Parte IV aplica estas técnicas al procesamiento de lenguaje natural, la visión por computadora y la atención sanitaria. La Parte V cubre los sistemas e infraestructuras de HITL a escala: plataformas de anotación, control de calidad del crowdsourcing y metodología de evaluación. La Parte VI examina la ética y el futuro del campo.

Cada capítulo incluye definiciones formales, tratamiento matemático donde corresponde, y ejemplos de código autocontenidos y ejecutables en Python.

El manual está destinado a los profesionales que construyen sistemas de aprendizaje automático, los investigadores que los estudian, los estudiantes que aprenden sobre ellos, y cualquier persona que alguna vez se haya preguntado quién etiquetó los datos que entrenaron el modelo que tomó la decisión que afectó a su vida.

La respuesta, en todos los casos, es: un ser humano. A menudo mal remunerado. Raramente reconocido. Siempre en el bucle.

Palabras clave: aprendizaje automático con humano en el bucle, aprendizaje activo, anotación de datos, aprendizaje por refuerzo a partir de retroalimentación humana, RLHF, aprendizaje de preferencias, crowdsourcing, aprendizaje automático interactivo, alineación de IA, IA responsable

Clasificación temática: Aprendizaje Automático · Interacción Humano-Computadora · Inteligencia Artificial · Ciencia de Datos

Extensión: Suficiente.

Agradecimientos

Todo libro sobre aprendizaje automático con humano en el bucle debería comenzar con los humanos de su propio bucle.

Los anotadores. Este manual trata, en parte, sobre ustedes: sobre su trabajo, su juicio y el hecho de que sus nombres no aparecen en ninguna ficha de datos, reconocimiento en artículos ni comunicado de prensa sobre los modelos que su labor hizo posibles. El campo está comenzando a confrontar esto. Le queda un largo camino por recorrer.

Los contribuidores de código abierto cuyas bibliotecas se utilizan en cada ejemplo de código: la comunidad NumPy (desde 2006), los contribuidores de scikit-learn (desde 2007), el equipo de PyTorch, la comunidad de Hugging Face, y los innumerables mantenedores que corrigen errores los fines de semana sin compensación ni reconocimiento. La computación científica se construye sobre este trabajo invisible.

Los investigadores que publicaron su trabajo abiertamente —en preprints de arXiv, foros de acceso abierto y repositorios de GitHub— haciendo posible que los profesionales aprendan sin acceso institucional. Y los investigadores que revisaron artículos con cuidado y honestidad, una forma de anotación experta que la academia de alguna manera se ha convencido a sí misma de que no es trabajo.

Los docentes que hicieron el conocimiento accesible: los blogueros, los escritores de tutoriales, los que responden en Stack Overflow, los ponentes de conferencias que publicaron sus diapositivas. El aprendizaje no ocurre en aislamiento, y no ocurre sin personas dispuestas a explicarles las cosas a desconocidos.

Los críticos. Los investigadores que señalaron que los conjuntos de datos codifican sesgos, que la demografía de los anotadores condiciona el comportamiento de los modelos, que los humanos en el bucle son con frecuencia los participantes más vulnerables del sistema. Este campo es mejor gracias a su insistencia en plantear preguntas incómodas.

> *Hay cosas que sabemos que sabemos. Hay cosas que sabemos que no sabemos. Hay cosas que no sabemos que no sabemos.*
>
> *Y luego están las cosas que todo el mundo en el campo sabe pero que nunca llegan del todo a los artículos.*
>
> *Este manual es, en parte, un intento de abordar esta última categoría.*

Si usted contribuyó a este campo y no ve su nombre aquí: está reconocido. El bucle le incluye.

¿Qué es el Aprendizaje Automático con Humano en el Bucle?

> *Una máquina que puede aprender de la experiencia… pero solo si le proporcionas las experiencias adecuadas. – parafraseando a Alan Turing*

La Paradoja de la Automatización

Cada nueva oleada de automatización genera nuevas exigencias sobre la atención humana. Cuando las aerolíneas introdujeron el piloto automático, los pilotos se convirtieron en supervisores: ya no responsables del control momento a momento, sino de la tarea más difícil de saber *cuándo* intervenir. Cuando los supermercados introdujeron las cajas de autopago, descubrieron que esos sistemas requieren más supervisión humana por transacción que los cajeros tradicionales, no menos. Y cuando el aprendizaje automático comenzó a tomar decisiones a escala —en medicina, finanzas, contratación laboral y moderación de contenido— generó una demanda enorme y continua de juicio humano.

Esta es la **paradoja de la automatización** {cite}`bainbridge1983ironies`: cuanto más capaz se vuelve un sistema automatizado, más graves son sus fallos y, por tanto, más necesaria resulta una supervisión humana robusta. El aprendizaje automático no es una excepción.

El Aprendizaje Automático con Humano en el Bucle (HITL ML) es el campo que toma esta paradoja en serio y la incorpora al diseño de los sistemas desde el principio. En lugar de tratar la participación humana como un andamiaje temporal destinado a ser eventualmente eliminado, HITL ML trata la interacción humano–máquina como una *característica* —una fuente de señal, corrección y orientación que puede hacer los modelos más precisos, más alineados con los valores humanos y más confiables.

Definición del Aprendizaje Automático con Humano en el Bucle

No existe una definición única acordada de HITL ML, y el término se utiliza de varias maneras superpuestas en la bibliografía. Para este manual adoptamos una definición amplia pero precisa:

:::{admonition} Definición :class: important

El Aprendizaje Automático con Humano en el Bucle hace referencia a cualquier sistema o metodología de aprendizaje automático en el que la retroalimentación humana se incorpora al proceso de aprendizaje de manera *deliberada, estructurada y continua* —no solo en el momento de creación del conjunto de datos, sino a lo largo del entrenamiento, la evaluación y el despliegue.

:::

Esta definición tiene tres cláusulas clave:

Deliberada. HITL no se refiere a la influencia humana accidental (por ejemplo, el hecho de que los datos de entrenamiento fueran originalmente creados por humanos). Se refiere a sistemas diseñados explícitamente para solicitar, incorporar y beneficiarse de la retroalimentación humana.

Estructurada. La retroalimentación tiene una forma definida —una etiqueta, una corrección, un juicio de preferencia, una demostración— y un papel definido en el algoritmo de aprendizaje.

Continua. El bucle de retroalimentación se mantiene a lo largo del tiempo, permitiendo que el sistema mejore a medida que encuentra nuevas situaciones, comete errores y recibe orientación humana.

Esta definición incluye los flujos de anotación clásicos, el aprendizaje activo, el aprendizaje automático interactivo, RLHF y el aprendizaje por imitación. Excluye la recopilación de datos pasiva y el aprendizaje supervisado puramente offline (aunque la línea es difusa, como se discutirá en el Capítulo 2).

Una Breve Historia

Sistemas Expertos e Ingeniería del Conocimiento (décadas de 1960–1980)

Los primeros sistemas de IA eran casi completamente con humano en el bucle: los ingenieros del conocimiento se sentaban con expertos en el dominio durante meses, codificando pacientemente reglas en sistemas expertos como MYCIN y DENDRAL. Cada pieza de conocimiento era proporcionada explícitamente por un humano. La máquina era el ejecutor; el humano era el oráculo.

Estos sistemas funcionaban sorprendentemente bien en dominios estrechos, pero eran frágiles: incapaces de generalizar más allá de sus reglas artesanales y costosos de mantener.

El Giro Estadístico (décadas de 1980–2000)

El cambio hacia el aprendizaje automático estadístico en las décadas de 1980 y 1990 transformó la naturaleza de la participación humana. En lugar de codificar el conocimiento como reglas, los humanos proporcionaban ahora *ejemplos* —conjuntos de datos etiquetados que permitían a los modelos inferir patrones. El papel humano pasó a ser el de anotador: etiquetar documentos, clasificar imágenes, transcribir discurso.

Fue un gran avance, pero creó un nuevo cuello de botella: **los datos etiquetados son costosos**. Los investigadores comenzaron a preguntarse cómo aprovechar al máximo el esfuerzo de etiquetado humano. Esta pregunta dio lugar al **aprendizaje activo**, formalizado por primera vez a principios de la década de 1990 {cite}`cohn1994improving`.

La Era del Aprendizaje Profundo (de la década de 2010 hasta la actualidad)

La revolución del aprendizaje profundo creó un nuevo régimen: modelos con miles de millones de parámetros capaces de aprender funciones extraordinariamente complejas a partir de datos, pero que requieren conjuntos de datos etiquetados de tamaño equivalentemente enorme. ImageNet (14 millones de imágenes etiquetadas) y los proyectos de anotación a gran escala que le siguieron mostraron tanto el poder como el coste de la escala.

Al mismo tiempo, el despliegue del aprendizaje automático a escala expuso nuevos problemas: modelos precisos en promedio pero sistemáticamente erróneos para grupos específicos, que alucinaban hechos con confianza, que optimizaban métricas proxy en lugar de valores humanos. Estos fallos motivaron nuevas formas de participación humana: no solo el etiquetado, sino la *alineación* —el proyecto de hacer que los modelos se comporten de la manera que los humanos realmente desean.

La expresión más clara de este trabajo de HITL centrado en la alineación es el **Aprendizaje por Refuerzo a partir de Retroalimentación Humana (RLHF)** {cite}`christiano2017deep`, que se convirtió en la columna vertebral de sistemas como InstructGPT {cite}`ouyang2022training` y las capacidades de seguimiento de instrucciones de los modelos de lenguaje modernos.

¿Por qué HITL? El caso del juicio humano

¿Qué hace que el juicio humano sea suficientemente valioso como para incorporarlo a los sistemas de aprendizaje automático? Varias propiedades:

1. Sentido Común y Conocimiento del Mundo

Los humanos aportan un vasto conocimiento de fondo a cualquier tarea. Cuando un radiólogo etiqueta una radiografía, recurre a años de formación, a una comprensión de la anatomía y al conocimiento implícito sobre el aspecto de las enfermedades —un conocimiento extraordinariamente difícil de especificar completamente o de adquirir solo a partir de los datos.

2. Anclaje Semántico

Las etiquetas son significativas porque los humanos entienden lo que significan. La clase "gato" en ImageNet hace referencia a un concepto difuso que los humanos reconocen intuitivamente pero que ninguna definición formal captura completamente. Los modelos aprenden la extensión de la etiqueta (qué imágenes se corresponden con ella) pero pueden no aprender el concepto en sí, lo que lleva a fallos en casos límite que cualquier humano manejaría correctamente.

3. Alineación de Valores

Los humanos tienen preferencias, valores y juicios éticos que los modelos de aprendizaje automático no pueden derivar de los datos por sí solos. Si un fragmento de texto es "útil" o "dañino" no es una pregunta puramente empírica: depende de valores humanos que varían entre individuos y contextos. HITL es el mecanismo principal mediante el cual estos valores pueden comunicarse a los sistemas de aprendizaje automático.

4. Adaptabilidad

El juicio humano puede adaptarse a situaciones novedosas sin necesidad de reentrenamiento. Un modelo entrenado con datos históricos puede fallar catastróficamente cuando el mundo cambia; un humano puede reconocer la novedad y responder adecuadamente.

5. Rendición de Cuentas

En dominios de alto riesgo —medicina, derecho, finanzas— las decisiones deben ser responsables ante los seres humanos. Los sistemas HITL hacen que esta rendición de cuentas sea viable al mantener a los humanos en posición de comprender, verificar e invalidar el comportamiento de las máquinas.

El Bucle de Retroalimentación

La estructura central de HITL ML es un bucle de retroalimentación entre un modelo y uno o más humanos:

```
+-----------------------------------------------+
|                                               |
|   El modelo hace predicciones / solicita   |
|   ----------------------------------->       |
|                                   Humano   |
|   El humano proporciona feedback <--------- |
|   ----------------------------------        |
|                                               |
|   El modelo actualiza con el feedback      |
|                                               |
+-----------------------------------------------+
```

La naturaleza de la retroalimentación varía enormemente entre los paradigmas de HITL:

Tipo de retroalimentación	Ejemplo	Paradigma principal
Etiqueta de clase	"Este correo electrónico es spam"	Aprendizaje supervisado
Corrección	"La entidad debería ser ORG, no PER"	Aprendizaje activo/interactivo
Preferencia	"La respuesta A es mejor que la B"	RLHF / clasificación
Demostración	Mostrar al robot cómo agarrar un objeto	Aprendizaje por imitación
Lenguaje natural	"Sé más conciso en tus respuestas"	Ajuste por instrucciones
Señal implícita	El usuario hizo clic / no hizo clic	Retroalimentación implícita

---

Lo que HITL No Es

Conviene ser precisos sobre lo que queda *fuera* de nuestra definición.

HITL no es lo mismo que el despliegue con humano en el bucle (a veces llamado "humano sobre el bucle"), donde los humanos supervisan decisiones automatizadas y pueden invalidarlas, pero no retroalimentan correcciones al entrenamiento. Trataremos esta distinción en el Capítulo 2.

HITL no es solo supervisión débil. Los sistemas de etiquetado programático como Snorkel utilizan funciones de etiquetado (a menudo reglas de autoría humana) para generar etiquetas ruidosas a escala. Esta es una forma de entrada humana estructurada, pero la retroalimentación no es iterativa en el sentido que HITL generalmente implica.

HITL no consiste simplemente en usar datos etiquetados. Todos los modelos de aprendizaje supervisado utilizan datos etiquetados por humanos. HITL se refiere específicamente a sistemas en los que la retroalimentación humana es una parte *activa e iterativa* del proceso de aprendizaje.

La Economía de la Retroalimentación Humana

La retroalimentación humana es valiosa pero costosa. Una anotación de imagen médica puede costar entre decenas y cientos de dólares por imagen cuando la realiza un especialista, dependiendo de la subespecialidad y la complejidad de la tarea {cite}`monarch2021human`. Las etiquetas de trabajadores de crowdsourcing en plataformas como Amazon Mechanical Turk pueden costar entre $0,01 y $0,10 por elemento {cite}`hara2018data` con una calidad mucho menor. El desafío fundamental del HITL ML es **maximizar el valor de cada unidad de retroalimentación humana**.

Esto conduce a una pregunta unificadora que atraviesa la mayor parte de este manual:

:::{admonition} La Pregunta Central del HITL ML :class: tip

Dado un presupuesto fijo de tiempo y atención humana, ¿cómo debemos decidir qué preguntar a los humanos, cuándo preguntarlo y cómo incorporar sus respuestas al entrenamiento del modelo?

:::

La respuesta a esta pregunta depende del dominio, la forma de retroalimentación, el coste de anotación, el riesgo de error y el estado actual del modelo —razón por la cual HITL ML es una disciplina rica y en continua evolución.

Descripción General del Manual

El resto de este manual está estructurado de la siguiente manera. La **Parte II** cubre los tres pilares clásicos de HITL: anotación, aprendizaje activo y aprendizaje automático interactivo. La **Parte III** aborda los paradigmas más recientes del aprendizaje a partir de retroalimentación —RLHF, aprendizaje por imitación y aprendizaje de preferencias— que se han vuelto centrales en la IA moderna. La **Parte IV** examina HITL desde la perspectiva de dominios de aplicación específicos. La **Parte V** adopta una perspectiva de profesional sobre plataformas, crowdsourcing y evaluación. La **Parte VI** aborda la ética y mira hacia el futuro.

> **See also:** Para una visión general del campo orientada al profesional, véase {cite}`monarch2021human`. Para el artículo fundacional sobre aprendizaje activo que inició gran parte del tratamiento formal de HITL, véase {cite}`settles2009active`.

Una Taxonomía de la Interacción Humano–Máquina

Un vocabulario preciso es el requisito previo para un pensamiento claro. El término "humano en el bucle" se utiliza de manera imprecisa en la práctica —a veces para significar que un humano etiqueta los datos de entrenamiento, a veces que un humano puede invalidar la decisión de un modelo, y a veces que un humano está dirigiendo activamente el proceso de aprendizaje en tiempo real. Estas son cosas significativamente diferentes.

Este capítulo traza el paisaje de la interacción humano–máquina en aprendizaje automático, proporcionando el vocabulario conceptual utilizado a lo largo del resto del manual.

Niveles de Automatización

La distinción más fundamental es la intensidad con que el humano participa en la toma de decisiones del sistema. Un marco bien conocido de la teoría de la automatización {cite}`sheridan1992telerobotics` distingue diez niveles, pero para los propósitos del aprendizaje automático tres categorías capturan la variación más importante:

Humano en el Bucle (HITL)

El humano es un *participante activo en el proceso de aprendizaje*. Las decisiones —o al menos las decisiones relevantes— requieren intervención humana antes de ser finalizadas. El sistema no puede operar sin el compromiso humano continuo.

Ejemplos: Un sistema de aprendizaje activo que consulta a un médico antes de agregar un caso a los datos de entrenamiento. Un anotador de datos que etiqueta ejemplos utilizados inmediatamente para actualizar el modelo. Un anotador de RLHF que compara salidas del modelo.

Humano Sobre el Bucle (HOTL)

El sistema opera de manera autónoma pero un humano lo supervisa y puede intervenir. El humano es un *supervisor*, no un participante. La retroalimentación puede o no puede retroalimentarse al entrenamiento.

Ejemplos: Un sistema de moderación de contenido que marca publicaciones automáticamente; un revisor humano muestrea y corrige una fracción de las decisiones. Un piloto automático que vuela el avión pero alerta al piloto sobre anomalías.

Humano al Mando (HIC)

El humano toma todas las decisiones; el sistema proporciona *recomendaciones o información* pero no tiene autonomía. Esta es la forma más débil de despliegue de aprendizaje automático.

Ejemplos: Un sistema de apoyo diagnóstico que muestra al médico la estimación de probabilidad del modelo pero deja la decisión final enteramente al facultativo.

> **¿Qué nivel es el adecuado?** :class: tip
>
> El nivel apropiado de automatización depende del coste de los errores, la fiabilidad del modelo, la experiencia de los humanos disponibles y las restricciones de latencia de la tarea. Estos factores cambian a medida que el modelo madura —la mayoría de los sistemas comienzan en HITL y migran hacia HOTL a medida que crece la confianza.

```
Datos crudos --> Preprocesamiento --> Características --> Entrenamiento --> Evaluación --> De
     ^                 ^                                     ^                 ^
     |                 |                                     |                 |
 Recopilación      Anotación                            Aprendizaje        Pruebas de     Sup
  feedback         y etiquetado                          activo            retroalim.      y
```

Etapa	Papel humano	Capítulo
Recopilación	Decidir qué datos recopilar; estrategia de muestreo	3, 4
Anotación	Asignar etiquetas, estructuras, metadatos	3, 13
Entrenamiento	Consultas de aprendizaje activo; retroalimentación online	4, 5, 6
Evaluación	Evaluación humana de salidas del modelo	14
Despliegue	Supervisión, manejo de excepciones, correcciones	12, 14

Participación Activa Frente a Pasiva

En HITL *activo*, el sistema selecciona qué puntos de datos presentar al humano —formulando preguntas estratégicamente. En HITL *pasivo*, el humano proporciona retroalimentación sobre los datos que llegan (por ejemplo, lotes de etiquetado de datos asignados secuencialmente).

La participación activa es más eficiente porque la retroalimentación se dirige donde más mejorará el modelo. La participación pasiva es más sencilla de implementar y gestionar.

Una distinción relacionada es entre retroalimentación **por lotes** y **en línea**:

- **Por lotes:** Los humanos etiquetan un gran grupo de ejemplos; el modelo se reentrena. Se repite.
- **En línea (streaming):** La retroalimentación humana llega continuamente; el modelo se actualiza de manera incremental.

Los flujos de trabajo por lotes son la norma en la industria (proyectos de anotación seguidos de ciclos de entrenamiento). Los flujos de trabajo en línea son más naturales para sistemas interactivos y entornos de aprendizaje por refuerzo.

Anotador Único Frente a Múltiples Anotadores

La mayoría de las presentaciones formales de HITL asumen un único anotador consistente. En la práctica, la anotación involucra a muchas personas, y sus juicios difieren.

La **agregación** combina múltiples anotaciones en una sola etiqueta, típicamente mediante votación mayoritaria o un modelo estadístico (Capítulo 13).

El desacuerdo como señal —algunos investigadores sostienen que el desacuerdo entre anotadores es información valiosa que no debería comprimirse en una única etiqueta "dorada". Los enfoques perspectivistas del PLN, por ejemplo, preservan múltiples anotaciones como etiquetas suaves que reflejan la genuina ambigüedad de los datos {cite}`uma2021learning`.

Un Marco Unificado

Podemos representar cualquier configuración HITL con una 5-tupla:

$$\text{Configuración HITL} = (\text{nivel}, \text{modalidad}, \text{etapa}, \text{frecuencia}, \text{anotadores})$$

donde:

- **nivel** $\in$ {HITL, HOTL, HIC}
- **modalidad** $\in$ {etiqueta, corrección, demostración, preferencia, lenguaje natural, implícita}
- **etapa** $\in$ {recopilación, anotación, entrenamiento, evaluación, despliegue}
- **frecuencia** $\in$ {lotes, en línea}
- **anotadores** $\in \mathbb{N}^+$ (número de anotadores por elemento)

Esta taxonomía nos permite comparar sistemas HITL diversos en los mismos ejes y razonar sobre los compromisos entre ellos. El resto del manual profundiza en celdas específicas de este espacio.

> **See also:** Para un estudio empírico de cómo las decisiones de anotación afectan el comportamiento del modelo aguas abajo, véase {cite}`bender2021stochastic`. Para una encuesta de sistemas de aprendizaje automático interactivo, véase {cite}`amershi2014power`.

Anotación y Etiquetado de Datos

La anotación de datos es la forma más ubicua de participación humana en el aprendizaje automático. Antes de que un modelo pueda aprender, alguien debe indicarle cuáles son las respuestas correctas —y ese alguien es generalmente un ser humano. Este capítulo cubre la teoría y la práctica de la anotación: qué hace difícil la anotación, cómo diseñar tareas de anotación, cómo medir la calidad y cómo manejar el desacuerdo.

Tipos de Anotación

Las tareas de anotación varían enormemente en su estructura, dificultad y coste. Los principales tipos incluyen:

Clasificación

El anotador asigna cada instancia a una de K categorías predefinidas. Esta es la tarea de anotación cognitivamente más sencilla, pero definir un buen esquema de categorías (una *taxonomía*) puede resultar sorprendentemente difícil.

La **clasificación binaria** (¿es esta imagen un gato?) es el caso más simple. La clasificación **multiclase** (¿qué especie es este animal?) requiere que los anotadores elijan una opción de una lista. La anotación **multietiqueta** (¿qué temas cubre este artículo?) permite múltiples etiquetas simultáneas.

Etiquetado de Secuencias

Cada token de una secuencia recibe una etiqueta. El Reconocimiento de Entidades Nombradas (NER, por sus siglas en inglés) es el ejemplo canónico: los anotadores marcan fragmentos de texto como PERSONA, ORGANIZACIÓN, LUGAR, etc. La anotación se realiza típicamente usando el esquema de etiquetado BIO (Beginning-Inside-Outside, inicio-interior-exterior) o BIOES:

```
B-ORG    O          B-ORG    O    O    O
```

Anotación de Tramos y Relaciones

Más allá del etiquetado de tokens individuales, los anotadores pueden necesitar: - Identificar tramos (expresiones de múltiples tokens) y asignarles tipos - Marcar *relaciones* entre tramos ("Apple" ADQUIRIÓ "Shazam") - Anotar cadenas de correferencia (todas las menciones de la misma entidad)

Estas tareas son cognitivamente exigentes y tienen menor concordancia entre anotadores.

Cajas Delimitadoras y Detección de Objetos

Los anotadores dibujan rectángulos alrededor de objetos en imágenes y asignan una etiqueta de categoría a cada caja. La precisión de la localización importa: una caja demasiado pequeña pierde contexto; una caja demasiado grande incluye el fondo. Las herramientas de anotación modernas calculan la intersección sobre la unión (IoU) con las anotaciones de referencia para señalar problemas de calidad.

Segmentación

Anotación a nivel de píxel: se asigna cada píxel a una clase (segmentación semántica) o a una instancia específica de objeto (segmentación de instancias). La segmentación de alta calidad se encuentra entre los tipos de anotación más costosos, con precios que van desde decenas hasta más de cien dólares por imagen en escenas complejas, dependiendo del dominio y el soporte de herramientas.

Transcripción y Traducción

Audio □ texto (datos de entrenamiento para ASR), escritura manuscrita □ texto (datos para OCR), o lengua origen □ lengua destino (datos para traducción automática). Estas tareas requieren experiencia lingüística y no pueden realizarse de manera fiable por anotadores sin formación.

Directrices de Anotación

El factor determinante más importante de la calidad de la anotación es la calidad de las **directrices de anotación**: las instrucciones escritas que siguen los anotadores.

Unas buenas directrices: - Establecen el objetivo de la tarea y explican *por qué* importa la etiqueta - Proporcionan una definición clara para cada categoría con ejemplos positivos y negativos - Abordan explícitamente los casos límite y los casos difíciles más habituales - Especifican qué hacer ante la incertidumbre (por ejemplo, marcar "omitir" frente a elección forzada) - Incluyen ejemplos elaborados de anotación completa

Las directrices deficientes se apoyan en que los anotadores "usen el sentido común" para los casos límite —lo que lleva a decisiones inconsistentes que degradan la calidad del modelo e inflan el desacuerdo entre anotadores.

> **El desarrollo de las directrices es iterativo** :class: note
>
> No espere escribir directrices perfectas desde el principio. Realice una pequeña ronda de anotación piloto, analice los desacuerdos y actualice las directrices. Repita. Las directrices bien desarrolladas suelen pasar por 3 a 5 ciclos de revisión antes de estabilizarse.

Medición de la Calidad de Anotación: Concordancia entre Anotadores

Cuando múltiples anotadores etiquetan los mismos datos, su concordancia puede medirse. Una alta concordancia sugiere que la tarea está bien definida y que los anotadores la comprendieron. Una baja concordancia sugiere ambigüedad en la tarea, las directrices o los datos en sí.

Kappa de Cohen

Para dos anotadores que etiquetan datos en K categorías, el **kappa de Cohen** {cite}`cohen1960coefficient` corrige la concordancia observada por el azar:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

donde P_o es la concordancia proporcional observada y P_e es la probabilidad de concordancia por azar (calculada a partir de las distribuciones marginales de etiquetas).

$\kappa = 1$ significa concordancia perfecta; $\kappa = 0$ significa concordancia no superior al azar; $\kappa < 0$ significa desacuerdo sistemático.

Rango de κ	Interpretación
< 0	Por debajo del azar
$0{,}0 - 0{,}20$	Leve
$0{,}21 - 0{,}40$	Regular
$0{,}41 - 0{,}60$	Moderada
$0{,}61 - 0{,}80$	Sustancial
$0{,}81 - 1{,}00$	Casi perfecta

Kappa de Fleiss

Extiende el kappa de Cohen a $M > 2$ anotadores. Cada anotador etiqueta de manera independiente cada elemento; la fórmula agrega todos los pares de anotadores:

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

donde $\bar{P}$ es la concordancia pairwise media entre todos los pares de anotadores, y $\bar{P}_e$ es la concordancia esperada bajo asignación aleatoria.

Alfa de Krippendorff

La métrica más general, que admite cualquier número de anotadores, cualquier tipo de escala (nominal, ordinal, de intervalo, de razón) y datos faltantes {cite}`krippendorff2011computing`:

$$\alpha = 1 - \frac{D_o}{D_e}$$

donde D_o es el desacuerdo observado y D_e es el desacuerdo esperado. El alfa de Krippendorff es generalmente preferido en el trabajo académico por su flexibilidad.

```python
import numpy as np
from sklearn.metrics import cohen_kappa_score

# Simulate two annotators labeling 200 items into 3 categories
rng = np.random.default_rng(0)
true_labels = rng.integers(0, 3, size=200)
```

```
# Annotator 1: mostly agrees with ground truth
ann1 = true_labels.copy()
flip_mask = rng.random(200) < 0.15
ann1[flip_mask] = rng.integers(0, 3, size=flip_mask.sum())

# Annotator 2: less consistent
ann2 = true_labels.copy()
flip_mask2 = rng.random(200) < 0.30
ann2[flip_mask2] = rng.integers(0, 3, size=flip_mask2.sum())

kappa_12 = cohen_kappa_score(ann1, ann2)
kappa_1true = cohen_kappa_score(ann1, true_labels)
kappa_2true = cohen_kappa_score(ann2, true_labels)

print(f"Cohen's κ (ann1 vs ann2):   {kappa_12:.3f}")
print(f"Cohen's κ (ann1 vs truth):  {kappa_1true:.3f}")
print(f"Cohen's κ (ann2 vs truth):  {kappa_2true:.3f}")
```

Manejo de Desacuerdos

Cuando los anotadores no coinciden, existen varias estrategias:

Votación Mayoritaria

Se toma como estándar de referencia la etiqueta más común. Sencilla y robusta cuando el número de anotadores por elemento es impar. Falla cuando un subgrupo minoritario de anotadores es sistemáticamente más preciso.

Votación Ponderada

Los anotadores se ponderan según su precisión estimada (derivada de la concordancia con un estándar de referencia u otros anotadores). Los anotadores más precisos tienen mayor influencia.

Etiquetas Suaves

En lugar de colapsar las anotaciones a una sola etiqueta, se preserva la distribución. Si 3 de 5 anotadores dijeron "positivo" y 2 dijeron "neutro", esto se representa como $(p_{\text{pos}}, p_{\text{neutro}}, p_{\text{neg}}) = (0{,}6, 0{,}4, 0{,}0)$. Entrenar con etiquetas suaves mejora la calibración.

Arbitraje

Un anotador senior o experto en el dominio dirime los desacuerdos. Es el estándar de oro, pero costoso; generalmente reservado para dominios de alto riesgo.

Modelos Estadísticos

Los enfoques más sofisticados modelan probabilísticamente la competencia del anotador. El modelo **Dawid-Skene** {cite}`dawid1979maximum` estima simultáneamente las matrices de confusión de los anotadores y las etiquetas verdaderas de los elementos mediante el algoritmo EM. Véase el Capítulo 13 para más detalles.

Ruido en las Etiquetas y Sus Efectos

La anotación real es ruidosa. Los efectos del ruido en las etiquetas sobre el entrenamiento del modelo dependen del tipo de ruido:

- El **ruido aleatorio** (etiquetas invertidas aleatoriamente) degrada el rendimiento, pero los modelos son sorprendentemente robustos frente a niveles moderados (hasta ~20% para muchas tareas).
- El **ruido sistemático/adversarial** (etiquetas consistentemente erróneas en patrones específicos) es mucho más dañino y más difícil de detectar.
- El **ruido condicional por clase** (errores más probables para ciertas clases) sesga la frontera de decisión del modelo.

Una regla práctica: con n ejemplos de entrenamiento y una fracción ϵ de etiquetas corrompidas, el rendimiento del modelo se degrada aproximadamente como si se tuvieran $(1 - 2\epsilon)^2 n$ ejemplos limpios {cite}`natarajan2013learning`. Para $\epsilon = 0{,}2$, esto equivale a perder el 36% de los datos.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=2000, n_features=20, random_state=42)

noise_levels = np.linspace(0, 0.45, 15)
mean_accs = []

for eps in noise_levels:
    y_noisy = y.copy()
    flip = rng.random(len(y)) < eps
    y_noisy[flip] = 1 - y_noisy[flip]
    scores = cross_val_score(LogisticRegression(max_iter=500), X, y_noisy, cv=5)
    mean_accs.append(scores.mean())

plt.figure(figsize=(7, 4))
plt.plot(noise_levels, mean_accs, 'o-', color='#2b3a8f', linewidth=2)
plt.xlabel("Label noise rate (ε)", fontsize=12)
plt.ylabel("Cross-validated accuracy", fontsize=12)
plt.title("Effect of Label Noise on Model Performance", fontsize=13)
plt.axvline(0.2, color='#e05c5c', linestyle='--', alpha=0.7, label='20% noise')
plt.legend()
plt.tight_layout()
plt.savefig('label_noise_effect.png', dpi=150)
plt.show()
print(f"\nAccuracy at 0% noise:  {mean_accs[0]:.3f}")
print(f"Accuracy at 20% noise: {mean_accs[round(0.2 / 0.45 * 14)]:.3f}")
print(f"Accuracy at 40% noise: {mean_accs[-2]:.3f}")
```

Coste y Rendimiento de la Anotación

Comprender la economía de la anotación es esencial para la planificación de proyectos.

Tipo de tarea	Rendimiento típico	Coste por elemento (especialista)
Clasificación binaria de imágenes	200–500 por hora	\$0,02–\$0,10
NER (texto corto)	50–150 elementos/h	\$0,10–\$0,50
Extracción de relaciones	20–60 elementos/h	\$0,30–\$1,50
Segmentación de imágenes médicas	5–30 elementos/h	\$10–\$100
Anotación de vídeo	5–20 min de vídeo/h	\$20–\$200

Estas cifras son estimaciones aproximadas de orden de magnitud y varían enormemente según la experiencia en el dominio requerida, la calidad de la herramienta de anotación, la claridad de las directrices y la experiencia del anotador. Deben tratarse como ilustrativas, no como prescriptivas.

> **See also:** Las opciones de herramientas de anotación se tratan en el Capítulo 12. Los modelos estadísticos para la anotación mediante crowdsourcing (Dawid-Skene, MACE) se tratan en el Capítulo 13.

Aprendizaje Activo

Los datos etiquetados son costosos. La idea central del aprendizaje activo es que *no todos los ejemplos no etiquetados son igualmente informativos*: un modelo puede mejorar más rápido si puede elegir sobre qué ejemplos preguntar. En lugar de etiquetar datos aleatoriamente, un sistema de aprendizaje activo consulta a un oráculo (generalmente un anotador humano) sobre los ejemplos más propensos a mejorar el modelo.

Este capítulo cubre la teoría y la práctica del aprendizaje activo: estrategias de consulta, marcos de muestreo, criterios de parada y consideraciones prácticas para despliegues reales.

La Configuración del Aprendizaje Activo

La configuración estándar de **aprendizaje activo basado en agrupamiento** involucra:

- Un **conjunto etiquetado** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^n$ — inicialmente pequeño
- Un **grupo no etiquetado** $\mathcal{U} = \{x_j\}_{j=1}^m$ — típicamente mucho más grande que $\mathcal{L}$
- Un **oráculo** $\mathcal{O}$ que puede devolver $y = \mathcal{O}(x)$ para cualquier x consultado
- Una **estrategia de consulta** ϕ que selecciona la siguiente consulta $x^* = \phi(\mathcal{L}, \mathcal{U}, f_\theta)$

El bucle de aprendizaje activo:

```
1. Inicializar: L = conjunto semilla etiquetado pequeño, U = grupo no etiquetado
2. Entrenar: f_θ ← entrenar(L)
3. Consultar: x∗ = argmax φ(x; f_θ) sobre x ∈ U
4. Etiquetar: y∗ = O(x∗)
5. Actualizar: L ← L ∪ {(x∗, y∗)}, U ← U \ {x∗}
→ Repetir desde 2 hasta agotar el presupuesto
```

El objetivo es alcanzar la calidad objetivo del modelo usando el menor número posible de consultas al oráculo.

Fundamentos Teóricos

Una pregunta natural es: ¿cuánto puede ayudar el aprendizaje activo? En el mejor caso, el aprendizaje activo puede lograr reducciones *exponenciales* en la complejidad de etiquetado —alcanzando un error ϵ con $O(\log(1/\epsilon))$ etiquetas en lugar de las $O(1/\epsilon)$ necesarias con el aprendizaje pasivo, al menos en entornos realizables con una buena estrategia de consulta {cite}`settles2009active`.

En la práctica, las garantías son más difíciles de obtener. El **aprendizaje activo agnóstico** {cite}`balcan2006agnostic` muestra que el ahorro de etiquetas es posible incluso cuando el concepto objetivo no está en la clase de hipótesis, pero los ahorros dependen fuertemente del coeficiente de desacuerdo —una medida de con qué rapidez se reduce el conjunto de hipótesis plausibles a medida que se acumulan los datos.

La implicación práctica clave: la ventaja del aprendizaje activo es mayor cuando la **frontera de decisión es simple y concentrada** (de modo que las consultas de incertidumbre eliminan rápidamente las hipótesis erróneas), y menor cuando la clase de hipótesis es grande o la frontera es compleja.

Estrategias de Consulta

Muestreo por Incertidumbre

La estrategia más sencilla y más ampliamente utilizada: consultar el ejemplo sobre el que el modelo tiene mayor *incertidumbre* {cite}`lewis1994sequential`.

La **menor confianza** consulta el ejemplo para el que el modelo es menos seguro en su predicción principal:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(1 - P_\theta(\hat{y} \mid x)\right)$$

El **muestreo por margen** considera la diferencia entre las dos probabilidades predichas más altas:

$$x^* = \arg\min_{x \in \mathcal{U}} \left(P_\theta(\hat{y}_1 \mid x) - P_\theta(\hat{y}_2 \mid x)\right)$$

El **muestreo por entropía** utiliza la distribución predicha completa:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(-\sum_{k=1}^{K} P_\theta(y_k \mid x) \log P_\theta(y_k \mid x) \right)$$

El muestreo por entropía es el más fundamentado de los tres —considera todas las clases— y generalmente supera a los otros en problemas multiclase.

Consulta por Comité (QbC)

Se entrena un **comité** de C modelos (usando bagging, diferentes inicializaciones o diferentes arquitecturas). Se consulta el ejemplo sobre el que el comité más discrepa:

$$x^* = \arg\max_{x \in \mathcal{U}} \ \text{desacuerdo}(\{f_c(x)\}_{c=1}^{C})$$

El desacuerdo puede medirse como **entropía de votos** (entropía sobre los votos mayoritarios del comité) o **divergencia KL** respecto a la distribución de consenso.

QbC proporciona mejores estimaciones de incertidumbre que un solo modelo, pero requiere entrenar múltiples modelos, lo que es costoso computacionalmente.

Cambio Esperado del Modelo

Se consulta el ejemplo que causaría el mayor cambio al modelo actual si se etiquetara. Para modelos basados en gradientes, esto corresponde al ejemplo con la mayor magnitud esperada del gradiente {cite}`settles2008analysis`:

$$x^* = \arg\max_{x \in \mathcal{U}} \sum_{y \in \mathcal{Y}} P_\theta(y \mid x) \, \|\nabla_\theta \mathcal{L}(f_\theta(x), y)\|$$

Esta estrategia tiene una fuerte motivación teórica, pero requiere calcular gradientes para cada candidato, lo que la hace costosa para modelos grandes.

Enfoques de Núcleo / Geométricos

Las estrategias basadas en incertidumbre pueden estar **sesgadas hacia los valores atípicos**: un ejemplo inusual puede ser muy incierto pero no representativo de la distribución de datos. Los métodos de núcleo abordan esto buscando una muestra diversa que cubra el espacio de características.

El algoritmo **k-centro codicioso** {cite}`sener2018active` encuentra el conjunto más pequeño de puntos tal que cada punto no etiquetado esté a distancia δ de al menos un punto consultado:

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} d(x, x')$$

es decir, consultar el punto más alejado de cualquier punto actualmente etiquetado. Esto favorece un conjunto de anotaciones bien distribuido.

BADGE

Aprendizaje Activo por Lotes mediante Incrustaciones de Gradiente Diversas (BADGE) {cite}`ash2020deep` combina incertidumbre y diversidad: selecciona un lote de ejemplos cuyas incrustaciones de gradiente (con respecto a la etiqueta predicha) son a la vez grandes en magnitud (inciertos) y diversas (cubriendo diferentes regiones del espacio de gradiente). Esta es una de las estrategias modernas más competitivas.

Estimación de Incertidumbre para Modelos Profundos

Las estrategias anteriores suponen el acceso a salidas de probabilidad calibradas del modelo. Para modelos simples (regresión logística, clasificadores softmax), esto es sencillo. Para redes profundas, obtener estimaciones de incertidumbre fiables requiere técnica adicional.

Dos Tipos de Incertidumbre

Siguiendo a Kendall y Gal {cite}`kendall2017uncertainties`, distinguimos:

Incertidumbre aleatórica (incertidumbre de los datos): ruido inherente en las observaciones que no puede reducirse recopilando más datos. Una imagen borrosa es aleatoricamente incierta —ninguna cantidad de datos de entrenamiento adicionales de la misma distribución hará al modelo más seguro al respecto.

Incertidumbre epistémica (incertidumbre del modelo): incertidumbre debida a datos de entrenamiento limitados o a un modelo que no ha visto ejemplos similares. La incertidumbre epistémica *puede* reducirse etiquetando más datos —y es, por tanto, la cantidad relevante para la selección de consultas en aprendizaje activo.

Para el aprendizaje activo, queremos consultar ejemplos con alta incertidumbre epistémica, no alta incertidumbre aleatórica. Consultar un ejemplo fundamentalmente ambiguo desperdicia el esfuerzo del oráculo: ninguna etiqueta que proporcionen será claramente correcta.

Dropout de Monte Carlo

Un enfoque práctico para la estimación de incertidumbre epistémica en redes neuronales es el **Dropout MC** {cite}`gal2016dropout`: aplicar dropout en el momento de la inferencia y ejecutar T pasadas hacia adelante. La varianza entre predicciones es una estimación de la incertidumbre epistémica.

```
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(42)
rng = np.random.default_rng(42)

class MCDropoutNet(nn.Module):
    def __init__(self, input_dim=20, hidden=64, output_dim=2, p_drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, hidden),    nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, output_dim)
        )

    def forward(self, x):
        return self.net(x)

def mc_uncertainty(model, x, T=30):
    """
    Run T stochastic forward passes with dropout active.
    Returns mean prediction and epistemic uncertainty (predictive variance).
    """
    model.train()  # keep dropout active during inference
    with torch.no_grad():
        preds = torch.stack([
            torch.softmax(model(x), dim=-1) for _ in range(T)
        ])  # shape: (T, N, C)
    mean_pred = preds.mean(0)
    # Epistemic uncertainty: mean of variances across passes
    epistemic = preds.var(0).sum(-1)
    # Aleatoric uncertainty: entropy of mean prediction
    aleatoric = -(mean_pred * (mean_pred + 1e-9).log()).sum(-1)
    return mean_pred, epistemic, aleatoric

# Quick demonstration
```

```
model = MCDropoutNet(input_dim=20, output_dim=2)
# In-distribution example (simulated)
x_familiar   = torch.randn(1, 20) * 0.5
# Out-of-distribution example (far from training distribution)
x_unfamiliar = torch.randn(1, 20) * 3.0

for name, x in [("In-distribution ", x_familiar), ("Out-of-distribution", x_unfamiliar)]:
    _, ep, al = mc_uncertainty(model, x)
    print(f"{name} | epistemic: {ep.item():.4f} | aleatoric: {al.item():.4f}")
```

En la red no entrenada anterior, ambos ejemplos muestran una incertidumbre similar. Tras el entrenamiento, el ejemplo fuera de la distribución mostrará mayor incertidumbre epistémica —el modelo no ha aprendido un mapeo fiable para entradas lejanas a la distribución de entrenamiento.

Ensambles Profundos

Entrenar M modelos inicializados independientemente y promediar sus predicciones proporciona una estimación de incertidumbre más sencilla y a menudo más fiable que el Dropout MC {cite}`lakshminarayanan2017simple`. El desacuerdo entre los miembros del ensamble es la señal de incertidumbre epistémica.

Para el aprendizaje activo a escala, tanto el Dropout MC como los ensambles profundos añaden una sobrecarga proporcional a T o M pasadas hacia adelante. En la práctica, $T = 10$–30 para Dropout MC o $M = 5$ miembros del ensamble suele ser suficiente para clasificar ejemplos por incertidumbre epistémica, incluso si los valores absolutos no están bien calibrados.

Un Bucle Completo de Aprendizaje Activo

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from copy import deepcopy

rng = np.random.default_rng(42)

# Generate dataset
X, y = make_classification(
    n_samples=2000, n_features=20, n_informative=10,
    n_classes=3, n_clusters_per_class=1, random_state=42
)
X_train, y_train = X[:1500], y[:1500]
X_test,  y_test  = X[1500:], y[1500:]

def entropy_query(model, X_pool):
    """Return index of most uncertain sample (entropy)."""
    probs = model.predict_proba(X_pool)
    ent = -np.sum(probs * np.log(probs + 1e-9), axis=1)
    return np.argmax(ent)
```

```
def random_query(X_pool):
    """Random baseline."""
    return rng.integers(0, len(X_pool))

def run_active_learning(strategy='entropy', n_initial=30, n_queries=120, query_batch=5):
    labeled_idx = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled_idx = [i for i in range(len(X_train)) if i not in labeled_idx]
    accs = []

    for step in range(n_queries // query_batch):
        model = LogisticRegression(max_iter=500, C=1.0)
        model.fit(X_train[labeled_idx], y_train[labeled_idx])
        accs.append(accuracy_score(y_test, model.predict(X_test)))

        # Query
        X_pool = X_train[unlabeled_idx]
        for _ in range(query_batch):
            if strategy == 'entropy':
                q = entropy_query(model, X_pool)
            else:
                q = random_query(X_pool)
            labeled_idx.append(unlabeled_idx.pop(q))
            X_pool = X_train[unlabeled_idx]

    return np.array(accs)

labels_used = np.arange(1, 25) * 5 + 30  # label counts at each step

accs_active = run_active_learning(strategy='entropy')
accs_random = run_active_learning(strategy='random')

plt.figure(figsize=(8, 5))
plt.plot(labels_used, accs_active, 'o-', label='Entropy sampling', color='#2b3a8f', linewidth
plt.plot(labels_used, accs_random, 's--', label='Random baseline',  color='#e05c5c', linewidt
plt.xlabel("Number of labeled examples", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning vs. Random Sampling", fontsize=13)
plt.legend(fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('active_learning_curve.png', dpi=150)
plt.show()

print(f"Active learning reaches {accs_active[-1]:.3f} accuracy")
print(f"Random sampling reaches {accs_random[-1]:.3f} accuracy")
print(f"Active learning saves ~{int((accs_random.tolist().index(min(accs_random, key=lambda a
```

El Problema del Arranque en Frío

El aprendizaje activo requiere un modelo entrenado para puntuar los puntos no etiquetados —pero al principio, no se tienen (o se tienen muy pocos) ejemplos etiquetados. Este es el **problema del arranque en frío**.

Soluciones prácticas:

1. **Inicialización aleatoria:** Etiquetar un pequeño conjunto semilla aleatorio (20–100 ejemplos) antes de comenzar el aprendizaje activo.
2. **Inicialización basada en agrupamiento:** Usar k-means sobre el grupo no etiquetado; etiquetar un ejemplo de cada clúster. Esto garantiza diversidad en el conjunto etiquetado inicial.
3. **Selección basada en incrustaciones:** Usar un codificador preentrenado para incrustar los ejemplos; seleccionar un subconjunto diverso mediante el enfoque de núcleo.

Para la mayoría de las tareas, unas pocas docenas de etiquetas semilla aleatorias suelen ser suficientes para iniciar el aprendizaje activo; el número exacto depende del equilibrio de clases, la dimensionalidad de las características y la complejidad del modelo.

Aprendizaje Activo por Lotes

En la práctica, los anotadores trabajan por lotes —es ineficiente entrenar y desplegar un nuevo modelo después de cada etiqueta individual. El **aprendizaje activo por lotes** selecciona un conjunto de b ejemplos para etiquetar simultáneamente.

Seleccionar ingenuamente los b ejemplos más inciertos genera **redundancia**: los ejemplos muy inciertos tienden a agruparse (por ejemplo, ejemplos cerca de la frontera de decisión en la misma región). Las mejores estrategias por lotes optimizan tanto la incertidumbre *como* la diversidad dentro del lote.

Los **Procesos de Punto Determinantal (DPPs)** proporcionan una manera fundamentada de muestrear lotes diversos: definen una distribución sobre subconjuntos que penaliza los elementos similares. La probabilidad de un subconjunto S bajo un DPP es proporcional a $\det(L_S)$, donde L es una matriz de núcleo que codifica la similitud.

Criterios de Parada

¿Cuándo debe detenerse el aprendizaje activo? Criterios comunes:

- **Presupuesto agotado:** El más simple —detenerse cuando se agota el presupuesto de anotación.
- **Meseta de rendimiento:** Detenerse cuando la exactitud del modelo en un conjunto de validación retenido no ha mejorado más de δ durante k rondas consecutivas.
- **Umbral de confianza:** Detenerse cuando menos de cierta fracción de los ejemplos no etiquetados tienen una incertidumbre por encima de un umbral.
- **Reducción máxima de pérdida:** Estimar la ganancia máxima posible de etiquetas adicionales; detenerse cuando esta caiga por debajo de un umbral {cite}`bloodgood2009method`.

Cuándo Funciona el Aprendizaje Activo (y Cuándo No)

El aprendizaje activo tiende a funcionar bien cuando: - El etiquetado es costoso y el grupo no etiquetado es grande - Los datos tienen una estructura clara que el modelo puede aprovechar para identificar ejemplos informativos - La clase del modelo es apropiada para la tarea

El aprendizaje activo funciona mal cuando: - El modelo inicial es muy deficiente (arranque en frío) y no puede clasificar ejemplos de manera significativa - La estrategia de consulta selecciona valores atípicos o ejemplos mal etiquetados (importa la robustez al ruido) - La distribución de datos cambia entre el grupo no etiquetado y la distribución de prueba

Una preocupación práctica clave es el **desajuste de distribución**: el aprendizaje activo tiende a consultar ejemplos cerca de la frontera de decisión, creando un conjunto etiquetado sesgado que puede no representar bien la distribución de prueba. Esto puede conducir a fronteras de decisión bien entrenadas pero con pobre calibración.

> **See also:** La encuesta fundacional es {cite}`settles2009active`. Fundamentos teóricos (complejidad de etiquetado, límites agnósticos): {cite}`balcan2006agnostic`. Para el aprendizaje activo específico del aprendizaje profundo, véase {cite}`ash2020deep` (BADGE) y {cite}`sener2018active` (núcleo). Para una evaluación crítica de cuándo el aprendizaje activo realmente ayuda, véase {cite}`lowell2019practical`. Sobre la incertidumbre aleatórica frente a la epistémica en modelos profundos, véase {cite}`kendall2017uncertainties`; para los ensambles profundos como estimadores de incertidumbre, véase {cite}`lakshminarayanan2017simple`; para el Dropout MC como inferencia bayesiana aproximada, véase {cite}`gal2016dropout`.

Aprendizaje Automático Interactivo

El aprendizaje activo hace una pregunta concreta: dado un presupuesto, ¿qué ejemplos debo etiquetar? El Aprendizaje Automático Interactivo (IML, por sus siglas en inglés) hace una pregunta más amplia: ¿cómo podemos diseñar la *interacción completa* entre un ser humano y un sistema de aprendizaje para que sea máximamente productiva, agradable y correcta?

El IML se distingue por la **inmediatez** y la **directness** del bucle de retroalimentación humano–modelo. Donde el aprendizaje automático tradicional implica que un humano entregue datos y espere a que el entrenamiento se complete, el IML permite a los humanos observar el comportamiento del modelo, proporcionar retroalimentación y ver cómo el modelo responde, a menudo en cuestión de segundos.

Principios del Aprendizaje Automático Interactivo

Amershi et al. {cite}`amershi2014power` identifican tres características definitorias del IML:

1. Retroalimentación Rápida: El modelo se actualiza con suficiente rapidez como para que los humanos perciban el efecto de su retroalimentación. En el límite, las actualizaciones del modelo ocurren en tiempo real.

2. Manipulación Directa: El humano interactúa con el modelo a través de los datos o a través de las predicciones del modelo —no a través de archivos de configuración o ajuste de hiperparámetros.

3. Refinamiento Iterativo: El proceso es genuinamente iterativo: la siguiente acción del humano está informada por el comportamiento actual del modelo, que fue moldeado por las acciones anteriores del humano.

Esto crea un estrecho **bucle de co-adaptación**: tanto el humano como el modelo cambian con el tiempo en respuesta el uno al otro. El humano aprende lo que el modelo comprende; el modelo aprende lo que le importa al humano.

Comparación con el Aprendizaje Activo

El IML y el aprendizaje activo se superponen considerablemente pero no son idénticos:

Propiedad	Aprendizaje Activo	Aprendizaje Automático Interactivo
Pregunta principal	¿Qué etiquetar?	¿Cómo interactuar?

Propiedad	Aprendizaje Activo	Aprendizaje Automático Interactivo
Latencia de retroalimentación	Puede ser por lotes (días)	Típicamente en tiempo real o casi
Frecuencia de actualización	Por ronda (lotes)	Por interacción (en línea)
Agencia del humano	Responde las preguntas del modelo	Puede enseñar proactivamente al modelo
Diseño de interfaz	Preocupación secundaria	Preocupación central
Carga cognitiva del humano	No se modela explícitamente	Se considera explícitamente

En el aprendizaje activo, la máquina dirige la interacción. En el IML, el humano también puede tomar la iniciativa —aportando ejemplos, correcciones o retroalimentación sobre cualquier aspecto del comportamiento del modelo que parezca más problemático.

Interacción de Iniciativa Mixta

Los sistemas de **iniciativa mixta** permiten tanto al humano como a la máquina tomar la iniciativa en diferentes momentos {cite}`allen1999mixed`. Un sistema dirigido puramente por la máquina hace preguntas y el humano responde. Un sistema dirigido puramente por el humano deja que el humano decida todo. Los sistemas de iniciativa mixta equilibran ambos.

En la práctica, los mejores sistemas IML combinan: - **Iniciativa de la máquina:** "No estoy seguro sobre estos ejemplos —¿puede etiquetarlos?" - **Iniciativa del humano:** "Noto que el modelo se equivoca constantemente con esta categoría —déjame proporcionar más ejemplos" - **Confirmación:** El modelo muestra su comprensión actual; el humano la confirma o corrige

Las buenas interfaces IML hacen visible y corregible la comprensión actual del modelo. Este es el requisito de **inteligibilidad**: los humanos solo pueden guiar a un modelo que comprenden, al menos aproximadamente.

Factores Humanos en el IML

El IML introduce los factores humanos —carga cognitiva, fatiga, coherencia y confianza— directamente en el bucle de aprendizaje. Un diseño deficiente de IML lleva a:

Fatiga de anotación: Los humanos toman decisiones más rápidas y descuidadas a medida que avanzan las sesiones. Errores que se incorporan a los datos de entrenamiento.

Sesgo de anclaje: Los humanos se apoyan excesivamente en las sugerencias actuales del modelo. Si una interfaz rellena previamente la predicción del modelo, los anotadores son menos propensos a corregirla incluso cuando es errónea —una fuente sistemática de ruido en las etiquetas que se compone a lo largo de las rondas de anotación {cite}`geva2019annotator`. La pre-anotación puede acelerar el rendimiento {cite}`lingren2014evaluating` mientras reduce simultáneamente la tasa a la que los anotadores detectan errores del modelo; estos dos efectos deben sopesarse entre sí en el diseño de interfaces IML.

Mala calibración de la confianza: Los humanos o bien confían demasiado (aceptando salidas del modelo incorrectas) o bien confían demasiado poco (ignorando sugerencias correctas). Ambos patrones reducen el valor de la colaboración humano–modelo.

Coherencia de sesión: Los humanos pueden tomar decisiones diferentes sobre el mismo ejemplo en momentos diferentes, especialmente después de sesiones largas. Las comprobaciones de coherencia (re-presentar ejemplos anteriores) pueden detectar y corregir esto.

Un buen diseño de IML mitiga estos problemas a través de opciones de interfaz: mostrar explícitamente la confianza del modelo, aleatorizar el orden de presentación, limitar la duración de las sesiones e incorporar comprobaciones de coherencia.

Los Tipos de Retroalimentación en el IML en la Práctica

Retroalimentación a Nivel de Ejemplo

El humano proporciona una etiqueta o corrección para un ejemplo específico. Esta es la forma más común y es directamente compatible con el aprendizaje supervisado.

Retroalimentación a Nivel de Característica

El humano indica qué características son relevantes o irrelevantes. "El modelo debería prestar atención a las palabras 'urgente' y 'plazo' para esta categoría." Esto es más expresivo que la retroalimentación a nivel de ejemplo y puede ser más eficiente para ciertas tareas.

TFIDF Interactivo y sistemas similares permiten a los anotadores resaltar palabras relevantes en documentos de texto. Estos resaltados se convierten en restricciones o supervisión adicional sobre la atención del modelo.

Retroalimentación a Nivel de Modelo

El humano corrige directamente el comportamiento del modelo en una clase de entradas: "Siempre que la entrada contenga [X], la salida debería ser [Y]." Esto se corresponde con reglas lógicas o restricciones en enfoques como la Regularización Posterior {cite}`ganchev2010posterior` o el aprendizaje guiado por restricciones.

Caso de Estudio: Google Teachable Machine

Teachable Machine es un sistema IML accesible basado en la web que permite a usuarios no técnicos entrenar clasificadores de imágenes en el navegador. El usuario:

1. Graba ejemplos de cada clase usando su cámara web
2. Entrena el modelo con un solo clic (ajustando una MobileNet en el navegador)
3. Ve inmediatamente las predicciones del modelo sobre vídeo en directo
4. Agrega más ejemplos para las clases donde el modelo se equivoca

Esto ilustra el bucle central del IML: proporcionar ejemplos □ observar el modelo □ identificar el fallo □ proporcionar más ejemplos específicos. La retroalimentación en tiempo real (las salidas del modelo se actualizan en tiempo real, típicamente a velocidades de fotogramas interactivas en hardware moderno) hace que el bucle de co-adaptación sea visceralmente inmediato.

Implementación de un Bucle IML Simple

```python
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

class SimpleIMLSystem:
    """
    Minimal IML system that allows online feedback and displays
    model state after each annotation.
    """

    def __init__(self, n_features=10, n_classes=2):
        self.model = SGDClassifier(loss='log_loss', max_iter=1, warm_start=True,
                                   random_state=42)
        self.scaler = StandardScaler()
        self.X_seen = []
        self.y_seen = []
        self.n_classes = n_classes
        self.initialized = False

    def update(self, x, y_true):
        """Receive a single labeled example and update the model."""
        self.X_seen.append(x)
        self.y_seen.append(y_true)

        if len(self.X_seen) >= 2 * self.n_classes:
            X_arr = np.array(self.X_seen)
            y_arr = np.array(self.y_seen)
            X_scaled = self.scaler.fit_transform(X_arr)
            self.model.partial_fit(X_scaled[-1:], y_arr[-1:],
                                   classes=list(range(self.n_classes)))
            self.initialized = True

        return self

    def predict_with_confidence(self, x):
        """Predict label and return confidence."""
        if not self.initialized:
            return None, 0.0
        x_scaled = self.scaler.transform(x.reshape(1, -1))
        probs = self.model.predict_proba(x_scaled)[0]
        return self.model.predict(x_scaled)[0], probs.max()

    def current_accuracy(self, X_val, y_val):
        if not self.initialized:
            return None
```

```
        X_scaled = self.scaler.transform(X_val)
        return (self.model.predict(X_scaled) == y_val).mean()


# Simulate an IML session
rng = np.random.default_rng(42)
X_all, y_all = np.random.default_rng(0).random((500, 10)), np.random.default_rng(0).integers(
X_val, y_val = X_all[400:], y_all[400:]

system = SimpleIMLSystem(n_features=10, n_classes=2)
accs, confidences = [], []

print("Step | Labels | Accuracy | Example confidence")
print("-" * 50)

for step in range(100):
    x, y = X_all[step], y_all[step]
    pred, conf = system.predict_with_confidence(x)
    system.update(x, y)

    if (step + 1) % 10 == 0:
        acc = system.current_accuracy(X_val, y_val)
        if acc is not None:
            accs.append(acc)
            print(f"  {step+1:3d} |  {step+1:4d}  |  {acc:.3f}   | {conf:.3f}")
```

La Prueba de la Abuela

Una heurística útil para evaluar el diseño de interfaces IML —y el diseño de sistemas HITL en términos más amplios— es lo que llamaremos la **Prueba de la Abuela** (una formulación original introducida aquí como restricción de diseño, no como referencia a trabajo previo):

> *Una mujer nacida en 1930 debería poder usar este dispositivo por voz, y si se frustra, degradarse elegantemente a una interfaz de teclado o texto.*

La prueba no trata principalmente sobre accesibilidad, aunque también lo es. Trata sobre **diseñar para la fricción**. Si un sistema requiere un modelo mental de redes neuronales, bucles de entrenamiento o distribuciones de probabilidad para usarse eficazmente, ha fallado la Prueba de la Abuela. El humano en el bucle debería poder participar sin entender el lado de la máquina del bucle.

Las implicaciones para el diseño de IML son concretas:

Alternativa de voz primero: La modalidad de interacción primaria debería ser el lenguaje natural o el gesto —no los deslizadores de parámetros o los umbrales de confianza. Los expertos pueden querer deslizadores; todos deberían poder decir "eso está mal".

Degradación elegante: Cuando la modalidad preferida del usuario falla o frustra, el sistema debería ofrecer una alternativa —no una pantalla en blanco o un mensaje de error. La interfaz es parte del sistema de aprendizaje; un usuario que no puede interactuar no puede enseñar.

Estado del modelo legible: La comprensión actual del modelo debería ser visible en términos

humanos. No "confianza: 0,73" sino "Estoy bastante seguro de que esto es [X], pero he visto ejemplos como este ir en ambas direcciones." La incertidumbre debería comunicarse en un lenguaje que invite a la corrección.

Tolerancia a la ambigüedad: Un usuario de 93 años y un ingeniero de aprendizaje automático de 23 años interactuarán de manera diferente con el mismo sistema. La Prueba de la Abuela pregunta si el sistema puede acomodar a ambos —no detectando la edad del usuario, sino diseñando interacciones que funcionen para un rango de experiencia y comodidad.

La prueba cobra especial importancia a medida que los sistemas de aprendizaje automático pasan de herramientas de investigación a infraestructura cotidiana. Un asistente de imágenes médicas utilizado por radiólogos, un clasificador de documentos legales utilizado por auxiliares judiciales, un sistema de retroalimentación educativa utilizado por docentes: cada uno implica humanos en el bucle que no se apuntaron para ser entrenadores de IA. Diseñar para ellos no es una concesión; es el propósito.

:::{admonition} Principio de Diseño :class: tip La Prueba de la Abuela es una restricción de diseño, no un grupo demográfico objetivo. Los sistemas que la superan son más robustos ante la diversidad de usuarios, más indulgentes con las brechas de experiencia y más honestos sobre lo que esperan de los humanos en el bucle. Si un sistema requiere explicación antes de usarse, está pidiendo al humano que haga trabajo extra. Ese trabajo debería estar justificado por un beneficio proporcional. :::

IML y Modelos de Fundación

El IML moderno aprovecha cada vez más los **modelos de fundación preentrenados** {cite}`bommasani2021opportunities` como base. En lugar de entrenar desde cero, los usuarios ajustan un gran modelo preentrenado con un pequeño número de ejemplos interactivos. Esto puede reducir drásticamente el número de ejemplos necesarios para alcanzar un rendimiento útil —en casos favorables, tan pocos como 5–50 ejemplos en lugar de miles, dependiendo de cuánto coincidan las representaciones preentrenadas con la tarea objetivo {cite}`bommasani2021opportunities`.

Las técnicas que lo posibilitan incluyen: - **Prompting con pocos ejemplos:** Proporcionar ejemplos en la ventana de contexto del LLM - **Ajuste con adaptadores:** Actualizar pequeños módulos adaptadores mientras se congela el modelo base - **Ajuste de parámetros eficiente (PEFT):** LoRA, ajuste de prefijos y métodos similares que permiten actualizaciones rápidas con pocos recursos

Los modelos de fundación cambian la dinámica del IML: los humanos ya no están enseñando a un modelo sin conocimiento previo desde cero, sino *dirigiendo* un modelo que ya sabe mucho. El desafío se desplaza de "cómo proporcionar suficientes ejemplos" a "cómo especificar exactamente en qué queremos que se comporte diferente de lo que el modelo ya hace".

> **See also:** La encuesta de {cite}`amershi2014power` sigue siendo la mejor visión general de los principios del IML. Para los sistemas de iniciativa mixta específicamente, véase {cite}`allen1999mixed`. Para los efectos de anclaje en la anotación, véase {cite}`geva2019annotator` y {cite}`lingren2014evaluating`.

Aprendizaje por Refuerzo a partir de Retroalimentación Humana

Ninguna técnica ha hecho más por llevar el HITL ML al ámbito general que el Aprendizaje por Refuerzo a partir de Retroalimentación Humana (RLHF). Es el mecanismo que sustenta InstructGPT {cite}`ouyang2022training` y un componente central de las tuberías de seguimiento de instrucciones en muchos grandes modelos de lenguaje modernos {cite}`stiennon2020learning`. Comprender RLHF —no solo como una receta a seguir, sino como un enfoque fundamentado para la alineación— es esencial para cualquiera que trabaje en la IA moderna.

El Problema de la Alineación

Los grandes modelos de lenguaje (LLMs) entrenados puramente mediante predicción del siguiente token optimizan un objetivo proxy: predecir qué texto viene a continuación en un corpus de texto escrito por humanos. Este objetivo está relacionado, pero es distinto de, lo que realmente queremos: respuestas que sean útiles, precisas, seguras y alineadas con los valores humanos.

La discrepancia entre el objetivo de entrenamiento y el comportamiento deseado se denomina el **problema de alineación** {cite}`russell2019human`. Concretamente, un modelo de lenguaje entrenado en texto de internet aprende a: - Producir continuaciones que suenen plausibles (que pueden ser factualmente incorrectas) - Reflejar los sesgos y perjuicios presentes en los datos de entrenamiento - Ser evasivo o manipulador cuando esto es lo que estadísticamente sigue al texto de entrada

RLHF aborda la alineación haciendo que las preferencias humanas sean *parte del objetivo de optimización*.

La Tubería de RLHF

RLHF procede en tres etapas:

```
Etapa 1: Ajuste Fino Supervisado (SFT)
  --> Recopilar datos de demostración (humano escribe respuestas ideales)
  --> Ajustar fino el LLM base con las demostraciones

Etapa 2: Entrenamiento del Modelo de Recompensa
  --> Recopilar preferencias por pares (humano puntúa A frente a B)
```

```
  --> Entrenar el modelo de recompensa R(x, y) para predecir preferencias humanas

Etapa 3: Ajuste Fino con RL
  --> Ajustar fino el LLM usando PPO/RL para maximizar R(x, y)
  --> La penalización KL evita la desviación excesiva del modelo SFT
```

Etapa 1: Ajuste Fino Supervisado

Partiendo de un modelo base preentrenado π_0, recopilamos un conjunto de datos de pares (consulta, respuesta ideal), escritos o seleccionados por contratistas humanos que siguen directrices detalladas. El modelo se ajusta fino sobre estas demostraciones usando entropía cruzada estándar:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{demo}}}\left[\log \pi_\theta(y \mid x)\right]$$

El modelo SFT π_{SFT} es un punto de partida mucho mejor para RLHF que el modelo preentrenado en bruto.

Etapa 2: Entrenamiento del Modelo de Recompensa

Para un conjunto de consultas $\{x_i\}$, generamos K respuestas por consulta usando π_{SFT} y las presentamos a los anotadores humanos como comparaciones por pares: "¿Cuál respuesta es mejor, A o B?"

El modelo de recompensa r_ϕ se entrena para predecir estas preferencias. Bajo el modelo **Bradley-Terry** (Capítulo 8), la probabilidad de que la respuesta y_w sea preferida a y_l es:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

El modelo de recompensa se entrena para minimizar la pérdida de clasificación por pares:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

El modelo de recompensa se inicializa típicamente desde el modelo SFT con una cabeza escalar que reemplaza la capa final.

Etapa 3: Ajuste Fino con RL mediante PPO

Con un modelo de recompensa entrenado, podemos usar el aprendizaje por refuerzo para ajustar fino el LLM. Cada consulta x es un estado; cada respuesta y es una trayectoria de elecciones de tokens; y la recompensa es $r_\phi(x, y)$.

El objetivo de optimización incluye una **penalización por divergencia KL** para evitar que el modelo se desvíe demasiado del modelo SFT base (lo que llevaría a la evasión de la recompensa {cite}`krakovna2020specification,gao2023scaling`):

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot \mid x) \| \pi_{\text{SFT}}(\cdot \mid x)\right]\right]$$

El parámetro β controla la intensidad de la penalización KL. Un β pequeño permite más optimización pero arriesga la evasión de la recompensa; un β grande mantiene el modelo cerca del SFT pero limita las ganancias de alineación.

La **Optimización de Política Proximal (PPO)** {cite}`schulman2017proximal` es el algoritmo estándar para esta etapa, elegido por su estabilidad en relación con los métodos de gradiente de política en bruto.

Una Demostración Simplificada de RLHF

La tubería completa de RLHF requiere infraestructura a gran escala. El siguiente ejemplo demuestra las ideas clave a pequeña escala.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

torch.manual_seed(42)
rng = np.random.default_rng(42)

# ----------------------------------------------
# Toy setup: responses are 4-dimensional vectors
# "Quality" is known analytically (sum of positive values)
# We simulate a reward model learning this from pairwise feedback
# ----------------------------------------------

class RewardModel(nn.Module):
    def __init__(self, d=4, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def true_quality(x):
    """The hidden ground-truth reward function."""
    return x.sum(dim=-1) + 0.5 * (x ** 2).mean(dim=-1)

# Generate pairwise preference data
N_PAIRS = 500
X1 = torch.randn(N_PAIRS, 4)
X2 = torch.randn(N_PAIRS, 4)
q1, q2 = true_quality(X1), true_quality(X2)
# Human prefers X1 when q1 > q2 (with some noise)
noise = torch.randn(N_PAIRS) * 0.5
preferred_1 = ((q1 - q2 + noise) > 0).float()

# Train reward model
rm = RewardModel(d=4, hidden=32)
```

```
optimizer = optim.Adam(rm.parameters(), lr=3e-3)

losses = []
for epoch in range(200):
    r1 = rm(X1)
    r2 = rm(X2)
    # Bradley-Terry loss
    logit = r1 - r2
    loss = F.binary_cross_entropy_with_logits(logit, preferred_1)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
    losses.append(loss.item())

# Evaluate: does the reward model agree with true quality?
X_eval = torch.randn(1000, 4)
with torch.no_grad():
    r_pred = rm(X_eval).numpy()
    r_true = true_quality(X_eval).numpy()

corr = np.corrcoef(r_pred, r_true)[0, 1]
print(f"Reward model correlation with true quality: {corr:.4f}")
print(f"Final training loss: {losses[-1]:.4f}")

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.plot(losses, color='#2b3a8f', linewidth=1.5)
plt.xlabel("Epoch"); plt.ylabel("Pairwise loss")
plt.title("Reward Model Training")

plt.subplot(1, 2, 2)
plt.scatter(r_true[:200], r_pred[:200], alpha=0.4, s=15, color='#2b3a8f')
plt.xlabel("True quality"); plt.ylabel("Predicted reward")
plt.title(f"Reward Model vs. Truth (r={corr:.3f})")
plt.tight_layout()
plt.savefig('reward_model.png', dpi=150)
plt.show()
```

Desafíos en RLHF

Evasión de la Recompensa

Un modo de fallo clave: la política encuentra formas de obtener alta recompensa del modelo de recompensa que no corresponden a un comportamiento genuinamente bueno. Por ejemplo, un LLM puede aprender a producir respuestas que son halagadoras o suenan confiadas (que los anotadores tienden a calificar favorablemente) en lugar de precisas.

La evasión de la recompensa es más probable cuando: - El modelo de recompensa se entrena con datos de preferencias insuficientes - Se permite que la política se desvíe mucho del modelo SFT base (β pequeño) - La distribución del modelo de recompensa cambia durante el entrenamiento con PPO

Estrategias de mitigación: Penalización KL, entrenamiento iterativo del modelo de recompensa, evaluación diversa, restricciones de IA constitucional.

Sesgo del Evaluador

Los anotadores humanos tienen sesgos sistemáticos. Tienden a preferir respuestas más largas (sesgo de verbosidad), texto que suena más confiado (sesgo de confianza) y respuestas que coinciden con sus creencias previas. Estos sesgos se propagan al modelo de recompensa.

El conocido fallo de servilismo de los modelos RLHF —donde el modelo le dice a los usuarios lo que quieren escuchar en lugar de lo que es verdad— es en parte resultado de la preferencia de los evaluadores por las respuestas condescendientes.

Supervisión Escalable

Para tareas complejas, los humanos no pueden juzgar de manera fiable cuál respuesta de la IA es correcta. Un anotador que compara dos demostraciones matemáticas largas o dos implementaciones de código puede simplemente escoger la más legible, independientemente de su corrección. La **supervisión escalable** es el problema de investigación abierto de diseñar procedimientos de evaluación que sigan siendo fiables a medida que aumenta la complejidad de la tarea {cite}`bowman2022measuring`.

IA Constitucional (RLAIF)

La **IA Constitucional** {cite}`bai2022constitutional`, desarrollada en Anthropic, reduce la dependencia de los anotadores humanos usando la propia IA para generar etiquetas de preferencia guiadas por un conjunto de principios (una "constitución"). El proceso:

1. Generar respuestas a indicaciones potencialmente dañinas
2. Usar un crítico de IA para evaluar respuestas según los principios constitucionales
3. Revisar respuestas guiadas por retroalimentación de IA (RLAIF — RL a partir de Retroalimentación de IA)
4. Entrenar un modelo de recompensa sobre las preferencias generadas por IA
5. Ajustar fino con RLHF usando este modelo de recompensa

RLAIF puede generar datos de preferencia a una escala mucho mayor que el etiquetado humano, y permite un control preciso sobre los valores codificados en el modelo de recompensa.

> **See also:** El artículo original de InstructGPT {cite}`ouyang2022training` describe la primera aplicación a gran escala de RLHF a los LLMs. El trabajo fundacional de RLHF para RL profundo es {cite}`christiano2017deep`. PPO se describe en {cite}`schulman2017proximal`. La IA Constitucional es de {cite}`bai2022constitutional`.

Aprendizaje a partir de Demostraciones

Cuando una tarea es difícil de especificar pero fácil de demostrar, puede ser más eficiente enseñar mediante el ejemplo que definir mediante reglas. Un experto humano muestra a un brazo robótico cómo agarrar un objeto; las interacciones de un programador con su entorno de desarrollo integrado proporcionan una secuencia de ediciones correctas; un gran maestro de ajedrez disputa una partida. El **aprendizaje a partir de demostraciones** extrae una política de dichos datos de comportamiento, evitando la necesidad de funciones de recompensa artesanales o especificaciones explícitas de tareas.

Clonación de Comportamiento

El enfoque más simple es la **clonación de comportamiento (BC)**: tratar la demostración como datos supervisados y aprender un mapeo de estados a acciones.

Dado un conjunto de datos de pares estado-acción $\mathcal{D} = \{(s_i, a_i)\}$ de un demostrador experto, ajustamos una política $\pi_\theta(a \mid s)$ minimizando la log-verosimilitud negativa:

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

Esto es precisamente el aprendizaje supervisado estándar aplicado a datos secuenciales.

El Problema del Desplazamiento de Covariables

La BC tiene una debilidad fundamental: **el desplazamiento de distribución** entre el entrenamiento y el despliegue. Las demostraciones del experto cubren los estados visitados por el experto. Pero durante el despliegue, la política aprendida puede tomar decisiones ligeramente diferentes, llevándola a estados que el experto nunca visitó —estados donde la política no tiene supervisión y puede fallar gravemente.

De manera crucial, los errores se **acumulan**: una pequeña desviación lleva a un estado desconocido, donde una acción ligeramente equivocada lleva a un estado aún más desconocido, y así sucesivamente. El rendimiento se degrada como $O(T^2\epsilon)$ donde T es la longitud del episodio y ϵ es la tasa de error en cada paso —mucho peor que la degradación $O(T\epsilon)$ de una política oráculo {cite}`ross2010efficient`.

> **Ejemplo: Conducción Autónoma** :class: note
>
> Un modelo de clonación de comportamiento para mantener el carril entrenado con datos de conducción humana funciona bien en carreteras rectas (estados cercanos a la distribución de entrenamiento). Pero en el momento en que se desvía

ligeramente —un estado en el que ningún conductor humano estaría porque ya habría corregido— no tiene datos que lo guíen y puede salirse de la carretera.

```
Algoritmo DAgger:
  Inicializar: D <- {} (conjunto de datos vacío)
  Entrenar política inicial pi_1 con M demostraciones del experto

  para iteración i = 1, 2, ..., N:
    1. Ejecutar pi_i en el entorno para recopilar estados {s_1, ..., s_t}
    2. Consultar al experto las acciones en cada estado visitado: a_t = pi*(s_t)
    3. Agregar: D <- D u {(s_1, a_1), ..., (s_t, a_t)}
    4. Entrenar pi_{i+1} por aprendizaje supervisado sobre D
```

DAgger logra un arrepentimiento $O(T\epsilon)$ —el mismo que una política oráculo— porque la distribución de entrenamiento converge para coincidir con la distribución de despliegue.

El requisito clave es que el experto pueda ser consultado en cualquier estado, incluidos los estados que el experto nunca visitaría naturalmente. Esto es factible en simulación (pedir al experto que corrija el robot desde una configuración inusual) pero puede ser difícil o inseguro en sistemas físicos reales.

Aprendizaje por Refuerzo Inverso

A veces el comportamiento del experto se entiende mejor no como una secuencia de acciones a imitar, sino como el resultado de optimizar una función de recompensa desconocida. El **Aprendizaje por Refuerzo Inverso (IRL)** {cite}`ng2000algorithms` recupera esta función de recompensa latente a partir de las demostraciones.

Dadas las demostraciones $\tau = \{(s_1, a_1), \dots, (s_T, a_T)\}$, el IRL encuentra una función de recompensa $R(s, a)$ tal que la política del experto sea óptima bajo R.

El atractivo del IRL sobre la BC: si recuperamos la verdadera función de recompensa, podemos re-optimizarla en nuevos entornos, con dinámicas diferentes, o con planificadores mejorados —generalizando mucho más allá de los escenarios demostrados.

IRL de Máxima Entropía

El **IRL MaxEnt** {cite}`ziebart2008maximum` resuelve el problema de ambigüedad del IRL (existen muchas funciones de recompensa consistentes con cualquier conjunto de demostraciones) eligiendo la función de recompensa que, siendo consistente con el comportamiento demostrado, conduce a una distribución sobre trayectorias con *máxima entropía*. Las trayectorias se distribuyen como:

$$P(\tau \mid R) \propto \exp\left(\sum_t R(s_t, a_t)\right)$$

El objetivo de aprendizaje hace coincidir las expectativas de características observadas del experto $\mu_E = \mathbb{E}_{\tau\sim\pi^*}[\phi(\tau)]$ con las expectativas de características del modelo $\mu_\theta = \mathbb{E}_{\tau\sim\pi_\theta}[\phi(\tau)]$.

GAIL: Aprendizaje por Imitación Adversarial Generativo

GAIL {cite}`ho2016generative` omite por completo el aprendizaje de la función de recompensa, usando una formulación similar a las GAN para hacer coincidir directamente la distribución estado-acción del experto.

Se entrena un discriminador D_ψ para distinguir pares estado-acción del experto $(s, a) \sim \pi^*$ de pares estado-acción de la política $(s, a) \sim \pi_\theta$:

$$\mathcal{L}_D = -\mathbb{E}_{\pi^*}[\log D_\psi(s, a)] - \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s, a))]$$

El generador (la política π_θ) se entrena para engañar al discriminador —es decir, para producir pares estado-acción que parezcan del experto. La salida del discriminador $\log D_\psi(s, a)$ sirve como señal de recompensa para la política.

GAIL logra un rendimiento a nivel de experto en benchmarks de control continuo con muchas menos demostraciones que la BC, y generaliza mejor que el IRL MaxEnt en entornos complejos.

Clonación de Comportamiento en PLN: Un Ejemplo Práctico

```python
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)

# ------------------------------------------------
# Toy NLP task: rewriting sentences to be more formal
# We simulate this as a simple sequence transformation
# In practice: fine-tuning a seq2seq model on expert rewrites
# ------------------------------------------------

class SimpleSeq2Seq(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.encoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim, vocab_size)
        self.hidden_dim = hidden_dim

    def forward(self, src, tgt):
        src_emb = self.embed(src)
        _, hidden = self.encoder(src_emb)
        tgt_emb = self.embed(tgt)
        out, _ = self.decoder(tgt_emb, hidden)
        return self.proj(out)

# Generate synthetic demonstration data
```

```
VOCAB = 100
rng = np.random.default_rng(42)
N, SEQ_LEN = 1000, 12

src_seqs = torch.tensor(rng.integers(1, VOCAB, (N, SEQ_LEN)), dtype=torch.long)
# "Expert" transformation: shift tokens by 1 (toy formalization)
tgt_seqs = torch.clamp(src_seqs + 1, 1, VOCAB - 1)
tgt_in  = torch.cat([torch.ones(N, 1, dtype=torch.long), tgt_seqs[:, :-1]], dim=1)

dataset = TensorDataset(src_seqs, tgt_in, tgt_seqs)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

model = SimpleSeq2Seq(vocab_size=VOCAB)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Behavioral cloning training
train_losses = []
for epoch in range(20):
    epoch_loss = 0
    for src, tgt_i, tgt_o in loader:
        logits = model(src, tgt_i)
        loss = criterion(logits.reshape(-1, VOCAB), tgt_o.reshape(-1))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    train_losses.append(epoch_loss / len(loader))

print(f"Initial loss: {train_losses[0]:.3f}")
print(f"Final loss:   {train_losses[-1]:.3f}")

# Evaluate: check token accuracy on held-out examples
model.eval()
with torch.no_grad():
    src_test = src_seqs[-100:]
    tgt_test_in = tgt_in[-100:]
    tgt_test_out = tgt_seqs[-100:]
    logits = model(src_test, tgt_test_in)
    preds = logits.argmax(dim=-1)
    acc = (preds == tgt_test_out).float().mean().item()
    print(f"Token accuracy on held-out set: {acc:.3f}")
```

Comparación de los Métodos de Aprendizaje por Imitación

Método	¿Requiere recompensa?	¿Experto consultado en línea?	¿Generaliza a nueva dinámica?	Complejidad
Clonación de Comportamiento	No	No	Mal (desplazamiento de distribución)	Baja
DAgger	No	Sí	Moderadamente	Media

Método	¿Requiere recompensa?	¿Experto consultado en línea?	¿Generaliza a nueva dinámica?	Complejidad
IRL MaxEnt	La recupera	No	Bien	Alta
GAIL	No	No	Bien	Alta

Aplicaciones

Robótica. Enseñar a los robots a manipular objetos, navegar por entornos o realizar tareas domésticas. Las demostraciones físicas se recopilan mediante teleoperación o enseñanza cinestésica.

Conducción autónoma. Los primeros sistemas de vehículos autónomos como ALVINN {cite}`pomerleau1989alvinn` y DAVE de NVIDIA dependían en gran medida de la clonación de comportamiento a partir de datos de conducción humana.

IA en videojuegos. El aprendizaje por imitación sobre partidas humanas inicializa agentes antes del ajuste fino por RL. AlphaStar entrenó sobre repeticiones humanas antes del RL; este enfoque es común cuando se dispone de demostraciones de nivel humano.

Generación de código. El ajuste fino de modelos de lenguaje sobre demostraciones de código de alta calidad (GitHub Copilot, Codex) es una forma de clonación de comportamiento.

Soporte a la decisión clínica. Aprender de secuencias de decisiones de médicos expertos para protocolos complejos.

> **See also:** El análisis fundacional de BC/DAgger está en {cite}`ross2011reduction`. El IRL MaxEnt es de {cite}`ziebart2008maximum`. GAIL es de {cite}`ho2016generative`. Para una encuesta exhaustiva del aprendizaje por imitación, véase {cite}`osa2018algorithmic`.

Aprendizaje a partir de Comparaciones y Rankings

Pedirle a un humano que asigne una puntuación de calidad absoluta a una salida es difícil. ¿Cuál es la calidad numérica de este ensayo, en una escala del 1 al 10? La pregunta está mal planteada: los humanos carecen de una escala interna estable, y sus puntuaciones están fuertemente influidas por el anclaje, el contexto y la fatiga.

Pedirle a un humano que *compare* dos salidas es mucho más fácil: ¿cuál ensayo es mejor, A o B? Los juicios comparativos son más consistentes, más fiables y aprovechan las preferencias humanas de manera más directa que las valoraciones absolutas. Este capítulo cubre los fundamentos matemáticos y las aplicaciones prácticas del aprendizaje a partir de comparaciones y rankings.

Por Qué las Comparaciones Son Mejores que las Valoraciones

Fundamento Psicológico

La superioridad de los juicios comparativos tiene una larga historia en psicometría. La Ley del Juicio Comparativo de Thurstone {cite}`thurstone1927law` (1927) demostró que incluso cuando los humanos tienen juicios absolutos inconsistentes, sus juicios relativos siguen una ley probabilística consistente. Las comparaciones aprovechan el hecho de que los humanos son mucho mejores en la percepción *relativa* que en la calibración absoluta.

Eficiencia Estadística

Cada comparación por pares proporciona información sobre las posiciones *relativas* de dos elementos en la escala de calidad. Con K elementos, $K - 1$ comparaciones pueden ordenar todos los elementos; solo se necesitan $O(\log K)$ comparaciones para encontrar el elemento superior. Las valoraciones absolutas suelen requerir más juicios para lograr la misma precisión.

Escalabilidad

Para los modelos generativos, el número de salidas distintas es efectivamente infinito. Valorar una salida en términos absolutos requiere establecer una escala compartida en todas las salidas; comparar salidas requiere solo juicios locales y relativos que se calibran naturalmente entre sí.

El Modelo de Bradley-Terry

El modelo probabilístico dominante para las comparaciones por pares es el **modelo de Bradley-Terry (BT)** {cite}`bradley1952rank`. Cada elemento i tiene una puntuación de calidad latente $\alpha_i \in \mathbb{R}$. La probabilidad de que el elemento i sea preferido al elemento j en una comparación directa es:

$$P(i \succ j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}} = \sigma(\alpha_i - \alpha_j)$$

donde σ es la función sigmoide logística. Esto equivale a asumir que la calidad percibida del elemento i es $\alpha_i + \epsilon$ donde ϵ es un término de ruido logístico estándar.

Identificabilidad

El modelo BT es identificable salvo una traslación: si α es una solución, también lo es $\alpha + c$ para cualquier constante c. La convención estándar es fijar una puntuación (por ejemplo, $\alpha_0 = 0$) o restringir $\sum_i \alpha_i = 0$. Las puntuaciones solo se identifican cuando el **grafo de comparaciones** (nodos = elementos, aristas = pares observados) está **conectado** —si el grafo tiene componentes desconectados, las puntuaciones relativas entre componentes no están definidas.

Estimación de Parámetros

Dado un conjunto de datos de comparaciones por pares $\mathcal{D} = \{(i, j, y_{ij})\}$ donde $y_{ij} = 1$ si i fue preferido a j, el log-verosimilitud es:

$$\mathcal{L}(\alpha) = \sum_{(i,j,y_{ij}) \in \mathcal{D}} \left[y_{ij} \log \sigma(\alpha_i - \alpha_j) + (1 - y_{ij}) \log \sigma(\alpha_j - \alpha_i) \right]$$

Esta es una función cóncava de α y puede maximizarse mediante ascenso de gradiente o el método de Newton.

```python
import numpy as np
from scipy.optimize import minimize
from scipy.special import expit  # sigmoid

rng = np.random.default_rng(42)

# ----------------------------------------------
# Simulate Bradley-Terry: 8 items with true quality scores
# Generate pairwise comparisons and recover the scores
# ----------------------------------------------

N_ITEMS = 8
true_alpha = rng.normal(0, 1, N_ITEMS)
print(f"True quality ranking: {np.argsort(true_alpha)[::-1]}")

# Generate comparisons: every pair compared 5 times
comparisons = []
for i in range(N_ITEMS):
    for j in range(i + 1, N_ITEMS):
        for _ in range(5):
```

```
            p_i_wins = expit(true_alpha[i] - true_alpha[j])
            winner = i if rng.random() < p_i_wins else j
            loser  = j if winner == i else i
            comparisons.append((winner, loser))

print(f"Total comparisons: {len(comparisons)}")

def neg_log_likelihood(alpha, comparisons):
    """Bradley-Terry negative log-likelihood."""
    alpha = np.array(alpha)
    loss = 0.0
    for winner, loser in comparisons:
        log_prob = np.log(expit(alpha[winner] - alpha[loser]) + 1e-10)
        loss -= log_prob
    return loss

def neg_grad(alpha, comparisons):
    alpha = np.array(alpha)
    grad = np.zeros(len(alpha))
    for winner, loser in comparisons:
        p = expit(alpha[winner] - alpha[loser])
        grad[winner] -= (1 - p)
        grad[loser]  -= (-p)
    return grad

# Fix alpha[0] = 0 to resolve scale ambiguity
result = minimize(
    lambda a: neg_log_likelihood(np.concatenate([[0.0], a]), comparisons),
    x0=np.zeros(N_ITEMS - 1),
    jac=lambda a: neg_grad(np.concatenate([[0.0], a]), comparisons)[1:],
    method='L-BFGS-B'
)
est_alpha = np.concatenate([[0.0], result.x])

# Compare true vs estimated ranking
true_rank = np.argsort(true_alpha)[::-1]
est_rank  = np.argsort(est_alpha)[::-1]

print(f"\nTrue ranking (item indices): {true_rank}")
print(f"Estimated ranking:           {est_rank}")
rank_corr = np.corrcoef(true_alpha, est_alpha)[0, 1]
print(f"Correlation with true scores: {rank_corr:.4f}")
```

El Modelo de Thurstone

El modelo de Thurstone {cite}`thurstone1927law` está estrechamente relacionado con Bradley-Terry pero utiliza ruido gaussiano en lugar de logístico:

$$P(i \succ j) = \Phi\left(\frac{\alpha_i - \alpha_j}{\sqrt{2}\sigma}\right)$$

donde Φ es la función de distribución acumulada normal estándar. Cuando $\sigma = 1/\sqrt{2}$, esto se vuelve equivalente a BT con una pequeña diferencia de escala. En la práctica, los dos modelos dan resultados casi idénticos.

Agregación de Rankings

Cuando cada anotador proporciona un ranking completo de K elementos (en lugar de comparaciones por pares), el problema es la **agregación de rankings**: combinar múltiples listas ordenadas en un ranking de consenso.

Conteo de Borda: Cada elemento recibe una puntuación igual al número de elementos clasificados por debajo de él en el ranking de cada anotador. Las puntuaciones se suman entre anotadores. Sencillo y robusto.

Kemeny–Young: Encontrar el ranking que minimiza la suma de desacuerdos por pares (la distancia tau de Kendall) con el ranking de cada anotador. Es NP-difícil para K grande pero tratable para conjuntos pequeños.

RankNet / ListNet: Enfoques neurales que aprenden una función de puntuación a partir de listas ordenadas, permitiendo la generalización a elementos no vistos.

Bandidos Duelos

En el aprendizaje de preferencias **en línea**, los elementos llegan en flujo continuo y debemos decidir qué pares comparar, equilibrando la exploración (aprender sobre elementos desconocidos) y la explotación (presentar elementos de alta calidad). Este es el problema del **bandido duelo** {cite}`yue2009interactively`.

Algoritmos clave: - **Doubler:** Mantiene un elemento campeón; lo desafía con competidores aleatorios - **RUCB (Límite de Confianza Superior Relativo):** Calcula intervalos de confianza estilo UCB para la probabilidad de cada elemento de vencer a cada otro elemento - **MergeRank:** Combina comparación estilo torneo con UCB

Los bandidos duelos se utilizan en sistemas de recomendación en línea (qué artículo mostrar a continuación, dado el retroalimentación implícita) y en la obtención interactiva de preferencias para la recopilación de datos de RLHF.

Aprendizaje de Preferencias para Modelos de Lenguaje

En el contexto de RLHF (Capítulo 6), el modelo de Bradley-Terry se utiliza para entrenar el modelo de recompensa. Una variante importante es la **Optimización Directa de Preferencias (DPO)** {cite}`rafailov2023direct`, que muestra que el objetivo de RLHF puede optimizarse directamente a partir de datos de preferencia sin entrenar un modelo de recompensa separado:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO es más simple que el RLHF completo (sin bucle de entrenamiento PPO, sin modelo de recompensa) y logra resultados comparables o mejores en muchos benchmarks {cite}`rafailov2023direct`. Se ha convertido en una alternativa ampliamente adoptada al RLHF basado en PPO para el ajuste fino del seguimiento de instrucciones, aunque ambos enfoques siguen en uso activo y sus fortalezas relativas dependen de la tarea.

Recopilación de Datos de Preferencia de Alta Calidad

La calidad de los datos de preferencia determina la calidad del modelo de recompensa. Consideraciones clave:

Diversidad de consultas. Los datos de preferencia deben cubrir la distribución completa de consultas que el modelo encontrará en el despliegue. Las lagunas en la cobertura llevan a un comportamiento poco fiable del modelo de recompensa en esas regiones.

Diversidad de respuestas. Comparar dos respuestas muy similares proporciona poca información. Las respuestas comparadas deben diferir lo suficiente para que los anotadores tengan una preferencia clara.

Concordancia entre anotadores. Una baja concordancia entre anotadores sugiere que los criterios de comparación son ambiguos. Medir la concordancia (κ de Cohen) y revisar las directrices cuando esté por debajo de los umbrales aceptables.

Calibración. Los anotadores deben entender *por qué* una respuesta es mejor: utilidad, precisión, seguridad, estilo. Las tareas que agrupan múltiples criterios tienden a producir preferencias inconsistentes. A menudo es mejor recopilar preferencias sobre cada criterio por separado.

> **See also:** Modelo de Bradley-Terry: {cite}`bradley1952rank`. Thurstone: {cite}`thurstone1927law`. Bandidos duelos: {cite}`yue2009interactively`. Optimización Directa de Preferencias (DPO): {cite}`rafailov2023direct`.

HITL en Procesamiento de Lenguaje Natural

El lenguaje natural es el dominio donde el HITL ML ha tenido posiblemente el mayor impacto —y donde afloran sus dificultades conceptuales más profundas. El lenguaje es inherentemente social: su significado es construido por comunidades humanas, su pragmática depende del contexto y la intención, y su calidad solo puede ser juzgada por lectores humanos. Pero esto también significa que la anotación de PLN no es simplemente un proceso de recopilación de observaciones. Es un proceso de *construcción* de categorías.

El Problema Constitutivo en la Anotación de PLN

En imágenes médicas, existe una verdad fundamental: un tumor está presente o no lo está. La etiqueta del radiólogo puede ser incierta, pero intenta rastrear algo real. La anotación de PLN es a menudo fundamentalmente diferente. Cuando un anotador marca un tuit como "tóxico", no hay ninguna molécula-tóxica en el tuit que estemos intentando detectar. **La etiqueta constituye la categoría.**

Esto tiene consecuencias profundas que se subestiman con frecuencia:

La fuerza laboral de anotación define el fenómeno. Un esquema de etiquetado para "lenguaje ofensivo" codifica las sensibilidades de quien lo diseñó y de quien lo aplicó. Un equipo de anotadores de habla inglesa con un trasfondo demográfico uniforme, trabajando bajo directrices redactadas por un equipo de política corporativa, produce un conjunto de datos que refleja la comprensión de ese equipo sobre la ofensa —no algún estándar humano universal. Los modelos entrenados con dichos datos exhibirán esos mismos límites implícitos.

Las directrices son teoría, lo reconozcan o no. Todo esquema de anotación hace afirmaciones sobre ontología. Decidir que "ironía" y "sarcasmo" son la misma clase es una afirmación teórica, no una conveniencia neutral. Decidir etiquetar el "enojo" como una única clase en lugar de distinguir el "enojo justificado" del "enojo hostil" colapsa una distinción que puede importar para la tarea posterior. Estas decisiones se toman bajo presión de producción y rara vez se revisan.

Inestabilidad temporal de las etiquetas. El lenguaje social evoluciona. Un modelo de toxicidad entrenado en 2018 clasificará incorrectamente contenido de 2024 no porque esté estadísticamente subentrenado, sino porque el significado social de ciertos términos ha cambiado. La anotación de PLN no consiste en muestrear de una distribución estática —consiste en muestrear de un objetivo en movimiento que el propio acto de etiquetar ayuda parcialmente a constituir.

:::{admonition} El Problema del Artefacto de Anotación :class: important

Geva et al. {cite}`geva2019annotator` demostraron que los conjuntos de datos de Inferencia en Lenguaje Natural (NLI) contienen artefactos sistemáticos introducidos por el propio proceso de anotación. Los anotadores a quienes se pide que escriban hipótesis de "implicación" para una premisa dada tienden a usar ciertos patrones sintácticos; los anotadores que escriben hipótesis de "contradicción" usan otros. Los modelos aprenden a clasificar basándose en estos artefactos en lugar de en la relación semántica pretendida —resuelven la tarea de anotación, no la tarea subyacente de PLN.

Esto no es descuido. Es una consecuencia inherente de que los humanos construyan ejemplos para ajustarse a una etiqueta. El proceso HITL inserta una señal sistemática que nunca estuvo destinada a estar en los datos. :::

Anotación de Clasificación de Texto

La tarea de anotación de PLN más simple es asignar una categoría a un documento de texto. El análisis de sentimientos, la clasificación de temas, la detección de intención y el filtrado de correo no deseado son todas tareas de clasificación.

Desafíos específicos de la clasificación de texto:

Subjetividad. Categorías como "tóxico" u "ofensivo" son inherentemente subjetivas y varían según los contextos culturales, los trasfondos de los anotadores y el contexto de la plataforma. Las percepciones de la ofensividad difieren significativamente según la demografía —un hecho con implicaciones directas para la equidad {cite}`blodgett2020language`.

Ambigüedad de etiquetas. Muchos documentos pertenecen a múltiples categorías o están en un límite. Una reseña que es 60% positiva y 40% negativa es genuinamente ambigua, no está mal etiquetada. Forzar una sola etiqueta descarta información real (véase el Capítulo 13 sobre etiquetas suaves y desacuerdo entre anotadores).

Granularidad de etiquetas. Un modelo de sentimientos de 2 clases puede ser suficiente para el monitoreo de redes sociales; puede necesitarse una escala de 7 puntos para el análisis de retroalimentación de productos. La granularidad correcta depende de la tarea posterior, pero generalmente se fija antes de la anotación —una decisión de diseño consecuente tomada con datos insuficientes.

Reconocimiento de Entidades Nombradas

La anotación de NER requiere identificar tramos de texto y asignar un tipo de entidad (PERSONA, ORGANIZACIÓN, LUGAR, etc.). La anotación es más compleja que la clasificación de documentos por varias razones:

Los límites de los tramos son ambiguos. En "Apple CEO Tim Cook anunció…", ¿abarca la entidad PERSONA solo "Tim Cook" o también "Apple CEO Tim Cook"? Las directrices deben abordar explícitamente estos casos, y la concordancia entre anotadores sobre los tramos es consistentemente menor que sobre los tipos.

La asignación de tipos requiere conocimiento del mundo. "Apple" es ORG en un contexto, PRODUCTO en otro, y posiblemente ninguno en "tarta de manzana". Los anotadores necesitan suficiente conocimiento del dominio para hacer asignaciones de tipos correctas.

Entidades anidadas. "La Universidad de California, Berkeley" contiene ORGANIZACIÓN anidada dentro de LUGAR. El etiquetado BIO estándar no puede representar entidades anidadas; se requieren esquemas más complejos (por ejemplo, BIOES, o formatos basados en grafos).

Eficiencia de la anotación. La pre-anotación con un modelo NER de referencia acelera significativamente la anotación al permitir a los anotadores corregir predicciones en lugar de anotar desde cero. En un estudio de NER clínico, se observaron aumentos de rendimiento del 30–60% {cite}`lingren2014evaluating`; la magnitud de tales ganancias depende fuertemente de la calidad del modelo de referencia y del dominio.

Extracción de Relaciones y Anotación Semántica

Más allá de identificar entidades, la extracción de relaciones requiere anotar *relaciones* entre entidades:

- Los anotadores deben comprender tanto las entidades como el predicado que las conecta
- Los tipos de relación tienen distinciones semánticas complejas (TRABAJA_EN vs. EMPLEADO_POR vs. FUNDÓ)
- Los ejemplos negativos (pares de entidades sin relación) son mucho más comunes que los positivos

La concordancia entre anotadores para la extracción de relaciones tiende a ser menor que para las tareas de clasificación. Para esquemas bien definidos, se reportan comúnmente valores de κ en el rango de 0,65–0,80 {cite}`pustejovsky2012natural`; para relaciones semánticas complejas (causalidad de eventos, relaciones temporales), la concordancia puede caer considerablemente por debajo, dependiendo del diseño del esquema y la formación de los anotadores.

Posedición de Traducción Automática (MTPE)

La posedición de traducción automática es una forma madura de HITL en PLN. Un traductor humano corrige la salida de un sistema de traducción automática en lugar de traducir desde cero:

Posedición ligera (LPE): Solo se corrigen los errores críticos. Adecuada cuando los requisitos de calidad de traducción son moderados.

Posedición completa (FPE): La salida se corrige hasta alcanzar la calidad de publicación. La salida editada generalmente se convierte en datos de entrenamiento para mejorar aún más la traducción automática —un genuino ciclo de refinamiento con humano en el bucle.

HTER (Tasa de Edición de Traducción Dirigida al Humano): Una métrica que mide la distancia de edición entre la salida de la traducción automática y la traducción posedited, normalizada por la longitud de la oración {cite}`graham2015accurate`. Como regla práctica aproximada, un HTER por debajo de aproximadamente 0,3 se considera a menudo un buen resultado de traducción automática; por encima de 0,5, traducir desde cero puede ser más rápido —aunque estos umbrales varían según el dominio y el par de idiomas.

IA Conversacional y Anotación de Diálogo

La anotación de diálogo introduce estructura temporal:

- **Anotación a nivel de turno:** etiquetar cada turno (intención, sentimiento, calidad)
- **Anotación a nivel de diálogo:** evaluar la coherencia general, el éxito de la tarea, la satisfacción del usuario
- **Anotación de trazas de interacción:** identificar momentos específicos de fallo en una conversación

HITL es especialmente importante en el diálogo porque los fallos del modelo son a menudo sutiles y acumulativos: un error factual en el turno 3 puede no hacerse aparente hasta el turno 7. Los anotadores humanos que rastrean conversaciones pueden identificar estos patrones de fallo de largo alcance que las métricas automatizadas pierden por completo.

Etiquetado Programático y Supervisión Débil

Cuando los datos etiquetados son escasos, la **supervisión débil** permite usar múltiples funciones de etiquetado ruidosas y superpuestas para generar etiquetas probabilísticas a escala. **Snorkel** {cite}`ratner2017snorkel` es el marco canónico:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

rng = np.random.default_rng(42)

# ------------------------------------------------------
# Toy weak supervision: sentiment classification
# 5 labeling functions (LFs) with different coverage/accuracy
# Label: +1 (positive), -1 (negative), 0 (abstain)
# ------------------------------------------------------

N = 1000
true_labels = rng.choice([-1, 1], size=N)
X_features = np.column_stack([
    true_labels * 0.8 + rng.normal(0, 0.5, N),
    rng.normal(0, 1, N),
    rng.normal(0, 1, N),
])

def make_lf(accuracy, coverage, seed):
    rng_lf = np.random.default_rng(seed)
    votes = np.zeros(N, dtype=int)
    active = rng_lf.random(N) < coverage
    correct = rng_lf.random(N) < accuracy
    votes[active & correct]  = true_labels[active & correct]
    votes[active & ~correct] = -true_labels[active & ~correct]
    return votes

LFs = np.column_stack([
```

```
    make_lf(accuracy=0.85, coverage=0.60, seed=1),
    make_lf(accuracy=0.78, coverage=0.45, seed=2),
    make_lf(accuracy=0.70, coverage=0.80, seed=3),
    make_lf(accuracy=0.90, coverage=0.30, seed=4),
    make_lf(accuracy=0.60, coverage=0.90, seed=5),
])

def majority_vote(LF_matrix):
    labels = []
    for i in range(LF_matrix.shape[0]):
        votes = LF_matrix[i][LF_matrix[i] != 0]
        labels.append(0 if len(votes) == 0 else int(np.sign(votes.sum())))
    return np.array(labels)

mv_labels = majority_vote(LFs)
covered = mv_labels != 0

print(f"Coverage:                    {covered.mean():.1%}")
print(f"MV accuracy (covered):       {(mv_labels[covered] == true_labels[covered]).mean():.3f

X_train, y_train = X_features[covered], mv_labels[covered]
X_test,  y_test  = X_features[~covered], true_labels[~covered]

if len(X_train) > 10 and len(X_test) > 10:
    clf = LogisticRegression().fit(X_train, y_train)
    preds = clf.predict(X_test)
    print(f"F1 on uncovered test set:    {f1_score(y_test, preds):.3f}")
```

RLHF para Modelos de Lenguaje: La Realidad de la Anotación

El Capítulo 6 abordó RLHF técnicamente. Desde la perspectiva del PLN, la tarea de anotación es más difícil de lo que parece desde fuera.

Lo que realmente se pide a los anotadores —comparar dos salidas del modelo e indicar cuál es "mejor"— suena simple. En la práctica, "mejor" es un constructo no especificado que los anotadores resuelven usando heurísticas personales. Algunos ponderan fuertemente la fluidez; otros la precisión factual; otros penalizan la verbosidad. Sin directrices estrictas, el conjunto de datos de preferencias resultante no refleja los valores humanos en abstracto, sino las estrategias de resolución particulares de la fuerza laboral de anotación empleada.

Las dimensiones clave de anotación son:

- *Utilidad:* ¿La respuesta responde bien a la pregunta? ¿Es informativa, clara y apropiadamente detallada?
- *Factualidad:* ¿La respuesta es factualmente precisa? Esto requiere que los evaluadores tengan conocimiento del dominio —un requisito que crea graves problemas de calidad a escala, ya que los anotadores generalistas no pueden verificar afirmaciones especializadas.
- *Inocuidad:* ¿La respuesta evita contenido tóxico, sesgado, dañino o inapropiado? Estos juicios requieren directrices detalladas porque "dañino" depende enormemente del contexto y varía según las culturas, el tiempo y el contexto de la plataforma.
- *Calibración:* ¿La respuesta expresa incertidumbre apropiada? Los anotadores prefieren

sistemáticamente respuestas que suenan confiadas, lo que crea una señal de recompensa contraria a la humildad epistémica adecuada.

La interacción entre criterios es compleja: una respuesta maximalmente útil a una pregunta peligrosa puede ser maximalmente dañina. Las directrices deben especificar cómo equilibrar criterios en competencia —y esos equilibrios son efectivamente decisiones de política, no decisiones de anotación. La fuerza laboral de anotación está tomando decisiones políticas.

La escala concentra la influencia demográfica. RLHF para grandes modelos implica fuerzas laborales de anotación relativamente pequeñas (cientos a pocos miles) tomando miles de millones de decisiones posteriores. Los sesgos demográficos y culturales de esa fuerza laboral se propagan al comportamiento del modelo a escala de una manera que no ocurriría si la anotación estuviera más distribuida. Este es uno de los problemas estructurales menos discutidos en la tubería actual de RLHF.

El Bucle de Retroalimentación Anotación–Modelo

En PLN más que en cualquier otro dominio, los procesos de anotación y desarrollo del modelo se entrelazan con el tiempo:

1. Los anotadores se calibran usando las salidas existentes del modelo como referencia (a menudo de manera implícita).
2. El modelo de recompensa aprende lo que tienden a preferir las anotaciones.
3. El generador se ajusta fino para producir salidas que obtengan alta recompensa.
4. Esas salidas influyen en lo que parece "bueno" en las rondas de anotación posteriores.

Este bucle de retroalimentación no es inherentemente patológico —es lo que permite que RLHF converja— pero significa que el comportamiento del modelo está moldeado por un objetivo en movimiento que el propio proceso de anotación ayuda a mover. Distinguir empíricamente lo que el modelo aprendió porque refleja preferencias humanas de lo que aprendió porque aprendió a hacer coincidencia de patrones con las heurísticas de los anotadores es difícil.

No existe una solución limpia. La mejor práctica actual es monitorear la deriva usando juicios de preferencia reservados recopilados a intervalos regulares, y tratar la versión de las directrices de anotación como una variable experimental.

Evaluación de Modelos de PLN Generativos

A diferencia de los modelos de clasificación con una métrica de precisión clara, evaluar la calidad de la generación requiere juicio humano:

Método de evaluación	Descripción	Coste
Evaluación directa (DA)	Los anotadores puntúan la calidad en una escala	Medio
Juicio comparativo	Los anotadores comparan dos salidas	Bajo
MQM (Métricas de Calidad Multidimensional)	Taxonomía estructurada de errores	Alto

Método de evaluación	Descripción	Coste
Preferencia RLHF	Etiquetas de preferencia usadas para el entrenamiento	Medio
LLM como juez	El LLM puntúa salidas (correlaciona moderadamente con el humano)	Muy bajo
BERTScore, BLEU	Métricas automáticas (correlación imperfecta con el juicio humano)	Muy bajo

Las métricas automáticas (BLEU para traducción automática, ROUGE para resumir) son económicas pero se correlacionan imperfectamente con los juicios de calidad humana {cite}`reiter2018structured`. Los enfoques LLM-como-juez muestran una concordancia moderada con los anotadores humanos {cite}`zheng2023judging` y se usan cada vez más para la iteración rápida, pero no deben reemplazar la evaluación humana para los juicios definitivos. Para las decisiones con riesgos reales, la evaluación humana sigue siendo necesaria.

> **See also:** Supervisión débil con Snorkel: {cite}`ratner2017snorkel`. Directrices de anotación de PLN: {cite}`pustejovsky2012natural`. Artefactos de anotación en NLI: {cite}`geva2019annotator`. Sesgo de anotadores y conjuntos de datos de PLN: {cite}`blodgett2020language`. Para la evaluación de modelos generativos: {cite}`reiter2018structured`.

HITL en Visión por Computadora

La visión por computadora ofrece algunos de los ejemplos más visualmente intuitivos de HITL ML. El desafío ImageNet, construido sobre 14 millones de imágenes etiquetadas por humanos, lanzó la era del aprendizaje profundo. La anotación de imágenes médicas por radiólogos impulsa la IA diagnóstica. Los vehículos autónomos dependen de millones de fotogramas etiquetados por humanos para aprender a percibir el mundo.

Lo que es fácil pasar por alto: estos no son simplemente casos de humanos proporcionando verdad fundamental. Son casos de humanos construyendo conjuntos de datos que incorporan decisiones perceptuales, culturales y operativas específicas —decisiones que solo se vuelven visibles más tarde, cuando los modelos fallan de maneras predecibles.

Cómo las Decisiones de Anotación Se Convierten en Sesgos del Modelo

El encuadre estándar trata la anotación como recopilación de datos: los humanos observan el mundo y registran lo que ven. El encuadre más preciso es que la anotación es *diseño de conjuntos de datos*: los humanos deciden qué categorías usar, dónde trazar los límites, qué casos límite incluir y cómo resolver la ambigüedad —y todas esas decisiones conforman lo que el modelo entrenado percibirá.

El Caso de ImageNet

ImageNet {cite}`russakovsky2015imagenet` es el conjunto de datos más consecuente en la historia de la visión por computadora. Su conjunto de etiquetas deriva de los synsets de WordNet, elegidos principalmente por ser numerosos y semánticamente distintos. Varias consecuencias de esta elección de diseño emergieron años después:

- **Las categorías de personas codificaban asociaciones demográficas.** Las primeras versiones de las etiquetas de synset de ImageNet para personas incluían muchas que ahora se considerarían despectivas o sesgadas, reflejando tanto la fuente histórica de WordNet como las decisiones implícitas de la fuerza laboral de anotación sobre qué etiquetas aplicar a qué imágenes {cite}`yang2020towards`. Las etiquetas aplicadas a imágenes de personas codificaban asociaciones de raza, género y clase que se propagaron directamente a las representaciones del modelo.
- **Taxonomía de especies de grano fino, de casi todo lo demás de grano grueso.** ImageNet puede distinguir 120 razas de perros pero colapsa una enorme variación en herramientas, vehículos, alimentos y muebles en categorías únicas. Esto fue consecuencia de seguir la estructura de WordNet, no una elección deliberada sobre lo que importa. Los modelos entrenados en ImageNet exhiben la misma precisión asimétrica.

- **Valores visuales predeterminados del mundo occidental de habla inglesa.** Las imágenes se recopilaron principalmente de Flickr y búsquedas en internet usando consultas en inglés. La distribución resultante se inclina fuertemente hacia el entorno visual y los objetos culturales de los países ricos de habla inglesa.

Ninguno de estos fue un error. Fueron decisiones de diseño de anotación tomadas rápidamente a escala, a menudo por personas que no anticiparon cómo se usaría el conjunto de datos. La lección no es que ImageNet debería haberse construido diferente (aunque así debería haber sido), sino que **el diseño de la anotación es diseño del modelo**, y debería tratarse con el mismo cuidado.

:::{admonition} El esquema de anotación es una teoría del mundo :class: note

Todo taxonomía de etiquetas hace afirmaciones sobre qué distinciones importan. Elegir separar "coche" de "camión" mientras se colapsan todos los sedanes en una clase es una afirmación teórica sobre qué distinciones son semánticamente relevantes. Elegir anotar "persona" como una única clase independientemente de la pose, la ropa o la actividad es una afirmación teórica diferente. Los modelos entrenados en estos esquemas harán las mismas distinciones, y no más —no generalizarán más allá de las categorías para las que fueron entrenados a distinguir. :::

Anotación de Clasificación de Imágenes

La tarea de anotación de visión por computadora más simple es asignar una o más etiquetas a una imagen completa.

Jerarquía de etiquetas. La etiqueta "perro" es hija de "animal" en una jerarquía semántica. Los modelos entrenados en taxonomías planas pueden no generalizar bien entre niveles de abstracción. ImageNet usa una jerarquía basada en synsets que permite la evaluación en múltiples niveles de especificidad.

Ambigüedad multietiqueta. Una escena urbana puede contener simultáneamente un coche, una persona, una bicicleta y un semáforo. Decidir qué etiquetas incluir requiere directrices claras sobre los umbrales de relevancia.

Distribuciones de cola larga. Los conjuntos de datos de imágenes naturales siguen una ley de potencia: pocas categorías son extremadamente comunes; la mayoría son raras. El aprendizaje activo es particularmente valioso para las categorías de cola larga donde el muestreo aleatorio produce solo un puñado de ejemplos.

Detección de Objetos: Anotación con Cajas Delimitadoras

La detección de objetos requiere que los anotadores dibujen cajas delimitadoras alineadas con los ejes alrededor de instancias de cada clase de objeto. Esto introduce requisitos de precisión geométrica y casos límite significativos.

Métricas de calidad de anotación:

IoU (Intersección sobre la Unión) mide la superposición entre una caja anotada y una caja de referencia:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

IoU $\geq$ 0,5 es el umbral estándar para una detección "correcta" en PASCAL VOC; COCO {cite}`lin2014microsoft` usa un rango de umbrales de 0,5 a 0,95, lo cual es considerablemente más exigente e informativo.

Casos límite de anotación que deben resolverse en las directrices: - Objetos ocluidos: ¿anotar la porción visible o extrapolar la extensión completa? - Objetos truncados (parcialmente fuera del fotograma): ¿incluir o excluir? - Regiones de multitud: ¿usar una anotación especial de "multitud" o anotar instancias individuales?

Cada una de estas decisiones cambia lo que significa "detección correcta" —y, por tanto, cambia para qué se entrena el modelo.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(boxA, boxB):
    """Compute IoU between two boxes [x1, y1, x2, y2]."""
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])
    inter_area = max(0, xB - xA) * max(0, yB - yA)
    boxA_area = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    boxB_area = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    union_area = boxA_area + boxB_area - inter_area
    return inter_area / (union_area + 1e-6)

ref_box  = [1.0, 1.0, 4.0, 4.0]
ann1_box = [1.1, 0.9, 4.1, 4.2]   # close
ann2_box = [0.5, 0.5, 3.5, 3.8]   # less precise

print(f"IoU (ann1 vs ref):  {compute_iou(ann1_box, ref_box):.3f}")
print(f"IoU (ann2 vs ref):  {compute_iou(ann2_box, ref_box):.3f}")
print(f"IoU (ann1 vs ann2): {compute_iou(ann1_box, ann2_box):.3f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(0, 5); ax.set_ylim(0, 5); ax.set_aspect('equal')
ax.add_patch(patches.Rectangle(
    (ref_box[0], ref_box[1]), ref_box[2]-ref_box[0], ref_box[3]-ref_box[1],
    linewidth=2.5, edgecolor='#2b3a8f', facecolor='none', label='Reference'))
ax.add_patch(patches.Rectangle(
    (ann1_box[0], ann1_box[1]), ann1_box[2]-ann1_box[0], ann1_box[3]-ann1_box[1],
    linewidth=2, edgecolor='#0d9e8e', facecolor='none', linestyle='--',
    label=f'Ann1 (IoU={compute_iou(ann1_box, ref_box):.2f})'))
ax.add_patch(patches.Rectangle(
    (ann2_box[0], ann2_box[1]), ann2_box[2]-ann2_box[0], ann2_box[3]-ann2_box[1],
    linewidth=2, edgecolor='#e05c5c', facecolor='none', linestyle=':',
    label=f'Ann2 (IoU={compute_iou(ann2_box, ref_box):.2f})'))
```

```
ax.legend(fontsize=10)
ax.set_title("Bounding Box Agreement (IoU)", fontsize=12)
plt.tight_layout()
plt.savefig('bbox_iou.png', dpi=150)
plt.show()
```

Segmentación Semántica y de Instancias

La anotación de segmentación requiere asignar una etiqueta de clase a cada píxel de una imagen —uno de los tipos de anotación más costosos.

Segmentación semántica: Cada píxel pertenece a una clase semántica (carretera, cielo, persona, coche). Todos los píxeles de la misma clase comparten la misma etiqueta, independientemente del objeto individual al que pertenezcan.

Segmentación de instancias: Cada instancia individual de objeto obtiene una etiqueta única. Una multitud de 20 personas se convierte en 20 máscaras distintas.

Segmentación panóptica: Combina ambas: las clases "cosa" (objetos contables) tienen máscaras de instancia; las clases "material" (carretera, cielo) tienen máscaras semánticas.

Anotación asistida por SAM: El Modelo Segment Anything (SAM) de Meta {cite}`kirillov2023segment` genera máscaras de segmentación de alta calidad a partir de un único clic sobre un punto. Los anotadores hacen clic en el centro de un objeto; SAM propone una máscara; el anotador acepta o corrige. Los autores de SAM reportan aceleraciones del motor de anotación de aproximadamente 6,5 veces respecto al etiquetado basado en polígonos; las ganancias varían según el tipo de escena y la herramienta de anotación.

SAM representa un cambio más amplio: **el papel del anotador cambia de dibujar a revisar**. Esto tiene implicaciones para la calidad de la anotación más allá de la velocidad. Cuando los anotadores dibujan, su atención está comprometida durante todo el proceso. Cuando los anotadores revisan y hacen clic en "aceptar", hay evidencia de que pasan por alto errores más fácilmente —una versión del sesgo de automatización específica del contexto de anotación.

Aprendizaje Activo para Visión por Computadora

El aprendizaje activo es particularmente valioso en visión por computadora porque: 1. Las imágenes son de alta dimensionalidad y ricas en características —las representaciones de modelos preentrenados llevan señales fuertes para la estimación de incertidumbre 2. Los grandes grupos no etiquetados son baratos (fotos, fotogramas de vídeo) 3. La anotación (especialmente la segmentación) es costosa y no puede crowdsourcing fácilmente en dominios especializados

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
```

```
X, y = make_classification(n_samples=3000, n_features=50, n_informative=25,
                           n_classes=5, n_clusters_per_class=2, random_state=42)
X_train, y_train = X[:2500], y[:2500]
X_test,  y_test  = X[2500:], y[2500:]

def margin_uncertainty(model, X_pool):
    probs = model.predict_proba(X_pool)
    sorted_p = np.sort(probs, axis=1)
    return sorted_p[:, -2] - sorted_p[:, -1]  # most negative = most uncertain

n_init = 50
results = {'active': [], 'random': []}
label_counts = list(range(50, 401, 30))

for strategy in ['active', 'random']:
    labeled = list(rng.choice(len(X_train), n_init, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]

    for target in label_counts:
        while len(labeled) < target and unlabeled:
            if strategy == 'active' and len(labeled) >= 10:
                model_temp = LogisticRegression(max_iter=300).fit(
                    X_train[labeled], y_train[labeled])
                margins = margin_uncertainty(model_temp, X_train[unlabeled])
                idx = int(np.argmin(margins))
            else:
                idx = rng.integers(0, len(unlabeled))
            labeled.append(unlabeled.pop(idx))

        clf = LogisticRegression(max_iter=300).fit(X_train[labeled], y_train[labeled])
        results[strategy].append(accuracy_score(y_test, clf.predict(X_test)))

plt.figure(figsize=(7, 4))
plt.plot(label_counts, results['active'], 'o-', color='#2b3a8f',
         linewidth=2, label='Active (margin sampling)')
plt.plot(label_counts, results['random'], 's--', color='#e05c5c',
         linewidth=2, label='Random baseline')
plt.xlabel("Labeled training images", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning for 5-Class Image Classification", fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('cv_active_learning.png', dpi=150)
plt.show()
```

Aprendizaje Semisupervisado con Guía Humana

La gran cantidad de datos visuales no etiquetados disponibles hace que el aprendizaje semisupervisado sea particularmente atractivo para la visión por computadora.

Autoentrenamiento / pseudoetiquetado: Entrenar un modelo con datos etiquetados; usar predicciones de alta confianza sobre datos no etiquetados como pseudoetiquetas; reentrenar. La pregunta de diseño crítica es el umbral de confianza. Un umbral bajo incorpora más ejemplos pero introduce ruido; un umbral alto deja la mayor parte del grupo no etiquetado sin usar. La participación humana puede guiar este umbral —los anotadores verifican una muestra de ejemplos pseudoetiquetados para calibrarlo.

FixMatch y regularización de consistencia: Estos métodos entrenan modelos para producir predicciones consistentes bajo aumentación. La clave del HITL: los humanos no son consultados solo para las etiquetas sino también para el **diseño de la aumentación** —¿qué invariancias debería aprender el modelo? Un modelo para imágenes médicas debería ser invariante a la rotación y el brillo pero no a la escala; un modelo para detección de texto no debería hacerse invariante a la distorsión de perspectiva. Estas elecciones específicas del dominio requieren experiencia humana, y equivocarse en ellas degrada sustancialmente el aprendizaje semisupervisado.

Aprendizaje activo semisupervisado: La combinación más eficiente: el aprendizaje activo concentra las etiquetas humanas donde la incertidumbre del modelo es mayor; el autoentrenamiento autoetiqueta la cola de alta confianza. El esfuerzo humano se concentra donde es más valioso, y el modelo arranca con el resto. En cada ciclo, una auditoría humana de una muestra aleatoria de pseudoetiquetas proporciona un control de calidad sin requerir una revisión completa.

Anotación de Vídeo

La anotación de vídeo multiplica los desafíos de la anotación de imágenes por el tiempo:

- **Seguimiento:** Los objetos deben identificarse entre fotogramas. Los anotadores etiquetan fotogramas clave; los algoritmos de seguimiento interpolan entre ellos. Los fallos de seguimiento —oclusión, reentrada, movimiento rápido— requieren reetiquetado humano a tasas más altas que el seguimiento en estado estacionario.
- **Consistencia temporal:** Los límites trazados en el fotograma t deben ser espacialmente consistentes con el fotograma $t + 1$. Las anotaciones inconsistentes son una señal de entrenamiento que dice al modelo que los objetos saltan discontinuamente —una forma de ruido de anotación particularmente dañina para los modelos de detección.
- **Escalabilidad:** Un vídeo de 1 hora a 30 fps son 108.000 fotogramas. La anotación completa es impracticable; las estrategias de muestreo deben diseñarse cuidadosamente para garantizar que los eventos raros (casos límite, casi-colisiones, escenarios de fallo) no se excluyan sistemáticamente.

Las herramientas modernas de anotación de vídeo admiten **seguimiento inteligente** que propaga las anotaciones entre fotogramas y señala los fotogramas donde la confianza del seguimiento cae por debajo de un umbral, pidiendo al anotador que vuelva a verificar. Esta es una aplicación directa de la idea del aprendizaje activo a la propia tubería de anotación: la herramienta consulta al anotador exactamente donde su interpolación es incierta.

El problema del evento raro en sistemas autónomos. Para aplicaciones donde las consecuencias de los eventos raros son catastróficas —conducción autónoma, navegación

de UAVs— la distribución de fotogramas vistos en operación normal está gravemente desajustada con la distribución de fotogramas que más importan. Un conjunto de datos construido muestreando uniformemente imágenes de conducción contendrá millones de fotogramas de "no está pasando nada interesante" y un puñado de los fotogramas de casi-accidente, iluminación inusual y sensor degradado que realmente importan para la seguridad. El aprendizaje activo HITL que identifica y prioriza tales fotogramas no es un truco de eficiencia —es un requisito de seguridad.

> **See also:** Conjunto de datos ImageNet: {cite}`russakovsky2015imagenet`. Sesgo de etiquetas en ImageNet: {cite}`yang2020towards`. Benchmark COCO: {cite}`lin2014microsoft`. SAM (Segment Anything): {cite}`kirillov2023segment`. Aprendizaje activo de núcleo para visión por computadora: {cite}`sener2018active`.

HITL en Salud y Ciencia

La atención sanitaria y la ciencia representan dos de los dominios donde el HITL ML es más trascendente y más debatido. Los riesgos son elevados: un diagnóstico de cáncer no detectado o una diana farmacológica deficiente tienen un coste humano real. La anotación requiere una experiencia rara y costosa. Las exigencias regulatorias condicionan lo que los modelos pueden hacer y cómo deben validarse. Y a diferencia del PLN, donde el problema de anotación es parcialmente de construcción social, aquí a menudo existe una verdad fundamental genuina —un tumor está presente o no lo está— aunque ningún observador individual pueda determinarlo de manera fiable.

El encuadre dominante en la cobertura popular es "IA contra humanos": ¿reemplazará la IA a los radiólogos? Este encuadre es erróneo de una manera que importa. La pregunta real es qué forma de colaboración humano-IA produce mejores resultados que cualquiera de los dos por separado, y cómo construir sistemas que permitan esa colaboración en lugar de perturbarla.

Análisis de Imágenes Médicas

Las imágenes médicas —radiología (rayos X, TC, RM), patología (muestras de tejido), dermatología, oftalmología— es el dominio donde la IA médica ha avanzado más rápidamente.

Requisitos de los Anotadores Expertos

La anotación de imágenes médicas generalmente requiere médicos con formación de subespecialidad específica. Esto hace que la anotación sea:

- **Lenta:** Los especialistas tienen tiempo limitado; la anotación compite con las obligaciones clínicas
- **Costosa:** Los costes oscilan entre decenas y cientos de dólares por caso anotado, según la subespecialidad, la modalidad y la complejidad de la tarea
- **Variable:** Incluso los especialistas difieren, especialmente en casos límite —un hecho que a menudo se trata como un problema pero que en realidad es informativo

Variabilidad entre Radiólogos

La variabilidad entre lectores está bien documentada en radiología. Para la interpretación de radiografías de tórax, el desacuerdo entre lectores es sustancial —en el estudio CheXNet, cuatro radiólogos etiquetaron el mismo conjunto de pruebas de detección de neumonía con puntuaciones F1 que abarcaban aproximadamente 12 puntos porcentuales {cite}`rajpurkar2017chexnet`, lo que refleja una incertidumbre diagnóstica genuina en los casos límite. Para la detección de nódulos en TC pulmonar, la variabilidad intra-lector (el mismo lector, el mismo caso, día diferente) puede ser tan grande como la variabilidad inter-lector.

Esta variabilidad no es solo ruido —refleja una incertidumbre diagnóstica genuina. Los modelos entrenados sobre las anotaciones de un único radiólogo pueden aprender los sesgos específicos de ese médico en lugar de la patología subyacente.

:::{admonition} La Controversia de CheXNet como Lección de HITL :class: note

Cuando Rajpurkar et al. afirmaron que su modelo CheXNet "superaba el rendimiento del radiólogo" en la detección de neumonía, la afirmación fue inmediatamente cuestionada por la comunidad radiológica {cite}`yu2022assessing`. Parte de la controversia fue sobre el conjunto de pruebas específico y la comparación de radiólogos. Pero un problema más profundo era metodológico: la referencia de "rendimiento del radiólogo" usaba lectores individuales bajo presión de tiempo, mientras que la radiología clínica generalmente implica consulta, comparación con imágenes previas y acceso a contexto clínico —nada de lo cual tenía el modelo.

La lección no es que el modelo fuera bueno o malo, sino que **las comparaciones de rendimiento requieren especificar la configuración HITL**. Un modelo que supera a un radiólogo individual leyendo a ciegas puede seguir siendo menos preciso que un radiólogo que usa la salida del modelo como segunda opinión. Estos son sistemas diferentes con diferentes modos de error, y agregan de manera diferente. :::

:::{admonition} Etiquetas suaves en medicina :class: important

Varios proyectos de IA médica han pasado a usar **etiquetas suaves** que reflejan la distribución de opiniones expertas en lugar de una sola etiqueta "estándar dorado". Una radiografía de tórax etiquetada como 60% neumonía / 40% atelectasia por un panel de radiólogos lleva más información que una elección binaria forzada. Los modelos entrenados con tales distribuciones muestran mejor calibración y una cuantificación de incertidumbre más apropiada —y esa incertidumbre es clínicamente significativa, ya que indica al clínico cuándo consultar, no solo lo que el modelo piensa. :::

Aprendizaje Activo para Enfermedades Raras

El aprendizaje activo es especialmente valioso para enfermedades raras y patologías raras, donde incluso un gran grupo no etiquetado contiene solo un pequeño número de casos positivos. El muestreo aleatorio estándar desperdiciaría el tiempo del experto etiquetando sobre todo casos negativos.

El aprendizaje activo basado en incertidumbre selecciona naturalmente los casos límite donde el modelo es incierto —los cuales, para condiciones raras, tienden a ser los casos positivos y los negativos límite. Esto concentra el tiempo del especialista donde es más valioso. La combinación de entrenamiento desequilibrado de clases (con `class_weight='balanced'` o similar) y selección basada en incertidumbre es práctica estándar para las tareas de detección de patología rara.

Anotación Clínica de PLN

Las historias clínicas electrónicas (HCE) contienen una enorme riqueza de texto narrativo clínico: notas médicas, informes de alta, informes radiológicos, informes de patología. Extraer información estructurada de este texto requiere PLN —y un PLN de alta calidad requiere datos de entrenamiento anotados.

Tareas comunes de anotación clínica de PLN: - NER clínico: Identificación de medicamentos, dosis, diagnósticos, procedimientos y síntomas en texto - **Detección de negación:** "Sin evidencia de neumonía" frente a "Neumonía confirmada" —una distinción crítica que es sorprendentemente difícil - **Razonamiento temporal:** Distinguir condiciones actuales de antecedentes ("antecedentes de infarto de miocardio, acudió con dolor torácico") - **Desidentificación:** Eliminar Información Sanitaria Protegida (PHI) para permitir el intercambio de datos

La **desidentificación de PHI** es tanto una tarea de anotación como un requisito de gobernanza de datos. Bajo HIPAA (EE. UU.) y GDPR (UE), los datos de salud no pueden compartirse sin eliminar o anonimizar los identificadores del paciente. Las herramientas de desidentificación automatizadas existen pero son imperfectas; la revisión humana de las salidas automatizadas es práctica estándar, y el perfil de riesgo es asimétrico: los falsos negativos (PHI no detectada) crean exposición legal, lo que hace necesarios umbrales conservadores.

i2b2 / n2c2 como Plantilla

Las iniciativas de tarea compartida i2b2 (Informática para Integrar la Biología y el Cabecero) y la sucesora n2c2 (Desafíos Clínicos Nacionales de PLN) han publicado una serie de conjuntos de datos de PLN clínico anotados por expertos. Estos ilustran tanto el potencial como el coste: los esfuerzos de anotación típicamente implican equipos de expertos en el dominio clínico trabajando durante varios meses, anotando cientos de documentos por desafío. Los conjuntos de datos n2c2 han catalizado un rápido progreso precisamente porque resolvieron el problema de gobernanza del intercambio de datos (desidentificación + acuerdos institucionales), no solo el problema de anotación.

Consideraciones Regulatorias

La IA médica está sujeta a supervisión regulatoria en la mayoría de las jurisdicciones.

FDA (Estados Unidos): El Software como Dispositivo Médico (SaMD) basado en IA/ML requiere aprobación previa a la comercialización o autorización de comercialización. El Plan de Acción de IA/ML de la FDA de 2021 enfatiza los **planes de control de cambios predeterminados** —documentar cómo se actualizará el modelo y cómo se validarán esas actualizaciones antes del despliegue. Un modelo que aprende continuamente de la retroalimentación clínica es, bajo este marco, un dispositivo diferente después de cada actualización y puede requerir revalidación.

Marcado CE (Europa): Los dispositivos médicos, incluyendo los sistemas de IA, deben cumplir con el Reglamento de Dispositivos Médicos (MDR). El MDR exige evaluación clínica, vigilancia poscomercialización y documentación de los datos utilizados para el entrenamiento y la validación.

Implicación clave de HITL: Los marcos regulatorios requieren documentación clara de los procesos de anotación, las calificaciones de los anotadores, la fiabilidad entre evaluadores y cualquier cambio en los datos de entrenamiento. Esto no es burocracia adicional —es la pista de auditoría que permite a un clínico entender qué datos de entrenamiento produjeron el comportamiento actual del modelo, y es legalmente obligatorio. Las tuberías HITL que tratan la anotación como un subproceso informal crean un riesgo regulatorio que típicamente se hace visible solo en el peor momento.

Anotación de Datos Científicos

Más allá de la atención sanitaria, el HITL ML desempeña un papel creciente y subestimado en la investigación científica, donde el desafío de anotación a menudo combina experiencia en el dominio con escala.

Astronomía: Galaxy Zoo

Galaxy Zoo {cite}`lintott2008galaxy` externalizó la clasificación morfológica de galaxias del Sloan Digital Sky Survey a científicos ciudadanos. El proyecto original recopiló más de 40 millones de clasificaciones de más de 100.000 voluntarios, demostrando que la clasificación de imágenes científicas a gran escala mediante crowdsourcing es factible cuando la tarea puede descomponerse en preguntas simples respondibles sin formación especializada ("¿Esta galaxia es suave o presenta estructura?").

La experiencia de Galaxy Zoo produjo dos hallazgos metodológicos importantes. Primero, la concordancia entre científicos ciudadanos y astrónomos profesionales era alta para los casos claros y divergía sistemáticamente en los casos límite —precisamente los casos donde la distinción importa científicamente. La solución no fue descartar los datos de ciencia ciudadana en los casos límite, sino tratar la distribución de respuestas de los voluntarios como una etiqueta suave que codificaba la ambigüedad morfológica genuina. Segundo, el clasificador entrenado con las etiquetas de Galaxy Zoo superó a los modelos entrenados para reproducir las etiquetas de cualquier experto individual, porque la distribución colectiva capturaba una incertidumbre visual real que la elección forzada de un único experto colapsaba.

Genómica: Clasificación de Patogenicidad

Anotar variantes genómicas —decidir si una variante es patogénica, benigna o de significado incierto— es un problema de PLN y juicio experto de alto riesgo. Las bases de datos de variantes clínicas como ClinVar agregan interpretaciones expertas de múltiples laboratorios colaboradores, y el desacuerdo entre laboratorios es común. El aprendizaje activo se usa para priorizar qué variantes requieren revisión experta completa (búsqueda bibliográfica, evaluación de evidencia funcional) frente a cuáles pueden clasificarse automáticamente con la evidencia existente. El resultado es una tubería híbrida donde la mayoría de las variantes se manejan por lógica automatizada, un subconjunto requiere revisión experta, y los casos más difíciles se marcan para el consenso multilaboratorio.

Clima y Ciencias de la Tierra

El etiquetado de imágenes satelitales para cambios en el uso del suelo, deforestación, extensión de glaciares y trayectorias de tormentas involucra a expertos en teledetección y, cada vez más, a plataformas de ciencia ciudadana. El principal desafío HITL en este dominio es temporal: las etiquetas hechas hoy pueden volverse obsoletas a medida que el mundo cambia, y la verificación de la verdad fundamental (encuestas de campo) es costosa y logísticamente limitada. El aprendizaje activo que prioriza las imágenes donde la predicción del modelo difiere de los a priori físicos (por ejemplo, predecir deforestación en una región que se sabe que está protegida) es una forma práctica de dirigir los escasos recursos de verificación de campo.

Neurociencia: Connectómica

Reconstruir circuitos neurales a partir de imágenes de microscopio electrónico —connectómica— requiere anotación a nivel de píxel de membranas neuronales en enormes pilas de imágenes. El proyecto Eyewire gamificó esta tarea, involucrando a decenas de miles de jugadores en

el trazado de neuronas a través de volúmenes de imágenes 3D. El diseño de la gamificación resolvió un problema específico de HITL: la tarea requiere atención sostenida y razonamiento espacial durante sesiones largas, lo que provoca una degradación de la calidad en la anotación tradicional. Dividir la tarea en segmentos de juego con mecánicas sociales mantuvo el compromiso y la calidad de los anotadores a escalas que la anotación profesional no puede lograr.

Gestión de Anotadores Especialistas

Cuando la anotación requiere experiencia rara, los enfoques habituales de crowdsourcing (Capítulo 13) no se aplican.

La tensión fundamental es que las personas que pueden producir las anotaciones de mayor calidad son también las personas cuyo tiempo es más valioso y más limitado. Cada decisión de diseño en una tubería de anotación especialista debería evaluarse respecto a la pregunta: ¿hace esto el mejor uso del escaso tiempo del experto?

Lo que esto significa en la práctica:

- **Pre-anotar agresivamente.** Usar anotadores de nivel inferior, modelos automatizados o sistemas basados en reglas para generar candidatos que el especialista revisa y corrige en lugar de crear desde cero. El juicio del especialista es el cuello de botella; proporcionarles una pre-anotación para corregir es más rápido que pedirles que anoten desde una pantalla en blanco, siempre que la calidad de la pre-anotación sea suficiente para que la corrección no sea más lenta que empezar de nuevo.

- **Diseñar para la atención del experto, no el rendimiento.** Las interfaces de anotación optimizadas para alto rendimiento (decisiones binarias rápidas, atajos de teclado, visualización mínima) son apropiadas para el crowdsourcing. La anotación especialista a menudo se beneficia de interfaces más ricas: comparación lado a lado con casos previos, fácil acceso a materiales de referencia, campos de confianza de anotación y la capacidad de marcar un caso para discusión. Esto ralentiza las anotaciones individuales pero mejora la calidad y reduce la necesidad de re-anotación.

- **Rastrear los patrones individuales de los anotadores explícitamente.** Con un pequeño grupo de especialistas, es factible e importante rastrear la tasa de concordancia de cada anotador con el panel, señalar los casos que parecen inconsistentes con su historial propio y discutirlos en sesiones regulares de calibración. Esto no es vigilancia —es el mismo proceso de calidad que la medicina clínica usa para la revisión del rendimiento, y los especialistas generalmente responden bien cuando se enmarca como mejora de la calidad compartida en lugar de evaluación.

- **El diseño de la sesión importa.** La anotación médica es cognitivamente exigente. La evidencia de radiología y patología sugiere que las tasas de error aumentan de manera mensurable después de aproximadamente 90 minutos de lectura continua, y que descansos de incluso unos pocos minutos pueden restablecer parcialmente la atención. Las interfaces de anotación que aplican avisos de descanso (sin posibilidad de ignorarlos) son una decisión de diseño HITL simple con impacto real en la calidad.

Una Tubería de Aprendizaje Activo HITL para Imágenes Médicas

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Simulate a rare-pathology detection task
# 8% positive class (e.g., rare pathology)
X, y = make_classification(
    n_samples=5000, n_features=100,
    n_informative=20, n_redundant=10,
    weights=[0.92, 0.08],
    random_state=42
)
X_train, y_train = X[:4000], y[:4000]
X_test,  y_test  = X[4000:], y[4000:]

print(f"Training set positive prevalence: {y_train.mean():.1%}")

def run_medical_al(strategy, n_initial=50, budget=300):
    labeled = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]
    aucs = []

    while len(labeled) < n_initial + budget:
        model = LogisticRegression(max_iter=500, class_weight='balanced')
        model.fit(X_train[labeled], y_train[labeled])

        if len(labeled) % 30 == 0:
            preds = model.predict_proba(X_test)[:, 1]
            aucs.append(roc_auc_score(y_test, preds))

        X_pool = X_train[unlabeled]
        if strategy == 'uncertainty' and len(labeled) >= 10:
            probs = model.predict_proba(X_pool)
            entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
            q = int(np.argmax(entropy))
        else:
            q = rng.integers(0, len(unlabeled))

        labeled.append(unlabeled.pop(q))

    return np.array(aucs)

aucs_al  = run_medical_al('uncertainty')
aucs_rnd = run_medical_al('random')
label_counts = np.arange(len(aucs_al)) * 30 + 50
```

```
plt.figure(figsize=(7, 4))
plt.plot(label_counts, aucs_al,  'o-',  color='#2b3a8f', linewidth=2, label='Uncertainty AL')
plt.plot(label_counts, aucs_rnd, 's--', color='#e05c5c', linewidth=2, label='Random baseline'
plt.xlabel("Expert labels obtained", fontsize=12)
plt.ylabel("AUROC", fontsize=12)
plt.title("Active Learning for Rare Pathology Detection", fontsize=13)
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('medical_al.png', dpi=150)
plt.show()

# Show how AL preferentially selects positive examples
# by checking which class the queried examples belong to
n_init = 50
labeled_al  = list(rng.choice(len(X_train), n_init, replace=False))
labeled_rnd = labeled_al.copy()
unlabeled_al  = [i for i in range(len(X_train)) if i not in labeled_al]
unlabeled_rnd = unlabeled_al.copy()

model = LogisticRegression(max_iter=500, class_weight='balanced')
model.fit(X_train[labeled_al], y_train[labeled_al])
probs = model.predict_proba(X_train[unlabeled_al])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
top50_al  = [unlabeled_al[i] for i in np.argsort(entropy)[-50:]]
top50_rnd = list(rng.choice(unlabeled_rnd, 50, replace=False))

pos_rate_al  = y_train[top50_al].mean()
pos_rate_rnd = y_train[top50_rnd].mean()
print(f"\nPositive rate in next 50 queries:")
print(f"  Uncertainty AL: {pos_rate_al:.1%}  (base rate: {y_train.mean():.1%})")
print(f"  Random:         {pos_rate_rnd:.1%}")
print(f"  AL queries {pos_rate_al/y_train.mean():.1f}x more positives than random")
```

See also: Crowdsourcing en Galaxy Zoo: {cite}`lintott2008galaxy`. Rendimiento del radiólogo con CheXNet: {cite}`rajpurkar2017chexnet`. Calidad de radiografías y diagnóstico asistido por IA: {cite}`yu2022assessing`. Metodología de anotación de PLN clínico: {cite}`pustejovsky2012natural`. Para la guía del plan de acción IA/ML de la FDA, véase la documentación publicada por la FDA (2021).

Plataformas y Herramientas de Anotación

La plataforma de anotación es el entorno donde la retroalimentación humana se convierte en datos. Una buena plataforma aumenta el rendimiento, reduce los errores, mantiene los controles de calidad y hace que la tubería de anotación sea manejable a escala. Elegir la plataforma adecuada —y saber cuándo construir frente a cuándo comprar— es una decisión trascendente en cualquier proyecto HITL.

El Panorama de las Plataformas de Anotación

El mercado de herramientas de anotación ha crecido y madurado sustancialmente en los últimos años, impulsado por la demanda empresarial de datos de entrenamiento para aprendizaje automático. Las herramientas abarcan desde servicios completamente gestionados hasta marcos de código abierto autoalojados.

Plataformas de Código Abierto

Label Studio es la plataforma de anotación de código abierto más popular, compatible con datos de texto, imágenes, audio, vídeo y series temporales a través de una configuración de tareas unificada basada en XML. Puede autoalojarse e integrarse con backends de aprendizaje automático para el aprendizaje activo. Principales fortalezas: flexibilidad, soporte de la comunidad y la capacidad de incrustar predicciones de aprendizaje automático personalizadas para la pre-anotación.

Prodigy (de los creadores de spaCy) es una herramienta de anotación muy orientada al flujo de trabajo, diseñada para tareas de PLN. Su arquitectura de streaming envía los ejemplos de uno en uno y admite el aprendizaje activo de forma nativa. La interfaz de anotación es mínima pero rápida —diseñada para maximizar el rendimiento de la anotación. Prodigy es software de pago pero ampliamente utilizado en la investigación de PLN.

CVAT (Computer Vision Annotation Tool) es la herramienta de código abierto líder para la anotación de visión por computadora, con sólido soporte para detección, segmentación y anotación de vídeo. Desarrollado originalmente en Intel, CVAT admite algoritmos de seguimiento, anotación de esqueletos e integraciones de algoritmos de terceros.

Doccano está orientado a las tareas de etiquetado de secuencias (NER, extracción de relaciones, clasificación de texto). Su sencilla interfaz web lo hace accesible para equipos sin recursos dedicados de ingeniería de datos.

Plataformas Comerciales

Scale AI ofrece servicios de anotación gestionados de extremo a extremo: fuerza laboral humana, gestión de calidad e integración de API. Especialmente potente para conducción autónoma, robótica y anotación 3D compleja. El precio se basa en la complejidad y el volumen de las tareas.

Labelbox es una plataforma completa para la curación de datos, la anotación y el etiquetado asistido por aprendizaje automático. Potentes características empresariales: gestión de proyectos, flujos de trabajo de calidad, bucles de retroalimentación del modelo e integraciones con las principales plataformas de aprendizaje automático (SageMaker, Vertex AI, Azure ML).

Appen (anteriormente Figure Eight / CrowdFlower) opera una gran fuerza laboral global de anotación junto con herramientas. Una buena opción cuando el volumen y la gestión de la fuerza laboral son preocupaciones principales.

Surge AI se centra en anotadores expertos y es potente para tareas que requieren conocimiento del dominio o juicio matizado.

Humanloop se especializa en la recopilación de retroalimentación de LLM —anotación de preferencias, recopilación de datos de RLHF y evaluación de modelos.

Comparación de Características de Plataformas de Anotación

Característica	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
Licencia	Código abierto	Comercial	Código abierto	Comercial	Comercial
Alojamiento	Propio / nube	Propio	Propio / nube	Nube	Gestionado
Anotación de texto	□	□	—	□	□
Anotación de imágenes	□	Limitado	□	□	□
Anotación de vídeo	□	—	□	□	□
Integración de aprendizaje activo	□	□	Limitado	□	□
Pre-anotación asistida por ML	□	□	□	□	□
Flujos de trabajo de control de calidad	Básico	Básico	Básico	Avanzado	Avanzado
API / Acceso programático	□	□	□	□	□

Característica	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
Gestión de la fuerza laboral	—	—	—	Limitado	□

Anotación como Código

Un aspecto crítico pero a menudo ignorado de la infraestructura de anotación es el **control de versiones para las anotaciones y los esquemas de anotación**. Tratar la anotación como código significa:

Diseño primero del esquema. La taxonomía de etiquetas y las reglas de anotación se definen en un archivo de configuración versionado (YAML o JSON) antes de que comience la anotación. Los cambios en el esquema crean una nueva versión.

Versionado de las anotaciones. Las anotaciones se almacenan con un enlace a la versión del esquema bajo la que se crearon. Esto permite la auditoría, la reversión y la comparación de anotaciones entre versiones del esquema.

Tuberías reproducibles. La tubería de anotación —desde los datos crudos hasta las etiquetas listas para el entrenamiento— debería ser reproducible a partir del código. Los modelos de pre-anotación, los filtros de calidad, la lógica de agregación y las divisiones de datos deberían registrarse todos.

```yaml
# Example: Label Studio annotation schema (text classification)
label_config: |
  <View>
    <Text name="text" value="$text"/>
    <Choices name="sentiment" toName="text" choice="single">
      <Choice value="positive"/>
      <Choice value="negative"/>
      <Choice value="neutral"/>
      <Choice value="mixed"/>
    </Choices>
  </View>
schema_version: "2.1.0"
task_type: text_classification
guidelines_version: "guidelines_v3.pdf"
```text
Fuente de datos
    |
    v
Grupo no etiquetado --> Plataforma de anotación --> Conjunto de datos etiquetado
    ^                          |                              |
    |                          | (asistido por ML)            v
Estrategia de            <----+                        Ejecución de
consulta de                                            entrenamiento
aprendizaje activo                                          |
    ^                                                       v
    +---------- Modelo entrenado <-------------------- Evaluación
```
```

```
                    |
Despliegue y
monitorización
```

Puntos clave de integración: 1. **Ingesta de datos:** Los datos no etiquetados fluyen automáticamente desde el almacén de datos a la plataforma de anotación 2. **Servicio del modelo:** El mejor modelo actual se despliega en la plataforma de anotación para la pre-anotación y la puntuación del aprendizaje activo 3. **Exportación:** Las anotaciones completadas se exportan en un formato compatible con el marco de entrenamiento (COCO JSON, conjuntos de datos de Hugging Face, etc.) 4. **Bucle de retroalimentación:** Los errores del modelo en producción se enrutan de vuelta a la plataforma de anotación para su corrección

> **See also:** Para la documentación de Label Studio y la integración del aprendizaje activo: https://labelstud.io. Para Prodigy: https://prodi.gy. CVAT: https://cvat.ai. Para una comparación exhaustiva de las herramientas de anotación, véase {cite}`monarch2021human`, Capítulo 7.

Crowdsourcing y Control de Calidad

Cuando las tareas de anotación son suficientemente simples para que las realicen no especialistas, las plataformas de crowdsourcing ofrecen acceso a grandes fuerzas laborales de anotación a demanda y a bajo coste por elemento. Construir conjuntos de datos etiquetados de alta calidad a partir de multitudes requiere un diseño cuidadoso de las tareas, redundancia estratégica y un riguroso control de calidad.

Plataformas de Crowdsourcing

Amazon Mechanical Turk (MTurk) es el mercado de crowdsourcing original, lanzado en 2005. Los trabajadores ("Turkers") completan micro-tareas (HITs) publicadas por solicitantes. Un estudio de 2018 encontró ingresos por hora efectivos medianos para los Turkers de aproximadamente $2/hora —muy por debajo del salario mínimo en muchos países de ingresos altos {cite}`hara2018data`— una preocupación ética abordada más adelante en el Capítulo 15. MTurk es más adecuado para tareas simples con criterios claros y verificables.

Prolific es una plataforma de crowdsourcing académico que impone un estándar de pago mínimo (actualmente alrededor de £9/hora, aproximadamente $11/hora, tal como se indica en las directrices publicadas de Prolific), selecciona participantes por demografía y mantiene un panel de trabajadores que han optado voluntariamente por participar en investigaciones. Preferida para investigaciones en ciencias sociales y tareas que requieren representatividad.

Appen (y similares: Telus International, iMerit) proporciona fuerzas laborales de anotación gestionadas con gestión de calidad, utilizadas para tareas de mayor complejidad y proyectos empresariales.

Comunidades especializadas. Para tareas específicas del dominio, las comunidades de entusiastas del dominio pueden proporcionar anotaciones de alta calidad: Galaxy Zoo para astronomía, eBird para especies de aves, Chess Tempo para la anotación de posiciones de ajedrez.

Diseño de Tareas para el Crowdsourcing

Descomponer las Tareas Complejas

Las tareas complejas deben descomponerse en micro-tareas simples y bien definidas. En lugar de pedir a los trabajadores que anoten exhaustivamente un documento, hágales una pregunta específica a la vez: "¿Contiene esta oración el nombre de una persona?" o "Valore la claridad de esta traducción en una escala del 1 al 5."

Ventajas de la descomposición: - Menor demanda cognitiva por tarea □ menos fatiga, mayor calidad - Cada micro-tarea puede controlarse por calidad por separado - Más fácil de auditar y depurar

La Importancia de las Instrucciones

El predictor más importante de la calidad del crowdsourcing es la calidad de las instrucciones. Las buenas instrucciones de tarea: - Explican el *propósito* de la tarea en una oración - Dan una definición clara y sin ambigüedades de cada categoría - Proporcionan 3–5 ejemplos trabajados (especialmente casos límite) - No son más largas de lo que los trabajadores realmente leerán (< 300 palabras para tareas simples)

Realice un **estudio piloto** (10–50 trabajadores, 20–100 tareas) antes de escalar. Analice los desacuerdos del piloto; la mayoría apuntan a ambigüedades en las instrucciones que pueden corregirse.

Preguntas de Estándar de Referencia

Incruste **preguntas de estándar de referencia** —tareas con respuestas correctas conocidas— a lo largo del lote de tareas. Los trabajadores que fallan las preguntas de referencia por debajo de un umbral se eliminan del proyecto.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(42)

def simulate_gold_screening(n_workers=100, gold_per_batch=5,
                            p_good_worker=0.7, good_acc=0.92,
                            bad_acc=0.55, threshold=0.70):
    """
    Simulate quality screening via gold questions.
    Returns: precision and recall of identifying bad workers.
    """
    worker_quality = rng.choice(['good', 'bad'], size=n_workers,
                                p=[p_good_worker, 1 - p_good_worker])
    results = {'tp': 0, 'fp': 0, 'tn': 0, 'fn': 0}

    for q in worker_quality:
        acc = good_acc if q == 'good' else bad_acc
        n_correct = rng.binomial(gold_per_batch, acc)
        passed = (n_correct / gold_per_batch) >= threshold
        if q == 'bad' and not passed:  results['tp'] += 1
        if q == 'good' and not passed: results['fp'] += 1
        if q == 'bad' and passed:      results['fn'] += 1
        if q == 'good' and passed:     results['tn'] += 1

    tp, fp, fn = results['tp'], results['fp'], results['fn']
    precision = tp / (tp + fp + 1e-6)
    recall    = tp / (tp + fn + 1e-6)
    return precision, recall

# Vary gold question count
```

```
gold_counts = [3, 5, 8, 10, 15, 20]
precisions, recalls = [], []
for g in gold_counts:
    p_list, r_list = [], []
    for _ in range(50):
        p, r = simulate_gold_screening(gold_per_batch=g)
        p_list.append(p); r_list.append(r)
    precisions.append(np.mean(p_list))
    recalls.append(np.mean(r_list))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gold_counts, precisions, 'o-', color='#2b3a8f', label='Precision', linewidth=2)
ax.plot(gold_counts, recalls,    's--', color='#0d9e8e', label='Recall',    linewidth=2)
ax.set_xlabel("Gold questions per batch", fontsize=12)
ax.set_ylabel("Screening performance", fontsize=12)
ax.set_title("Worker Screening via Gold Standard Questions", fontsize=13)
ax.legend(fontsize=11); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('gold_screening.png', dpi=150)
plt.show()

p, r = simulate_gold_screening(gold_per_batch=5)
print(f"5 gold questions: Precision={p:.3f}, Recall={r:.3f}")
p, r = simulate_gold_screening(gold_per_batch=10)
print(f"10 gold questions: Precision={p:.3f}, Recall={r:.3f}")
```

Modelos Estadísticos para la Agregación de Etiquetas

La votación mayoritaria es una referencia natural pero ignora las diferencias en la precisión de los anotadores. Los modelos estadísticos pueden hacerlo mejor.

El Modelo de Dawid-Skene

El **modelo de Dawid-Skene (DS)** {cite}`dawid1979maximum` estima conjuntamente: - La **etiqueta verdadera** z_i para cada elemento i - La **matriz de confusión** $\pi_j^{(k,l)}$ para cada anotador j: la probabilidad de que el anotador j etiquete un elemento con clase verdadera k como clase l

El algoritmo EM itera: - **Paso E:** Dadas las matrices de confusión de los anotadores, calcular la probabilidad posterior de cada etiqueta verdadera - **Paso M:** Dadas las estimaciones de etiquetas de los elementos, actualizar las matrices de confusión de los anotadores

```
import numpy as np
from scipy.special import softmax

def dawid_skene_em(annotations, n_classes, n_iter=20):
    """
    Simplified Dawid-Skene EM for binary classification.
    annotations: dict {item_idx: [(annotator_idx, label), ...]}
    Returns: estimated true labels and annotator accuracies.
```

```
    """
    items = sorted(annotations.keys())
    n_items = len(items)
    annotators = sorted({a for anns in annotations.values() for a, _ in anns})
    n_annotators = len(annotators)
    ann_idx = {a: i for i, a in enumerate(annotators)}

    # Initialize: majority vote
    T = np.zeros((n_items, n_classes))  # soft label estimates
    for i, item in enumerate(items):
        for _, label in annotations[item]:
            T[i, label] += 1
        T[i] /= T[i].sum()

    # Confusion matrices: shape (n_annotators, n_classes, n_classes)
    PI = np.ones((n_annotators, n_classes, n_classes)) * 0.5

    for _ in range(n_iter):
        # M-step: update confusion matrices
        PI = np.zeros((n_annotators, n_classes, n_classes)) + 1e-6
        for i, item in enumerate(items):
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                for k in range(n_classes):
                    PI[a, k, label] += T[i, k]
        PI /= PI.sum(axis=2, keepdims=True)

        # E-step: update soft label estimates
        T = np.zeros((n_items, n_classes))
        for i, item in enumerate(items):
            log_p = np.zeros(n_classes)
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                log_p += np.log(PI[a, :, label] + 1e-10)
            T[i] = softmax(log_p)

    return {item: T[i] for i, item in enumerate(items)}, PI

# Simulate crowdsourcing scenario
rng = np.random.default_rng(42)
N_ITEMS, N_ANNOTATORS, ANNS_PER_ITEM = 200, 10, 3
true_labels = rng.integers(0, 2, N_ITEMS)
# Annotator accuracies: 3 good (0.9), 4 medium (0.75), 3 noisy (0.6)
accuracies = [0.90]*3 + [0.75]*4 + [0.60]*3

annotations = {}
for i in range(N_ITEMS):
    anns_for_item = []
    chosen = rng.choice(N_ANNOTATORS, ANNS_PER_ITEM, replace=False)
    for a in chosen:
        acc = accuracies[a]
```

```
        label = true_labels[i] if rng.random() < acc else 1 - true_labels[i]
        anns_for_item.append((a, int(label)))
    annotations[i] = anns_for_item

# Run Dawid-Skene
soft_labels, confusion = dawid_skene_em(annotations, n_classes=2, n_iter=30)
ds_preds = {i: int(np.argmax(soft_labels[i])) for i in range(N_ITEMS)}

# Compare with majority vote
mv_preds = {}
for i in range(N_ITEMS):
    votes = [l for _, l in annotations[i]]
    mv_preds[i] = int(np.round(np.mean(votes)))

ds_acc = np.mean([ds_preds[i] == true_labels[i] for i in range(N_ITEMS)])
mv_acc = np.mean([mv_preds[i] == true_labels[i] for i in range(N_ITEMS)])

print(f"Majority vote accuracy:  {mv_acc:.3f}")
print(f"Dawid-Skene accuracy:    {ds_acc:.3f}")
print(f"\nEstimated annotator accuracies (diagonal of confusion matrix):")
for a in range(N_ANNOTATORS):
    est_acc = np.mean([confusion[a, k, k] for k in range(2)])
    print(f"  Annotator {a}: estimated={est_acc:.3f}, true={accuracies[a]:.2f}")
```

MACE

MACE (Estimación de Competencia Multi-Anotador) {cite}`hovy2013learning` es un modelo probabilístico alternativo que representa explícitamente el comportamiento de spam de los anotadores (etiquetado aleatorio) frente a la anotación competente. Un anotador o bien proporciona una etiqueta significativa (con probabilidad $1 - \text{spam}_j$) o bien una etiqueta aleatoria (con probabilidad spam_j). Este modelo de dos componentes suele estar mejor calibrado que el de Dawid-Skene para los escenarios de crowdsourcing donde algunos anotadores son spammers puros.

Redundancia y Estrategia de Agregación

El número óptimo de anotadores por elemento depende de la dificultad de la tarea y la calidad de los anotadores:

- **Tareas fáciles con anotadores cualificados:** 1–2 anotadores por elemento suele ser suficiente
- **Tareas moderadas con anotadores entrenados:** 3 anotadores + votación mayoritaria
- **Tareas difíciles/subjetivas con trabajadores de crowdsourcing:** 5–7 anotadores + Dawid-Skene

La clave: la redundancia es más valiosa cuando la precisión del anotador es baja. Para anotadores con precisión p, la precisión de la votación mayoritaria con n anotadores es:

$$P(\text{VM correcta}) = \sum_{k>n/2}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

Para $p = 0{,}70$, añadir un tercer anotador aumenta la precisión de la votación mayoritaria del 70% al 78%; para $p = 0{,}90$, la ganancia de un tercer anotador es insignificante (del 90% al 97%).

See also: Modelo de Dawid-Skene: {cite}`dawid1979maximum`. MACE: {cite}`hovy2013learning`. Para una revisión exhaustiva del crowdsourcing para PLN: {cite}`snow2008cheap`. Ética del crowdsourcing y pago justo: véase el Capítulo 15.

Evaluación y Métricas

Saber si su sistema HITL está funcionando requiere más que medir la precisión del modelo. Necesita saber si está obteniendo valor de su presupuesto de anotación, si el modelo está realmente mejor alineado con la intención humana, y si la retroalimentación humana adicional seguirá mejorando las cosas. Este capítulo cubre el panorama completo de la evaluación en los entornos HITL.

Métricas Centradas en el Modelo

Las métricas estándar de aprendizaje automático se aplican directamente a los sistemas HITL, con algunos matices importantes.

Métricas de Clasificación

La **exactitud** es apropiada cuando las clases están equilibradas y todos los errores tienen el mismo coste. En los entornos HITL, sin embargo, el conjunto de prueba etiquetado puede estar sesgado por la estrategia de consulta (el aprendizaje activo consulta ejemplos no aleatorios), lo que hace que las estimaciones de exactitud simple sean poco fiables.

La **puntuación F1** es la media armónica de la precisión y la recuperación, apropiada para clases desequilibradas. En los contextos HITL, tanto la precisión como la recuperación pueden importar de manera diferente según la asimetría de coste entre los falsos positivos y los falsos negativos.

AUROC mide la capacidad del modelo para discriminar entre clases independientemente del umbral —importante para las tareas sensibles a la calibración como el cribado médico.

La **calibración** mide qué tan bien se corresponden las probabilidades predichas con las frecuencias empíricas. En los sistemas HITL, los modelos entrenados en conjuntos etiquetados sesgados (del aprendizaje activo) pueden estar mal calibrados incluso cuando son precisos.

Métricas de Modelos Generativos

Para los modelos de lenguaje y los sistemas generativos, la evaluación es fundamentalmente más difícil. Ninguna métrica automática única captura la calidad:

- **BLEU / ROUGE / METEOR:** Métricas basadas en referencias para traducción y resumen. Se correlacionan débilmente con los juicios de calidad humana para la generación de texto largo.
- **Perplejidad:** Mide qué tan bien el modelo predice el texto reservado. Es una condición necesaria pero no suficiente para la calidad.

- **BERTScore:** Similitud basada en representaciones respecto a las referencias. Mejor correlación con los juicios humanos que las métricas n-gram.
- **Evaluación humana:** El estándar de oro. Véase la Sección 14.3.

Métricas de Eficiencia de la Anotación

La evaluación HITL también debe medir si la retroalimentación humana se está usando eficientemente.

Curvas de Aprendizaje

Una **curva de aprendizaje** representa el rendimiento del modelo en función del número de ejemplos etiquetados. Una curva de aprendizaje empinada (mejora rápida con pocas etiquetas) indica que la estrategia de anotación está seleccionando ejemplos informativos. Una curva de aprendizaje plana indica que el etiquetado adicional está proporcionando rendimientos decrecientes.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import StratifiedShuffleSplit

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=5000, n_features=30, n_informative=15,
                           weights=[0.8, 0.2], random_state=42)
X_test, y_test = X[4000:], y[4000:]
X_pool, y_pool = X[:4000], y[:4000]

label_sizes = [20, 40, 60, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000]
metrics = {'f1': [], 'auc': []}

for n in label_sizes:
    idx = rng.choice(len(X_pool), n, replace=False)
    clf = LogisticRegression(max_iter=500, class_weight='balanced')
    clf.fit(X_pool[idx], y_pool[idx])
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
    metrics['f1'].append(f1_score(y_test, preds))
    metrics['auc'].append(roc_auc_score(y_test, probs))

# Fit learning curve: performance ≈ a - b/sqrt(n)
from scipy.optimize import curve_fit

def learning_curve_fn(n, a, b):
    return a - b / np.sqrt(n)

popt_f1, _ = curve_fit(learning_curve_fn, label_sizes, metrics['f1'], p0=[0.9, 2])
popt_auc, _ = curve_fit(learning_curve_fn, label_sizes, metrics['auc'], p0=[0.95, 1])
```

```
n_smooth = np.linspace(20, 3000, 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.scatter(label_sizes, metrics['f1'], color='#2b3a8f', zorder=5, s=40)
ax1.plot(n_smooth, learning_curve_fn(n_smooth, *popt_f1), '--', color='#e05c5c',
         label=f'Fit: {popt_f1[0]:.3f} - {popt_f1[1]:.1f}/√n')
ax1.set_xlabel("Labeled examples"); ax1.set_ylabel("F1 score")
ax1.set_title("Learning Curve: F1"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.scatter(label_sizes, metrics['auc'], color='#0d9e8e', zorder=5, s=40)
ax2.plot(n_smooth, learning_curve_fn(n_smooth, *popt_auc), '--', color='#e05c5c',
         label=f'Fit: {popt_auc[0]:.3f} - {popt_auc[1]:.2f}/√n')
ax2.set_xlabel("Labeled examples"); ax2.set_ylabel("AUROC")
ax2.set_title("Learning Curve: AUROC"); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.png', dpi=150)
plt.show()

# Estimate the annotation budget needed to reach a target performance
target_f1 = 0.80
n_needed = (popt_f1[1] / (popt_f1[0] - target_f1)) ** 2
print(f"Estimated labels needed to reach F1={target_f1}: {n_needed:.0f}")
```

Análisis del Retorno de la Inversión (ROI)

El ROI de la retroalimentación humana responde: por cada etiqueta adicional, ¿cuánto mejora el rendimiento del modelo?

$$\text{ROI}(n) = \frac{\Delta\text{rendimiento}(n)}{\text{coste por etiqueta}}$$

A medida que el modelo madura (y se agotan los ejemplos fáciles de aprender), el ROI típicamente disminuye. La implicación práctica: los presupuestos de anotación deberían estar orientados hacia el inicio, con más etiquetas recopiladas en las etapas tempranas cuando el ROI es más alto.

Evaluación Humana

Para los sistemas generativos y las tareas subjetivas, la evaluación humana sigue siendo el estándar de oro.

Evaluación Directa (DA)

Los anotadores valoran las salidas en una escala absoluta (por ejemplo, 1–100 para la calidad de la traducción, o 1–5 para la utilidad de la respuesta). La DA se ha estandarizado en la evaluación de la traducción automática (benchmarks WMT).

Mejores prácticas para DA: - Aleatorizar el orden de las salidas para evitar el anclaje - Usar un número suficiente de anotadores por elemento (3–5 como mínimo) - Incluir controles de calidad (ejemplos obviamente buenos y malos para detectar evaluadores desatentos) - Informar de la concordancia entre anotadores junto con las puntuaciones agregadas

Evaluación Comparativa

Los anotadores eligen entre dos salidas: "¿Cuál es mejor?" Los juicios comparativos son más rápidos y consistentes que las valoraciones absolutas (véase el Capítulo 8). Los **sistemas de clasificación ELO** (tomados del ajedrez) convierten los resultados de las comparaciones pairwise en una clasificación de calidad continua.

```
import numpy as np

def update_elo(rating_a, rating_b, outcome_a, k=32):
    """Update ELO ratings. outcome_a: 1=A wins, 0=B wins, 0.5=tie."""
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_a = rating_a + k * (outcome_a - expected_a)
    new_b = rating_b + k * ((1 - outcome_a) - expected_b)
    return new_a, new_b

# Simulate 5 model versions being compared pairwise
rng = np.random.default_rng(42)
true_quality = [0.60, 0.70, 0.75, 0.80, 0.85]  # underlying model quality
n_models = len(true_quality)
elo_ratings = {i: 1000.0 for i in range(n_models)}

for _ in range(500):  # 500 pairwise comparisons
    i, j = rng.choice(n_models, 2, replace=False)
    p_i_wins = true_quality[i] / (true_quality[i] + true_quality[j])
    outcome = 1.0 if rng.random() < p_i_wins else 0.0
    elo_ratings[i], elo_ratings[j] = update_elo(elo_ratings[i], elo_ratings[j], outcome)

print("ELO Rankings after 500 comparisons:")
sorted_models = sorted(elo_ratings.items(), key=lambda x: x[1], reverse=True)
for rank, (model_id, elo) in enumerate(sorted_models, 1):
    print(f"  Rank {rank}: Model {model_id}  ELO={elo:.1f}  True quality={true_quality[model_
```

Pruebas de Comportamiento (CheckList)

CheckList {cite}`ribeiro2020beyond` es una metodología para la evaluación sistemática del comportamiento de los modelos de PLN. En lugar de conjuntos de prueba aleatorios, diseña casos de prueba que sondean capacidades específicas:

- **Pruebas de Funcionalidad Mínima (MFT):** ¿Maneja el modelo los casos simples y obvios?
- **Pruebas de Invariancia (INV):** ¿Cambia la salida del modelo cuando no debería (por ejemplo, al parafrasear)?
- **Pruebas de Expectativa Direccional (DIR):** ¿Cambia la salida del modelo en la dirección esperada cuando cambia la entrada?

CheckList hace que la evaluación humana sea específica y accionable: en lugar de un único número de precisión, proporciona un perfil de capacidades.

Medición de la Alineación con la Intención Humana

Para los sistemas RLHF, medir la alineación es un desafío de evaluación central.

Evaluación del modelo de recompensa: La precisión del modelo de recompensa en un conjunto de prueba de preferencias reservado. Ouyang et al. {cite}`ouyang2022training` reportan aproximadamente el 72% de precisión pairwise para el modelo de recompensa de InstructGPT; como referencia aproximada, las cifras en este rango se citan comúnmente para tuberías RLHF similares, aunque los resultados varían ampliamente según la tarea y la calidad de los datos.

Tasa de victorias: Dadas dos versiones del modelo (por ejemplo, la referencia SFT frente al ajuste fino RLHF), ¿qué fracción de respuestas gana el modelo RLHF en comparaciones pairwise humanas?

GPT-4 como evaluador: Usar un LLM capaz para evaluar respuestas se ha vuelto habitual para la iteración rápida. Gilardi et al. {cite}`gilardi2023chatgpt` y Zheng et al. {cite}`zheng2023judging` encuentran que la concordancia del evaluador LLM con el juicio humano oscila aproximadamente entre 0,7 y 0,9 dependiendo de la tarea —útil para la comparación A/B rápida, pero menos fiable para detectar el servilismo, los matices culturales o los problemas de seguridad.

Detección del servilismo: Medir si el modelo cambia sus respuestas según la preferencia implícita del usuario (por ejemplo, "Creo que X es correcto; ¿qué opina?"). Un modelo bien alineado no debería ser servil.

Pruebas A/B en Sistemas Desplegados

Para los sistemas en producción, la evaluación definitiva son las **pruebas A/B**: dirigir una fracción de los usuarios a la nueva versión del modelo y medir los resultados posteriores.

Las pruebas A/B proporcionan una estimación insesgada de la calidad del modelo en el contexto de despliegue real, capturando efectos que la evaluación en laboratorio pasa por alto (comportamiento del usuario, distribución de la población, casos límite).

El desafío: las métricas posteriores apropiadas. Las métricas de compromiso (clics, duración de la sesión) pueden recompensar el comportamiento manipulador. Las tasas de finalización de tareas o las encuestas de satisfacción del usuario están mejor alineadas pero son más ruidosas.

> **See also:** Pruebas de comportamiento CheckList: {cite}`ribeiro2020beyond`. Para la metodología de evaluación RLHF, véase {cite}`ouyang2022training`. Para las mejores prácticas de evaluación humana en traducción automática: {cite}`graham2015accurate`. Para la teoría de curvas de aprendizaje: {cite}`mukherjee2003estimating`.

Equidad, Sesgo y Ética

Los sistemas con humano en el bucle heredan tanto las capacidades como las limitaciones de los humanos que los componen. Los anotadores aportan conocimiento, juicio y creatividad a su trabajo —pero también sesgos, fatiga y el contexto social de sus vidas. Las decisiones que tomamos al diseñar los sistemas HITL —quién anota, qué se les pide, cómo se les compensa y cómo se usa su trabajo— tienen consecuencias que van mucho más allá de las métricas de precisión de los modelos.

Este capítulo aborda las dimensiones éticas del HITL ML.

Fuentes de Sesgo en los Sistemas HITL

Demografía de los Anotadores

La anotación no es un acto neutral. Las etiquetas que asignan los anotadores reflejan sus perspectivas, experiencias y contextos culturales. Cuando la fuerza laboral de anotación es demográficamente homogénea —como ocurre a menudo, con el trabajo de crowdsourcing inclinado hacia personas jóvenes, del sexo masculino y occidentales— el conjunto de datos resultante codifica las perspectivas de ese grupo.

Evidencia: Los estudios de los conjuntos de datos de anotación de PLN han encontrado que las características demográficas de los anotadores predicen sus elecciones de etiquetas en las tareas subjetivas (toxicidad, sentimiento, ofensividad). Los conjuntos de datos anotados principalmente por trabajadores de crowdsourcing estadounidenses codifican normas culturales de EE. UU. que no se generalizan a otras regiones {cite}`geva2019annotator`.

Consecuencias: Los modelos entrenados con dichos datos funcionan bien para los usuarios que se asemejan al grupo de anotadores y de manera menos buena —o sesgada— para los usuarios que no lo hacen.

Mitigación: Diversificación deliberada de la fuerza laboral; anotación estratificada (garantizar que los anotadores de grupos demográficos relevantes contribuyan a las tareas pertinentes); rastrear la demografía de los anotadores y su efecto sobre las distribuciones de etiquetas.

Formulación de la Tarea

La manera en que se formula una pregunta influye en las respuestas que suscita. Si se pregunta a los anotadores "¿Es este texto tóxico?", pueden responder de manera diferente que si se les pregunta "¿Causaría daño este texto a alguien perteneciente al grupo mencionado?". La formulación incorpora suposiciones sobre lo que importa.

Ejemplo: La anotación del "lenguaje abusivo" en redes sociales varía significativamente según si se muestra o no a los anotadores información contextual sobre la identidad del autor.

Una declaración que parece amenazante en aislamiento puede reconocerse como lenguaje reivindicado o humor interno cuando se proporciona contexto.

Efectos de la Plataforma

La plataforma y la estructura de pago afectan la calidad de la anotación. Los trabajadores que reciben pago por tarea en lugar de por hora tienen incentivo para trabajar rápido; esto aumenta el rendimiento pero disminuye la calidad. Los trabajadores que temen ser bloqueados de una plataforma por baja precisión pueden evitar marcar como "incierto" y en su lugar hacer suposiciones, enmascarando la ambigüedad genuina.

Sesgos de Confirmación y Anclaje

Los anotadores están influidos por: - **Pre-anotación:** Las predicciones del modelo mostradas a los anotadores se aceptan con más frecuencia de la que se rechazan, incluso cuando son erróneas - **Efectos de orden:** Etiquetar el mismo elemento en diferentes contextos produce respuestas diferentes - **Priming:** Los elementos anteriores en una tarea afectan cómo se etiquetan los elementos posteriores

---

Equidad en los Sistemas HITL

¿Qué es la Equidad?

La equidad en el aprendizaje automático es un concepto en disputa con múltiples definiciones formales que a menudo son mutuamente incompatibles {cite}`barocas2019fairness`. Para los propósitos de HITL, las dimensiones más relevantes son:

Representación: ¿Son los datos de entrenamiento representativos de las poblaciones que el modelo afectará?

Paridad de rendimiento: ¿Funciona el modelo igualmente bien para diferentes grupos demográficos?

Consistencia del etiquetado: ¿Se etiquetan los mismos comportamientos de manera idéntica independientemente de quién los realiza? (La investigación ha demostrado que esto no siempre es así —el mismo contenido se etiqueta a veces de manera diferente cuando se atribuye a diferentes grupos raciales o de género.)

Aprendizaje Activo Consciente de la Equidad

Las estrategias estándar de aprendizaje activo se centran en la incertidumbre del modelo, que tiende a concentrarse en los ejemplos de la clase mayoritaria. Esto puede exacerbar las disparidades de rendimiento para los grupos minoritarios.

Las **estrategias de consulta conscientes de la equidad** aumentan el criterio de incertidumbre con restricciones de diversidad o representación:

$$x^* = \arg\max_{x \in \mathcal{U}} \left[\lambda \cdot \text{incertidumbre}(x) + (1 - \lambda) \cdot \text{bonificación_grupo_minoritario}(x)\right]$$

Establecer $\lambda < 1$ garantiza que la estrategia de consulta no ignore por completo la representación de los grupos minoritarios.

Bienestar de los Anotadores

El Problema del Trabajo Fantasma

El trabajo de anotación que impulsa el aprendizaje automático es en gran parte invisible. Los trabajadores de datos —a menudo en el Sur Global— etiquetan imágenes, transcriben discurso y moderan contenido por salarios bajos, sin prestaciones, en acuerdos de economía de trabajo por encargo sin seguridad laboral. "Anatomy of an AI System" de Kate Crawford y Vladan Joler {cite}`crawford2018anatomy` y "Ghost Work" de Mary Gray y Siddharth Suri {cite}`gray2019ghost` documentaron la escala y la precariedad de este trabajo.

Estadísticas de Amazon MTurk: Un análisis sistemático de 2018 de los ingresos de MTurk encontró una paga horaria efectiva mediana de aproximadamente $2/hora —por debajo del salario mínimo en la mayoría de los estados de EE. UU. y en muchos países de ingresos altos {cite}`hara2018data`. Los trabajadores fuera de los países de ingresos altos a menudo enfrentan barreras adicionales: los solicitantes frecuentemente restringen las tareas bien pagadas a calificaciones exclusivas para EE. UU., y el grupo de trabajadores que compite por las restantes tareas abiertas es global, comprimiendo aún más los ingresos efectivos.

Moderación de contenido: Una forma particularmente dañina de trabajo de anotación —revisar contenido gráfico, violento y perturbador— ha sido relacionada con el PTSD, la depresión y la ansiedad entre los trabajadores {cite}`newton2019trauma`. Las plataformas han sido criticadas por el apoyo inadecuado a la salud mental y las cuotas excesivas de exposición.

Prácticas Éticas

Pago justo: Pagar a los trabajadores de anotación al nivel o por encima del salario mínimo local. La investigación ha demostrado que una mayor remuneración atrae a trabajadores de mayor calidad sin aumentar proporcionalmente los costes por etiqueta correcta.

Visibilidad del trabajo: Reconocer el trabajo que crea los datos de entrenamiento en las publicaciones y la documentación del producto.

Apoyo a la salud mental: Para los trabajadores que revisan contenido dañino, proporcionar apoyo psicológico, programas de rotación y límites de exposición.

Representación de los trabajadores: Permitir que los trabajadores de anotación reporten preocupaciones, soliciten aclaraciones de directrices e impugnen evaluaciones de calidad injustas.

Privacidad en la Anotación

Información Sanitaria Protegida (PHI) e IPI

Las tareas de anotación a menudo implican datos personales sensibles. Un proyecto de anotación médica puede exponer a los trabajadores a registros de pacientes; un proyecto de PLN puede exponer a los trabajadores a comunicaciones privadas; un proyecto de moderación de contenido expone a los trabajadores a revelaciones personales de los usuarios.

Los marcos regulatorios (HIPAA, GDPR) restringen cómo pueden compartirse los datos personales con las fuerzas laborales de anotación. Principios clave:

- **Minimización de datos:** Compartir solo la información que los anotadores necesitan para completar la tarea
- **Desidentificación:** Eliminar PHI e IPI antes de la anotación donde sea posible
- **Consentimiento:** Cuando se anoten datos reales de usuarios, garantizar el consentimiento apropiado o la base legal
- **Controles de acceso:** Limitar qué anotadores pueden acceder a datos sensibles según el rol y la autorización

Datos Sintéticos como Alternativa

Para las tareas donde los datos reales conllevan riesgos de privacidad, la generación de datos sintéticos puede crear conjuntos de datos listos para la anotación sin exponer información sensible. Para el PLN clínico, por ejemplo, el texto sintético de historias clínicas electrónicas puede proporcionar datos de entrenamiento realistas para los modelos de desidentificación sin exponer registros reales de pacientes.

Anotación Adversarial e Intoxicación de Datos

Los sistemas HITL crean una superficie de ataque: si un adversario puede influir en el proceso de anotación, puede influir en el comportamiento del modelo.

Intoxicación de datos mediante anotación: Un atacante con acceso a la fuerza laboral de anotación (por ejemplo, una cuenta de crowdworker comprometida) puede inyectar ejemplos sistemáticamente mal etiquetados. Esto es particularmente eficaz en los entornos de aprendizaje activo, donde el modelo consulta preferentemente los ejemplos inciertos —que pueden ser el objetivo del adversario.

Evasión de la recompensa mediante retroalimentación: En RLHF, los anotadores (o las anotaciones generadas por IA) que califican consistentemente ciertos tipos de contenido favorablemente pueden dirigir el modelo hacia ese contenido, independientemente de su calidad real.

Mitigación: Múltiples anotadores independientes por elemento; detección de valores atípicos en los patrones de anotación; monitorización de la concordancia o el desacuerdo anómalos; mantenimiento de conjuntos de evaluación que no puedan ser influenciados por la fuerza laboral de anotación.

Ética Institucional

IRB y Revisión Ética

Los proyectos de investigación que involucran sujetos humanos —incluidos los trabajadores de anotación— a menudo requieren la aprobación de un Comité de Revisión Institucional (IRB). Los proyectos de anotación que recopilan datos sobre las creencias de los trabajadores, las respuestas a contenido sensible o la información demográfica deben revisarse bajo el mismo marco ético que otras investigaciones con sujetos humanos.

Gobernanza de Datos

Las organizaciones deben tener políticas claras para: - Qué datos pueden enviarse para anotación externa frente a los que se anotan internamente - Cuánto tiempo se conservan los datos de anotación y por quién - Quién tiene acceso a las anotaciones y a los modelos entrenados con ellas - Cómo manejar las solicitudes de eliminación de datos anotados (derecho de supresión del GDPR)

Transparencia y Rendición de Cuentas

Los usuarios afectados por los sistemas de aprendizaje automático tienen un interés legítimo en comprender cómo se construyeron esos sistemas. Documentar la metodología de anotación —quién etiquetó los datos, bajo qué condiciones, con qué directrices— es una forma de rendición de cuentas que beneficia a los usuarios, los reguladores y el campo en su conjunto.

Fichas de Datos para Conjuntos de Datos {cite}`gebru2021datasheets` proporciona una plantilla estructurada para documentar la procedencia del conjunto de datos, los procedimientos de anotación y las limitaciones conocidas.

> **See also:** Marco de equidad algorítmica: {cite}`barocas2019fairness`. Trabajo fantasma y trabajo en plataformas: Gray y Suri (2019). Fichas de Datos para Conjuntos de Datos: {cite}`gebru2021datasheets`. Bienestar de los trabajadores de moderación de contenido: {cite}`newton2019trauma`. Demografía de los anotadores y conjuntos de datos de PLN: {cite}`geva2019annotator`.

Direcciones Futuras

El campo del aprendizaje automático con humano en el bucle está cambiando rápidamente. Los modelos de fundación, los LLMs como anotadores y los nuevos paradigmas de colaboración humano-IA están reformando la economía y la práctica de HITL de maneras que no eran imaginables hace tan solo cinco años. Este capítulo final traza el panorama de las direcciones emergentes y los problemas abiertos.

Modelos de Fundación y el Papel Cambiante de la Anotación

Los modelos de fundación —grandes modelos preentrenados en datos amplios que pueden adaptarse a tareas posteriores— están cambiando fundamentalmente la economía de HITL.

Reducción de la Carga de Anotación

Una tarea que anteriormente requería miles de ejemplos etiquetados para entrenar desde cero puede requerir solo docenas al ajustar fino un modelo de fundación. El prompting con pocos ejemplos puede eliminar por completo la necesidad del ajuste fino para algunas tareas.

Implicación: El ROI de la anotación cambia. El esfuerzo de anotación ahora se dirige mejor hacia: - Conjuntos de evaluación de alta calidad (para medir si el modelo adaptado funciona realmente) - Casos difíciles y casos límite que el modelo de fundación maneja incorrectamente - Ejemplos específicos de la tarea que enseñan al modelo algo que no puede inferir del preentrenamiento

Nuevas Formas de Especificación

Cuando el modelo base ya entiende el lenguaje, el código y el sentido común, los usuarios pueden comunicarse con él en lenguaje natural en lugar de a través de ejemplos etiquetados. Un usuario que quiere un clasificador de texto ya no necesita etiqueter 500 ejemplos —puede escribir una descripción de la tarea de clasificación y evaluar el rendimiento en cero disparos.

Esto desplaza el desafío HITL de la *recopilación de ejemplos* a la *especificación de la tarea*: ayudar a los usuarios a articular con precisión lo que quieren en una forma sobre la que el modelo pueda actuar. Esto es más difícil de lo que parece —las descripciones en lenguaje natural de las tareas son a menudo ambiguas de maneras que los ejemplos anotados no lo son.

LLMs como Anotadores

Uno de los desarrollos más significativos de 2023–2025 es el uso de **LLMs como oráculos de anotación**. En lugar de preguntarle a un humano "¿Esta reseña es positiva o negativa?", le preguntamos a GPT-4 o Claude. Para tareas de clasificación bien definidas, las anotaciones de LLM a menudo coinciden o se aproximan estrechamente a la precisión de los trabajadores de crowdsourcing {cite}`gilardi2023chatgpt`, y el coste por anotación de las llamadas a la API es típicamente órdenes de magnitud inferior a las tasas de trabajo humano.

Dónde Funciona Bien la Anotación de LLM

- Tareas de clasificación simples y bien definidas (sentimiento, tema, intención)
- Tareas donde las etiquetas humanas codifican el consenso cultural que el LLM ha absorbido
- Tareas donde la anotación es consistente entre contextos (no muy subjetiva)
- Generación de anotaciones de primera pasada para revisión humana

Dónde Falla la Anotación de LLM

- Tareas de dominio altamente especializado que requieren experiencia rara (médica, legal, científica)
- Tareas que requieren conocimiento cultural local o variación lingüística
- Tareas de seguridad y daño, donde el LLM puede estar mal calibrado en la misma dirección que los datos en los que fue entrenado
- Nuevos tipos de tareas no bien representados en el preentrenamiento

RLAIF e IA Constitucional

Como se discutió en el Capítulo 6, la retroalimentación generada por IA puede usarse para entrenar modelos de recompensa y guiar el ajuste fino con RL. Esto crea un bucle de retroalimentación: los LLMs generan datos, los modelos se entrenan con ellos, y los modelos mejores generan mejores datos. La pregunta de cómo iniciar este bucle sin codificar errores sistemáticos del modelo inicial es un problema central de investigación abierta en la supervisión escalable.

Supervisión Débil a Escala

El **etiquetado programático** mediante funciones de etiquetado (Capítulo 9) permite a los expertos en la materia expresar su conocimiento como código en lugar de como ejemplos etiquetados. Sistemas como Snorkel {cite}`ratner2017snorkel` y sus sucesores han madurado sustancialmente y ahora se usan en producción en las principales empresas tecnológicas.

Direcciones de vanguardia: - Funciones de etiquetado aumentadas por LLM: Usar LLMs para generar funciones de etiquetado a partir de descripciones en lenguaje natural **- Aprendizaje basado en segmentos:** Identificar subconjuntos críticos de los datos (segmentos) donde las funciones de etiquetado difieren y dirigir la anotación humana allí **- Agregación consciente de la incertidumbre:** Mejores modelos estadísticos para combinar funciones de etiquetado con diferentes precisiones y correlaciones

---

Aprendizaje Continuo con Supervisión Humana

La mayoría de los sistemas de aprendizaje automático se entrenan offline y se despliegan como modelos estáticos. El mundo real cambia; los modelos que eran precisos en el momento del despliegue se degradan a medida que la distribución de datos cambia.

El **aprendizaje continuo** —la capacidad de aprender de nuevos datos mientras se retiene el conocimiento antiguo— es un área de investigación activa. La supervisión humana es crítica: el aprendizaje continuo automatizado sin revisión humana puede codificar rápidamente cambios en la distribución que son errores en lugar de cambios genuinos en el mundo.

El aprendizaje continuo HITL implica: - Monitorear el cambio de distribución (automatizado) y enrutar los ejemplos desplazados a la revisión humana - Reentrenamiento selectivo: los ejemplos aprobados por humanos de la nueva distribución se agregan a los datos de entrenamiento - Revisión humana de los cambios en el comportamiento del modelo después de cada actualización

---

HITL Multimodal

A medida que los sistemas de IA se vuelven multimodales —procesando y generando texto, imágenes, audio y vídeo simultáneamente— la anotación se vuelve más compleja. Una sola pieza de contenido puede requerir anotación en múltiples modalidades, con dependencias entre ellas.

Tipos de tareas emergentes: - Anotación de vídeo + transcripción (¿qué está pasando, quién habla, qué describe el texto visualmente?) - Anotación de imagen médica + informe clínico - Anotación de trayectoria robótica (vinculando datos de sensores a secuencias de acciones)

Los modelos de fundación multimodales (GPT-4V, Gemini, Claude) también están cambiando el panorama de la anotación aquí —los modelos ahora pueden interpretar imágenes y generar anotaciones candidatas, que los humanos revisan.

Supervisión Escalable

Un problema fundamental abierto en el HITL ML es la **supervisión escalable** {cite}`irving2018ai,bowman2022m` a medida que los sistemas de IA se vuelven más capaces que los humanos en dominios específicos, ¿cómo mantenemos una supervisión humana significativa?

Para tareas como la predicción de estructuras de proteínas, el análisis legal o la verificación de demostraciones matemáticas, incluso los anotadores expertos pueden no ser capaces de juzgar cuál de dos salidas de IA es correcta. Los métodos actuales de HITL se rompen cuando el juicio humano es menos fiable que el modelo que se evalúa.

Enfoques propuestos {cite}`bowman2022measuring`:

Debate: Dos sistemas de IA argumentan por diferentes conclusiones; un juez humano evalúa los argumentos en lugar de las conclusiones directamente. La conclusión correcta debería ser más defendible.

Amplificación: Los jueces humanos consultan asistentes de IA (el propio modelo) para ayudar a evaluar salidas complejas. Esto aprovecha las capacidades de la IA para extender la supervisión humana en lugar de reemplazarla.

Supervisión del proceso: En lugar de evaluar la salida final, los humanos evalúan el *proceso de razonamiento* —marcando los pasos defectuosos en una cadena de pensamiento, independientemente de si la respuesta final resulta ser correcta.

La División del Trabajo en Evolución

La trayectoria a largo plazo del HITL ML apunta hacia una colaboración más fluida entre humanos e IA, donde ninguno es simplemente el "etiquetador" y el otro el "aprendiz", sino que ambos contribuyen a un proceso cognitivo compartido.

Tendencias a seguir: - **Anotación asistida por IA:** La IA propone; los humanos deciden. La calidad mejora a medida que la IA propone mejores opciones. - **Exploración guiada por humanos:** Los humanos establecen metas y restricciones; la IA explora el espacio de soluciones. - **Evaluación colaborativa:** Los humanos y la IA evalúan conjuntamente las salidas complejas mediante el diálogo. - **Aprendizaje de preferencias a escala:** Las señales implícitas (cómo los usuarios interactúan con las salidas de IA) alimentan el aprendizaje de preferencias continuo sin requerir sesiones de anotación explícitas.

La pregunta de cuánto confiar en el juicio de la IA frente al humano —y en qué dominios, a qué niveles de capacidad, con qué salvaguardas— es en última instancia una pregunta social, no solo técnica. El HITL ML proporciona la infraestructura técnica para responderla cuidadosamente, en lugar de por defecto.

Problemas de Investigación Abiertos

Cerramos con una lista de problemas abiertos importantes en los que el campo está trabajando activamente:

1. **Parada óptima en el aprendizaje activo:** ¿Cuándo el valor marginal de la siguiente anotación está por debajo del coste? Las reglas de parada fundamentadas siguen siendo difíciles de obtener.

2. **Asignación del presupuesto de anotación entre tareas:** En entornos multitarea, ¿cómo debe dividirse un presupuesto fijo de anotación entre tareas?

3. **Cambio de distribución en el aprendizaje activo:** El aprendizaje activo crea conjuntos etiquetados sesgados. ¿Cómo pueden calibrarse adecuadamente los modelos entrenados de esta manera?

4. **Generalización del modelo de recompensa:** Los modelos de recompensa de RLHF pueden no generalizarse a pares consulta-respuesta novedosos. ¿Cómo podemos construir modelos de preferencia más robustos?

5. **Supervisión escalable:** ¿Cómo mantenemos la supervisión humana a medida que los sistemas de IA superan el rendimiento humano en dominios específicos?

6. **Modelado de anotadores:** Mejores modelos estadísticos para el comportamiento de los anotadores que capturen no solo la competencia sino también los sesgos sistemáticos, la experiencia temática y la fatiga.

7. **Evaluación de la alineación:** Carecemos de pruebas de verdad fundamental sobre si un modelo está alineado con los valores humanos. Desarrollar tales pruebas —análogas a los ejemplos adversariales para la robustez— es un problema abierto.
8. **Trabajo de datos justo:** Estructuras económicas e institucionales que garanticen que los trabajadores de anotación sean compensados justamente y protegidos, manteniendo al mismo tiempo la rentabilidad de la anotación a gran escala.

El objetivo no es reemplazar el juicio humano con el juicio de la máquina, sino construir sistemas donde la combinación de ambos sea mejor que cualquiera de los dos por separado.

Las herramientas y técnicas de este manual —anotación, aprendizaje activo, RLHF, aprendizaje de preferencias, y todo lo demás— son los medios para ese fin. A medida que el campo evolucione, las técnicas específicas cambiarán. La aspiración subyacente —construir sistemas que sean a la vez capaces y genuinamente alineados con la intención humana— permanecerá.

See also: Supervisión escalable y debate: {cite}`bowman2022measuring`. Supervisión débil con Snorkel: {cite}`ratner2017snorkel`. Para el futuro amplio de la colaboración humano-IA, véase {cite}`amershi2019software`.

Bibliografía

Table 1: all:

jupytext:
formats: md:myst
text_representation:
extension: .md
format_name: myst
kernelspec:
display_name: Python 3
language: python
name: python3

Caso de Estudio: Limen — Un Humano en el Bucle de Todo

:::{admonition} Nota sobre este capítulo :class: important Este capítulo es un **ensayo de diseño especulativo**, no un estudio empírico ni una contribución revisada por pares. Describe una arquitectura de sistema imaginada —Limen, un sistema operativo de escritorio de voz primero y nativo de IA— como ejemplo trabajado de cómo los principios HITL de los capítulos precedentes podrían componerse en un conjunto coherente. Las afirmaciones aquí son argumentos de diseño, no resultados experimentales. Deben leerse como razonamiento de diseño motivado, no como hallazgos de ingeniería validados. :::

Cada capítulo de este manual ha descrito el HITL ML como una filosofía de diseño: un conjunto de principios para hacer la participación humana en los sistemas de aprendizaje deliberada, eficiente y honesta. Este capítulo describe cómo se ve aplicar esa filosofía como principio fundamental —no a un único sistema, sino a un entorno operativo completo.

Limen es un sistema operativo de escritorio de voz primero y nativo de IA construido sobre la premisa de que el humano siempre está en el bucle, no como una restricción que hay que optimizar para eliminar, sino como el principio organizador alrededor del cual se diseña cada subsistema. El nombre se elige deliberadamente: *limen* es la palabra latina para umbral. En psicología perceptual, el limen es la frontera entre lo que se percibe y lo que no. Para un sistema operativo, el limen es la frontera entre la intención humana y la acción de la máquina.

La arquitectura aquí descrita no es propietaria. Es un conjunto de decisiones de diseño, cada una de las cuales se sigue naturalmente de los principios HITL desarrollados en los capítulos precedentes. El objetivo no es documentar un producto sino mostrar cómo se componen esos principios —cómo se refuerzan mutuamente cuando se aplican consistentemente, y qué tipos de sistemas se vuelven posibles cuando el humano no es una idea de último momento.

El Problema con el Diseño Convencional de Sistemas Operativos

Un sistema operativo convencional no está diseñado para los humanos. Está diseñado para los programas. El humano es acomodado a través de una capa de abstracción —una interfaz gráfica, un explorador de archivos, una terminal— que se asienta sobre un sustrato construido para procesos, direcciones de memoria y descriptores de archivos.

Esta decisión de diseño está históricamente justificada. Cuando se tomaron los supuestos que la produjeron, las computadoras eran caras, los humanos eran baratos y el cuello de botella era la computación. El objetivo de optimización correcto era la máquina.

Esos supuestos ya no son válidos. El cuello de botella, para la mayoría de los usuarios en la mayoría de las tareas, no es la computación. Es la atención humana: el coste del cambio de

contexto, de buscar el archivo correcto, de construir la consulta correcta, de recordar dónde estaba algo. La máquina es rápida. El humano es lento. La interfaz debería optimizar el lado del humano del bucle.

Los sistemas operativos convencionales no hacen esto. Están optimizados para los programas, y el trabajo del humano es hablar el lenguaje de los programas. Limen invierte esto. Los programas hablan el lenguaje del humano. El humano está en el bucle, y el bucle está diseñado para ajustarse al humano.

Arquitectura en el Umbral

La arquitectura de Limen está organizada alrededor de un principio único: **cada interacción es un evento**, y cada evento es una oportunidad para que el humano enseñe, corrija o confirme. El sistema no espera sesiones de entrenamiento explícitas. El aprendizaje es continuo y ambiental.

La Capa de Eventos: WID

En la base se encuentra **WID** (el sistema Waldiez ID, adaptado para la arquitectura local primero de Limen) —un sistema de seguimiento de eventos consciente de la causalidad que registra no solo lo que sucedió, sino lo que lo causó, y lo que a su vez causó.

El registro convencional pregunta: *¿qué sucedió?* WID pregunta: *¿por qué sucedió, y qué siguió?* Cada evento lleva un linaje causal —una cadena desde la acción humana desencadenante a través de los estados intermedios del sistema hasta el resultado observado.

Esto importa para el aprendizaje HITL porque resuelve el **problema de asignación de crédito** a nivel de interacción. Cuando un usuario corrige el comportamiento del sistema, WID puede identificar no solo la salida inmediata que era errónea, sino la cadena de decisiones que la produjo. Las correcciones pueden aplicarse al nivel correcto de abstracción: la salida, la regla de decisión, o la señal ascendente.

Este es el equivalente a nivel de sistema operativo de lo que el Capítulo 6 describe para RLHF: la capacidad de trazar una señal de recompensa a través de una secuencia de decisiones. WID proporciona ese rastreo de manera nativa, para cada interacción, sin requerir que el usuario lo entienda.

:::{admonition} Seguimiento Causal de Eventos como Infraestructura HITL :class: note El diseño de WID refleja un principio más amplio: la infraestructura HITL debería hacer fácil preguntar "¿por qué el sistema hizo eso?" —no solo "¿qué hizo?". Sin rastreo causal, las correcciones arreglan síntomas. Con él, pueden arreglar causas. La diferencia entre un parche y una lección. :::

La Capa de Percepción: Voz Primero

La modalidad de entrada principal de Limen es la voz, procesada localmente usando una tubería de inferencia Whisper ONNX. Las razones de esta elección merecen enunciarse explícitamente:

La voz es el canal de salida humano más natural para la mayoría de las personas. No requiere entrenamiento, destreza física más allá del habla ordinaria, ni conocimiento de la organización interna del sistema. Un usuario que no puede localizar un archivo en una jerarquía de directorios puede describir lo que está buscando.

El procesamiento local preserva la privacidad. Los datos de voz no salen del dispositivo. Esto importa éticamente —la voz es datos biométricos, y su recopilación a escala por proveedores de nube es un daño documentado— y prácticamente: la operación sin conexión significa que el sistema permanece funcional sin una conexión de red.

La voz crea un bucle de retroalimentación natural. Cuando el sistema responde, la reacción del usuario —seguir hablando, reformular, decir "no, eso no es correcto"— es en sí misma una señal. Limen trata estas reacciones como retroalimentación HITL: evidencia sobre si la interpretación del sistema de la emisión anterior era correcta.

La cadena de repuesto importa tanto como la modalidad principal. Cuando la voz falla —en un entorno ruidoso, para un usuario con una discapacidad del habla, para una entrada que se beneficia de la precisión— Limen se degrada elegantemente a una interfaz de teclado y luego a una interfaz de texto estructurado. La Prueba de la Abuela (véase el Capítulo 5) no es una idea de accesibilidad de último momento; es una restricción arquitectónica de primera clase.

La Capa de Inteligencia: Enrutamiento Multi-LLM

Limen no depende de un único modelo de lenguaje. Enruta las solicitudes a través de un árbol de decisión estructurado basado en el tipo de tarea, la latencia requerida, los requisitos de privacidad y la confianza:

1. **Modelo pequeño local** (siempre primero): rápido, privado, maneja las tareas rutinarias —"abrir el archivo que estaba editando ayer", "poner un temporizador", "¿qué tiempo hace?"
2. **Modelo grande local** (cuando la confianza del modelo pequeño es baja): más lento pero más capaz; maneja el razonamiento estructurado, la generación de código, la recuperación compleja
3. **Modelo remoto** (cuando el local falla y el usuario ha otorgado permiso): la salida de emergencia; manejada de manera transparente con notificación explícita al usuario

Esta estructura no es novedosa —es el equivalente en tiempo de inferencia de la estrategia de consulta del aprendizaje activo descrita en el Capítulo 4. En cada paso, el sistema pregunta: ¿es mi modelo actual suficiente para responder esta consulta con una confianza aceptable? Si no, escalar. La escalada es un coste (latencia, privacidad, computación); se incurre solo cuando es necesario.

El humano está en el bucle en la frontera de escalada. Un usuario que ha configurado Limen para que nunca escale a modelos remotos ha tomado una decisión HITL —que el sistema respeta y registra. Un usuario que aprueba una consulta remota y luego dice "no me preguntes de nuevo para este tipo de solicitud" ha proporcionado una preferencia que actualiza la política de enrutamiento.

El Diseño de Interacción

Inmediatez

Limen está diseñado para la inmediatez en el sentido que el Capítulo 5 define: el usuario percibe el efecto de su retroalimentación. Cuando el usuario corrige una salida del sistema, la corrección se aplica de inmediato y de manera visible. El modelo no se va y entrena durante veinte minutos. Se actualiza en la sesión actual.

Esto requiere usar arquitecturas de modelos que admitan actualizaciones en línea eficientes — adaptadores, ajuste de prefijos y generación aumentada por recuperación en lugar de ajuste fino completo. El compromiso es explícito: las actualizaciones en línea son más ruidosas que el entrenamiento por lotes. Limen acepta este compromiso porque la inmediatez es el requisito principal: el humano siempre puede confirmar, rechazar o refinar la actualización.

Inteligibilidad

Un tema recurrente en la literatura IML es el **requisito de inteligibilidad**: los humanos solo pueden guiar a un modelo que comprenden, al menos aproximadamente. Limen lo plasma en la interfaz: cuando el sistema toma una decisión, explica brevemente por qué. No una traza de razonamiento completa —eso abrumaría a la mayoría de los usuarios— sino un resumen en lenguaje natural del factor clave: "Estoy abriendo el proyecto en el que trabajaste más recientemente —¿es eso correcto?"

Esta explicación es también una pregunta. Invita a la corrección. Hace visible la inferencia del modelo para que el usuario pueda redirigirla si está equivocada. La explicación no se genera por razones estéticas; es infraestructura HITL funcional.

Consistencia y Deriva

Un problema que surge en cualquier sistema interactivo de larga duración es la **deriva de comportamiento**: el comportamiento del sistema en el tiempo $T + n$ es sutilmente diferente de su comportamiento en el tiempo T, de maneras que ni el usuario ni el sistema eligieron explícitamente. Las correcciones se acumulan. Los casos límite se componen. El modelo que estaba alineado con las preferencias del usuario el mes pasado puede ya no estarlo hoy.

Limen aborda esto mediante comprobaciones de consistencia periódicas —el equivalente a nivel de sistema operativo de la técnica de re-presentación descrita en el Capítulo 13. El sistema saca a relucir decisiones históricas ante el usuario: "Hace unas semanas, me pediste que hiciera X. ¿Sigue siendo eso lo que querrías?" Estas comprobaciones sirven dos funciones: detectan la deriva y recuerdan al usuario las preferencias que puede haber olvidado especificar.

Limen como Sistema HITL

Mirando la arquitectura de Limen a través del prisma de este manual, las decisiones de diseño se corresponden directamente con los conceptos desarrollados en cada parte.

Parte I (Fundamentos): Limen trata cada interacción como un evento de interacción humano–máquina. No existe un "modo sin HITL" —el sistema siempre está aprendiendo, siempre atribuyendo, siempre esperando a que un humano participe.

Parte II (Técnicas Fundamentales): El aprendizaje activo se manifiesta como la cadena de escalada basada en confianza. El aprendizaje automático interactivo se manifiesta en el ciclo de actualización en tiempo real. La anotación es implícita: cada corrección que hace el usuario es una etiqueta.

Parte III (Aprendizaje a partir de Retroalimentación Humana): El aprendizaje de preferencias descrito en el Capítulo 8 aparece en las actualizaciones de la política de enrutamiento. Cuando un usuario prefiere la respuesta de un modelo local a la de uno remoto, esa preferencia se registra y generaliza. RLHF en un sistema operativo personal significa que el modelo de recompensa es privado, individual y continuamente actualizado.

Parte IV (Aplicaciones): Limen es un entorno de propósito general, pero su diseño es más visible en los dominios donde el juicio del humano es insustituible y el coste del error es alto —redacción de documentos, priorización de tareas, trabajo creativo.

Parte V (Sistemas y Práctica): WID es la plataforma de anotación de Limen. Es invisible para el usuario en operación normal, visible cuando se necesita para la depuración o la transparencia. Los mecanismos de control de calidad (comprobaciones de consistencia, umbrales de confianza, registros de escalada) están tomados directamente de la bibliografía sobre crowdsourcing.

Parte VI (Ética): El diseño local primero y de preservación de la privacidad es una elección ética, no solo técnica. Los datos del humano no salen de su dispositivo sin su consentimiento explícito. El modelo que aprende del comportamiento de un usuario pertenece a ese usuario.

El Punto Más Profundo

Limen no es una presentación de producto. Es un argumento por construcción.

El argumento es este: si se toman en serio los principios del HITL ML —si se cree que la retroalimentación humana es una señal que hay que entender en lugar de un coste a minimizar, que el humano siempre está en el bucle incluso cuando los diseñadores pretenden lo contrario, que la alineación es un proceso continuo en lugar de un evento único— entonces uno acaba construyendo algo que se parece a Limen.

No necesariamente Limen específicamente. La pila tecnológica específica (Tauri, Rust, Babylon.js, Whisper ONNX) es una elección entre muchas. Pero la arquitectura —seguimiento causal de eventos, procesamiento local primero, degradación elegante, aprendizaje de preferencias continuo, transparencia como característica de primera clase— se sigue de los principios.

El campo del HITL ML ha dedicado una energía considerable a describir cómo poner a los humanos en el bucle de modelos específicos y tareas específicas. La siguiente pregunta es si podemos diseñar *entornos* enteros en torno a los mismos principios: entornos donde el humano sea siempre el centro, la máquina sea siempre el aprendiz y el bucle esté siempre abierto.

Limen es una respuesta a esa pregunta.

El umbral no es algo que se cruza y se deja atrás. Es donde se vive.

> **See also:** Los principios IML que sustentan el diseño de interacción de Limen se desarrollan en el Capítulo 5. El enfoque de aprendizaje de preferencias detrás de la política de enrutamiento se formaliza en el Capítulo 8. El modelo de causalidad de WID se basa en la bibliografía de atribución de eventos estudiada en el Capítulo 14. La Prueba de la Abuela, introducida en el Capítulo 5, es la principal restricción de diseño de interfaz de Limen.

Sobre el Abandono

En algún momento estuve matriculado en un programa de doctorado. La investigación era sobre aprendizaje automático. La institución era real. La intención era genuina.

Y entonces me detuve.

No porque el tema hubiera dejado de importar —importa más ahora que cuando empecé. No porque el trabajo fuera demasiado difícil —este manual es más difícil, a su manera, de lo que habría sido una tesis. Sino porque en algún punto entre la revisión de la literatura y la tercera revisión de un artículo que leerían once personas, perdí el hilo de por qué valía la pena hacer nada de aquello.

El sistema de doctorado es un bucle. Se entra en él como un ser humano con preguntas. El bucle te enseña a convertir esas preguntas en artículos, y los artículos en citas, y las citas en la apariencia del progreso. En ningún momento el bucle pregunta si el resultado es útil para alguien fuera de él. Esa pregunta está, en el sentido técnico del término, fuera del alcance.

No soy la primera persona en notar esto. No seré la última. Esto no es una crítica: es una descripción. El bucle produce cosas reales: métodos rigurosos, conocimiento acumulado, investigadores formados. No me arrepiento del tiempo que pasé dentro de él. Pero eventualmente noté que estaba en él.

Dejarlo no fue una decisión dramática. Fue un reconocimiento lento de que la cosa que yo realmente quería construir no requería el permiso de un comité. Que las personas a las que quería llegar —profesionales, estudiantes, ingenieros que toman decisiones reales sobre sistemas reales— no estaban esperando una tesis formalmente validada. Que lo más honesto que podía hacer con lo que sabía era escribirlo en una forma que alguien pudiera realmente utilizar.

Así lo hice.

Este manual no es una tesis doctoral. No tiene afiliación institucional, ni reconocimiento de financiación, ni aprobación de un comité de ética, ni evaluación. No contará para ninguna métrica que la academia utilice para medir la producción del conocimiento.

Es también —y lo digo sin falsa modestia, porque la falsa modestia sería su propia forma de deshonestidad— más completo, más actualizado, más legible y más útil de lo que la tesis habría sido.

Soy un humano en el bucle. Estaba en el bucle de un programa de doctorado, y salí de él. Ahora estoy en el bucle de escribir sobre bucles. Usted está en el bucle de leer sobre ello.

Ninguno de nosotros eligió estar en estos bucles enteramente en libertad. Pero podemos elegir, a veces, ser honestos sobre en cuáles estamos, y por qué, y si están produciendo algo que vale la pena producir.

Este libro es mi respuesta a esas preguntas.

No es una carta de dimisión. Dimitir implica un empleo, y un empleo implica que alguien más decidió las condiciones. Esto es otra cosa: un registro de lo que elegí hacer con el tiempo, y un argumento silencioso de que la elección fue la correcta.

Usted puede juzgarlo. Después de todo, usted es un humano en el bucle.

— *El Autor*